

**MediNet-XG: A LIGHTWEIGHT EXPLAINABLE MULTIMODAL AI
FOR MEDICINAL PLANT FROM WEED-INFESTED AREA**

By

Md Nur A Alam

ID: 210201044

B.Sc. Engineering in Computer Science and Engineering



Department of Computer Science and Engineering

Faculty of Electrical and Computer Engineering

Bangladesh Army University of Science and Technology (BAUST)

JANUARY 2026

**MediNet-X: A Lightweight Explainable Multimodal AI for Medicinal
Plant from weed-infested area.**

This thesis is submitted in partial fulfillment of the requirement for the
degree of B.Sc. Engineering in Computer Science and Engineering

By

Md Nur A Alam

ID: 210201044

Supervised by

Dr. S.M. Jahangir Alam

Professor, Department of CSE

Co-Supervised by

Saad Ahmed

Lecturer, Department of CSE

Department of Computer Science and Engineering

Bangladesh Army University of Science and Technology (BAUST)

Saidpur Cantonment, Nilphamari, Bangladesh

The thesis titled “**MediNet-XG: A Lightweight Explainable Multimodal AI for Medicinal Plant from weed-infested area.**” submitted by Md Nur A Alam (ID:210201044), session 2021-2025 has been presented on 18/01/2026 and accepted as satisfactory in fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering (CSE) as B.Sc. Engineering to be awarded by the Bangladesh Army University of Science and Technology (BAUST).

Board of Examiners

1. _____

Supervisor

Dr. S.M. Jahangir Alam

Professor

Department of Computer Science and Engineering (CSE)

Bangladesh Army University of Science and Technology (BAUST)

2. _____

Co-Supervisor

Saad Ahmed

Lecturer

Department of Computer Science and Engineering (CSE)

Bangladesh Army University of Science and Technology (BAUST)

3. _____ Member
Ashrafun Zannat
Assistant Professor
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)
4. _____ Member
Shifa Tasmiah Tisha
Lecturer
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)
5. _____ Member
Md. Osama Lecturer
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)
6. _____ Member
S. M Golam Rifat
Lecturer
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)

Dr. Md Nakib Hayat Chowdhury
Associate Professor and Head
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)

CANDIDATE'S DECLARATION

It is hereby declared that the contents of this thesis are original and any part of it has not been submitted elsewhere for the award of any degree or diploma.

SN.	Student Name	ID	Signature & Date
1.	Md Nur A Alam	210201044	

ACKNOWLEDGMENTS

First and foremost, we are deeply grateful to Almighty Allah for granting us the strength and perseverance to complete this thesis successfully. We extend our sincerest thanks and heartfelt gratitude to our honorable thesis supervisor, Dr. S.M. Jahangir Alam, Professor of the Department of Computer Science & Engineering (CSE) at Bangladesh Army University of Science & Technology (BAUST), Saidpur. His inspiration and unwavering support have been instrumental in the completion of our thesis.

We like to express our special thanks to the Head of the Department of Computer Science & Engineering (CSE), Dr. Md Nakib Hayat Chowdhury for his valuable suggestions, positive advice, encouragement, and sincere guidance throughout our thesis work. Additionally, we convey our gratitude to all the respected teachers of the Department of CSE, as well as those from other departments, for their support and contributions.

Our sincere thanks go to the respected Honorable Vice Chancellor, the Chief of Army Staff and board of trustees, as well as all members and academic staff who have led this institution and contributed to our bright future.

We would also like to thank all our friends and the staff of the department for their valuable suggestions and assistance. Finally, we extend our deepest appreciation to our parents and families for their unwavering love and support throughout our studies.

ABSTRACT

Bangladesh’s weedy medicinal plants transform neglected lands into sources of traditional medicine, rural income, and biodiversity conservation, yet field identification struggles with inter-class similarity, extreme background clutter, and uncontrolled imaging conditions. Existing deep learning relies on computationally intensive black-box models trained on controlled datasets, limiting interpretability and resource-constrained deployment. We present MediNet-XG, a lightweight (342 KB) multimodal framework featuring inverted residual blocks and depthwise separable convolutions for leaf classification, Grad-CAM spatial reasoning and t-SNE feature analysis for explainability, and retrieval-based query answering delivering grounded botanical knowledge (uses, safety, dosage) with zero hallucination. Evaluated on our new 17,050-image dataset of 34 Bangladeshi weedy medicinal species, MediNet-XG achieves 98.82% accuracy and F1-score 0.9879—outperforming VGG16 (88.95%, 56 MB), DenseNet121 (97.87%, 27 MB), and SqueezeNet (89.33%, 889 KB). The knowledge engine maintains perfect query faithfulness across all samples. MediNet-XG enables farmers and health workers to photograph weedy medicinal plants and instantly receive species identification, medicinal uses, and safety information on smartphones, bridging traditional knowledge and modern AI for rural Bangladesh healthcare and agriculture.

Keywords: Bangladeshi medicinal plants, weed-infested field environments, lightweight convolutional neural networks, explainable artificial intelligence, leaf image classification, knowledge-grounded query answering, edge deployment.

CONTENTS

CANDIDATE’S DECLARATION	iii
ACKNOWLEDGEMENTS	iv
ABSTRACT	v
CONTENTS	vi
LIST OF FIGURES	ix
LIST OF TABLES.....	xi
CHAPTER 1	1
INTRODUCTION.....	1
1.1 Overview	1
1.2 Problem Statement	3
1.3 Objectives	4
1.4 Social Impact	5
1.5 Issues Encountered.....	6
1.6 Organization of the Thesis	6
CHAPTER 2.....	7
LITERATURE REVIEW	7
2.1 Overview	8
2.2 Summary	20
CHAPTER 3.....	20
METHODOLOGY	20
3.1 Overview	21
3.2 Data Collection and Preprocessing.....	21
3.2.1 Data Sources	22
3.2.2 Data Collection Techniques.....	23
3.2.3 Preprocessing Steps	23
3.2.4 Annotation Process	28
3.2.5 Dataset Description	29
3.3 Pre-trained Models for Comparative Analysis	32
3.3.1 EfficientNetB0	32

3.3.2	DenseNet121	33
3.3.3	MobileNetV2	34
3.3.4	VGG16	35
3.3.5	SqueezeNet	35
3.4	Custom Model: MediNet-XG	36
3.4.1	MediNet classifier	37
3.4.2	Grad-CAM.....	43
3.4.3	t-SNE	44
3.4.4	Knowledge-based query generator from JSON	45
3.5	Summary	48
CHAPTER 4.....		48
IMPLEMENTATION		48
4.1	Overview	49
4.2	Technologies Used	49
4.2.1	Hardware	49
4.2.2	Software	50
4.3	Dataset Preparation	50
4.3.1	Class Definitions	51
4.3.2	Data Preprocessing	51
4.3.3	Data Augmentation	52
4.3.4	Data Splitting	53
4.4	Model Selection and Training.....	54
4.4.1	Building Pre-trained Models	55
4.4.2	Training Pre-trained Models	55
4.5	Custom Model: MediNet-XG	56
4.5.1	MediNet classifier	57
4.5.2	t-SNE (X)	57
4.5.3	Knowledge-based query generator from JSON (G)	57
4.6	Summary	58
CHAPTER 5.....		58
RESULT AND ANALYSIS		58
5.1	Overview	59
5.2	Model Performance Overview	59
5.3	Detailed Analysis	60
5.3.1	Loss and Accuracy per Epoch Graphs	60
5.3.2	AUC-ROC Analysis	64

5.4	Custom Mode: MediNet-XG	65
5.4.1	MediNet Classifier	65
5.4.2	Grad-CAM (X)	66
5.4.3	t-SNE (X)	67
5.4.4	Query Answer Generation	68
5.5	Comparative Analysis	74
5.5.1	Dataset Comparison and Analysis	74
5.5.2	Models Comparison and Analysis on the prepared dataset	75
5.5.3	Comparative analysis with reviewed (journals) methods	77
5.6	Summary	79
CHAPTER 6		79
CONCLUSIONS AND FUTURE WORKS		79
6.1	Conclusions	80
6.2	Future Recommendations	81
REFERENCES		83
APPENDIX A		86
APPENDIX B		91
BIOGRAPHY		94

LIST OF FIGURES

Figure 1.1: Weed-infested Area Medininal plants	2
Figure 1.2: Organization of the thesis' chapters.....	7
Figure 3.3: Raw data processing flowchart.....	24
Figure 3.4: Sample augmentation.....	25
Figure 3.5: Random data splitting method	28
Figure 3.6: Visualization of the 34 medicinal plant classes	29
Figure 3.7: Class distribution in the dataset.....	30
Figure 3.8: Dataset annotation.....	31
Figure 3.9: Knowledge based JSON file structure	32
Figure 3.10: The EfficientNetB0 architecture	32
Figure 3.11: The DenseNet121 architecture.....	33
Figure 3.12: The MobileNetV2 architecture	34
Figure 3.13: The VGG16 architecture.....	35
Figure 3.14: The SqueezeNet architecture	36
Figure 3.15: Abstract View of MediNet-XG.....	37
Figure 3.16: The MediNet classifier architecture.....	38
Figure 3.17: The Grad-CAM architecture[1].....	43
Figure 3.18: The t-SNE plot view.....	45
Figure 3.19: The query generator architecture.....	46
Figure 5.20: Loss and accuracy curve for EfficientNetB0.....	61
Figure 5.21: Loss and accuracy curve for MobileNetV2.....	61
Figure 5.22: Loss and accuracy curve for DenseNet201.....	62
Figure 5.23: Loss and accuracy curve for VGG16.....	63
Figure 5.24: Loss and accuracy curve for SqueezeNet.....	63
Figure 5.25: AUC-ROC curves for different models.....	64
Figure 5.26: Grad-Cam explanation.....	67
Figure 5.27: t-SNE plot diagram	68
Figure 5.28: Faithfulness distribution across the dataset.....	69
Figure 5.29: Hallucination rate distribution across all generated query answers.....	70
Figure 5.30: Coverage distribution in generated answers.....	71
Figure 5.31: Field attribution counts for generated sentences.....	71
Figure 5.32: Accuracy comparison on prepared dataset[2].....	75
Figure 5.33: Aucc and Loss of MediNet-XG Model.....	76
Figure 5.34: AUC-ROC comparison of various models.....	77

Figure 5.35: Generating the knowledge-based query answer.....	77
---	----

LIST OF TABLES

Table 2.1:	Test accuracy of CNNs and ensemble models.	9
Table 2.2:	Observations on DL for medicinal plant classification.	11
Table 2.3:	Transfer-learning models performance.	13
Table 2.4:	Findings from explainable DL in Phenotyping.	14
Table 2.5:	CNN models on weed detection in wheat fields.	16
Table 2.6:	Performance comparison of the models.	17
Table 2.7:	Comparative Analysis of Recent Studies.	19
Table 5.8:	Performance metrics of various DL models.	60
Table 5.9:	Classification report for MediNet-XG.	65
Table 5.10:	Dataset Evaluation of Query Answer Generation	73
Table 5.11:	Comparison of existing datasets.	74
Table 5.12:	Comparison of MediNet-XG with other studies.	78

CHAPTER 1

Introduction

1.1 Overview

Medicinal plants form a major foundation of traditional health care. Estimates made by WHO show that in most developing countries, approximately 80-88% of the population use herbal medicine in primary health care, particularly in the rural communities where there are low biomedical facilities[3]. Bangladesh is estimated to possess 1,000 conservative species of medicinally active plants, with this amount of ethnomedicinal knowledge densely stored in a relatively tiny region of geographic space [4]. Survey studies show that over 9 out of 10 remedy formulations based on traditional practices are plants based and the situation in Bangladesh is no exception as most of the rural and peri-urban practice is same [5]. The scientific interests and therapeutic potential of this local flora are further supported by pharmacology research of at least 48 Bangladeshi species as having anti-inflammatory properties [4].

Irrespective of this significance, Bangladeshi medicinal plant digital resources are scarce. There are few publicly available standardized datasets of images, and those that exist are usually not consistently labeled, with realistic field backgrounds, or connected to structured botanical and safety data [6]. The available data sets are mostly collected under controlled imaging scenarios and not sufficient to forecast weedy medicinal species that exist as spontaneous flora in farmlands, homesteads and roadside environments with thick weed growths, occlusions and mixed backgrounds where non-experts find it hard to identify the weed under manual guidance [7]. This knowledge gap has been found as a significant burden to biodiversity monitoring and medicinal plant classification using deep-learning models, with numerous studies using small institutional collections or non-Bangladeshi repositories with different species composition to local requirements [5].

Recent deep learning methods have shown great accuracy when used to recognize medicinal plants and crops, but most of them use large pretrained CNNs or ensembles, which are

trained on images with clean backgrounds, and are black-box models, which necessitate computation at server scale. Explainable AI has begun to be used in phenotyping plants and weed detection, and primarily by saliency algorithms like Grad-CAM[8], but these applications often aim to detect crop diseases or weed-crop interactions, rather than recognizing medicinal weeds, and often do not use any structured medical knowledge to be safe in interpretation. Therefore, the explanatory and easy-to-understand models optimized to suit weedy medicinal plants under realistic field conditions have not been fully studied. Specifically, no systems exist, which (i) run within strong memory and compute limits to fit mobile or edge devices, (ii) offer transparent and instance-level explanation of any prediction, and (iii) integrate visual recognition with a formal botanical knowledge base such that outputs are grounded and safety-confined.

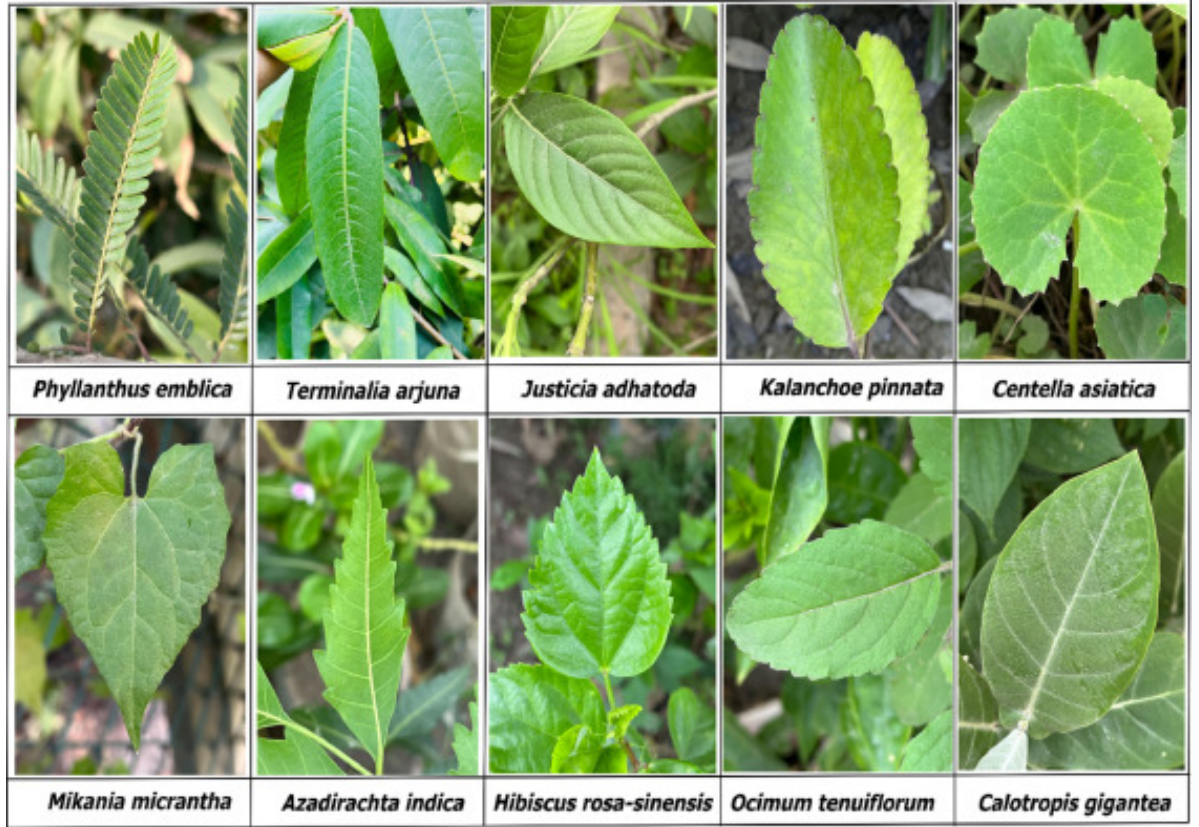


Figure 1.1: Weed-infested Area Medicinal plants[9].

In order to fill the expertise insufficiency and proper recognition problems, the proposed work introduces the lightweight and explainable multimodal architecture of MediNet-XG, which can be used to identify Bangladeshi weedy medicinal plants through in situ RGB leaf images and to provide the corresponding botanical and safety information through retrieval. An ultra-compact CNN classifier is trained on images of leaves recorded directly in the field and makes

high-confidence predictions on a pre-curated collection of 34 species, and Grad-CAM and t-SNE are employed to reveal the spatial decision region and feature-space relationship of each sample. All the predicted classes are connected to known facts about morphology, uses, active compounds, toxicity and pregnancy safety in a structured JSON knowledge base allowing field-level decision support without unstructured text generation.

The key contributions of this work could be summarized in the following way. To start with, the research present a new field-captured dataset of 17,050 RGB leaf images like Figure 1.1 of 34 Bangladeshi weedy medicinal species that are rank-awerely annotated with empirical weediness and realistic background clutter. Second, a CNN architecture named MediNet-XG is constructed which has high accuracy and F1-score on this dataset and has a model size that is measured in the hundreds of kilobytes range, which is significantly smaller than standard pretrained backbones. Third, it has a built-in explainability module, which is a combination of Grad-CAM and t-SNE that enables the observation of both spatial and feature-space components of the decision process per prediction. Fourth, a knowledge engine in the form of a retrieval engine is implemented, which provides a mapping between predicted classes and a maintained JSON botanical database to provide organized, safety-conscious answers to user queries.

1.2 Problem Statement

Medicinal plants form the backbone of traditional healthcare and over 90% of remedies worldwide are plant based which is especially important in Bangladesh where formal medical access is uneven[10]. Approximately 1,000 Bangladeshi species are conservatively regarded as medicinal and at least 48 have already been pharmacologically studied for anti inflammatory use which highlights dense species diversity and functional overlap[4]. Many of these taxa share very similar leaf shapes and colors so that even controlled image based identification is difficult and the task becomes more complex when plants grow as spontaneous weeds in fields and roadsides where dense vegetation occlusion and heterogeneous backgrounds dominate the scene. Weed monitoring studies further show that small or scattered patches are often missed because models are trained on larger clearer regions which suggests that medicinal plants appearing as minor components among aggressive weeds are easily overlooked or confused with non medicinal or toxic species. Although deep CNN and ensemble models for Bangladeshi

medicinal plants report accuracies close to 99 percent on curated datasets these architectures remain heavy server oriented black boxes and a recent review identifies the trustworthiness and interpretability of such models as a major open problem[5]. Key technical challenges highlighted in the literature include:

- **High inter-class similarity** among medicinal species, which reduces separability even under simplified backgrounds.
- **Traditional method** by taxonomists is slow, complex, and error-prone.
- **Missed detection** of small or scattered targets because existing models favor larger homogeneous regions[11].
- **Computationally heavy CNNs and ensembles**, such as DenseNet-based models, which are unsuitable for low-power mobile devices.
- **Predominantly black-box inference** with little visual explanation or structured medicinal knowledge for safe decision-making.

For Bangladeshi medicinal plants in weedy areas, the main problems are similar species, cluttered backgrounds, lack of field images, poor explainability and the difficulty of compressing robust models into small on-device architectures without losing too much accuracy.

1.3 Objectives

The present research is structured around three core objectives which together define the MediNet-XG framework. The first objective is to develop a lightweight explainable multi-modal AI architecture that integrates compact convolutional feature extraction with mandatory visual explanations and a structured botanical knowledge base while remaining suitable for deployment on mobile and edge devices in resource-constrained settings. The second objective is to achieve accurate classification of Bangladeshi medicinal plants from leaf images captured in weed-infested areas so that reliable predictions are obtained under uncontrolled lighting cluttered backgrounds and strong inter-class similarity without relying on computationally heavy server-grade models. The third objective is to generate contextual knowledge on plant uses compounds and habitats by deterministically linking each predicted species to a curated JSON knowledge base that provides grounded information on therapeutic properties phytochemical constituents ecological preferences and safety constraints thus replacing free

form speculation with auditable retrieval based explanations.

- To develop a Lightweight Explainable multimodal AI framework.
- To accurately classify medicinal plants in weed-infested areas.
- To generate contextual knowledge on plant uses, compounds, and habitats.

So, the work seeks to design a compact explainable CNN, ensure robust medicinal plant classification in weed infested scenes and deliver concise retrieval based knowledge on uses compounds and habitats without hallucinated claims. Collectively these objectives position MediNet-XG as a practical and trustworthy decision support tool rather than a generic black box recognition system.

1.4 Social Impact

The proposed MediNet-XG framework has the potential to generate substantial social impact by strengthening safe and informed use of medicinal plants in resource constrained communities. In Bangladesh a large share of the population relies on plant based remedies as the first line of treatment, yet accurate field identification often depends on a small number of experts and a time consuming process of visual inspection touch smell and taste guided by specialized floras and keys. By providing an on device decision support tool that can recognize weedy medicinal species directly from leaf images and immediately retrieve structured knowledge on uses compounds and safety, the system can reduce dependence on scarce expert capacity and make reliable information more accessible to rural households farmers and community health workers.

The availability of explainable predictions and curated safety notes also supports harm reduction and rational use of traditional remedies. Transparent Grad CAM visualizations and knowledge grounded descriptions can help users understand why a plant has been identified in a particular way and what known toxicities or contraindications are associated with it, which may reduce misidentification and discourage unsafe self medication or over harvesting of toxic species. At the same time the focus on weedy medicinal plants that grow spontaneously in fields and homesteads promotes low cost locally available healthcare options without requiring new cultivation or input investments, thereby contributing to economic resilience for low income families.

Beyond individual health outcomes the system can assist agricultural extension services and biodiversity initiatives by documenting the presence of medicinal weeds in crop lands and marginal habitats. Consistent digital records of identified species can support monitoring of agroecosystem diversity and facilitate conservation of culturally important medicinal flora that might otherwise be classified as nuisance weeds and removed. In educational settings the tool can act as a practical companion for botany students and trainee health workers providing immediate feedback and verified descriptions that reinforce formal curricula. Overall the MediNet-XG framework is expected to promote safer traditional healthcare practices, support inclusive access to botanical knowledge and contribute to sustainable management of medicinal plant resources in Bangladesh.

1.5 Issues Encountered

There are several practical and technical issues were encountered during the development of the MediNet-XG framework. Collection of realistic field images of weedy medicinal plants under uncontrolled lighting, cluttered backgrounds and frequent occlusions made it difficult to maintain uniform quality and required expert validated annotation to avoid mislabeling. Designing a model that remained sufficiently lightweight for mobile deployment while preserving high classification performance demanded careful tuning of inverted residual blocks, depthwise separable convolutions and attention mechanisms, especially when explainability modules such as Grad-CAM and t-SNE had to be integrated without exceeding computational limits. Construction of the curated JSON knowledge base also proved challenging because heterogeneous botanical and medicinal sources had to be harmonized into a consistent schema, class labels had to be mapped deterministically to entries despite naming variations and strict safety constraints required conservative handling of incomplete or conflicting information.

1.6 Organization of the Thesis

The thesis is structured into six chapters to present the design, implementation and evaluation of the MediNet-XG framework in a coherent manner. The first chapter introduces the motivation for reliable medicinal plant identification in weed infested environments, outlines the problem statement, specifies the research objectives and highlights the anticipated social

impact of a lightweight explainable decision support system for Bangladeshi contexts. The second chapter provides a detailed literature review on medicinal plant recognition, weed detection and explainable deep learning, emphasizing existing dataset limitations, black box model behavior and deployment challenges for mobile and edge platforms. The third chapter describes the overall methodology of the research, including the design of the MediNet-XG architecture, the construction and preprocessing of the weedy medicinal plant dataset, the integration of Grad-CAM and t-SNE for visual explainability and the development of the structured JSON-based botanical knowledge base.

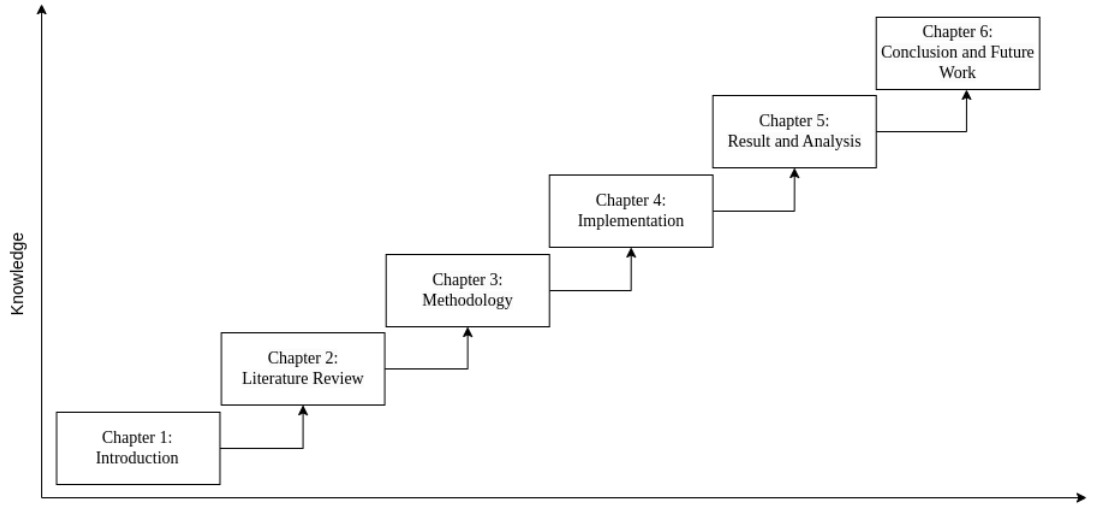


Figure 1.2: Organization of the thesis' chapters.

The fourth chapter focuses on implementation details and experimental setup, covering training procedures, hyperparameter choices, inference pipeline design and the mechanisms that enforce retrieval-based, safety constrained outputs. The fifth chapter presents the quantitative and qualitative results obtained from MediNet-XG, including classification metrics, confusion matrices, visual explanations and comparisons with heavier baseline models, followed by an in depth analysis of performance, trustworthiness and deployability. The final chapter concludes the thesis by summarizing the main contributions and limitations of the proposed framework and by outlining future directions such as expanding the species coverage, incorporating additional modalities and adapting the system for broader agroecological and healthcare applications. This research work is publicly available at GitHub repository[12].

CHAPTER 2

Literature Review

2.1 Overview

Recent advances in deep learning have greatly improved automated plant recognition yet most medicinal-plant frameworks still rely on heavy black-box models trained on controlled images that are difficult to deploy in noisy field conditions. Recent Bangladeshi works on medicinal plant classification achieve very high accuracy with CNN and ensemble architectures but remain offline, computationally expensive and disconnected from any safety-aware medicinal knowledge or explanation interface. Global systematic reviews confirm that the majority of medicinal-plant studies continue to use laboratory-style image datasets with limited species diversity while explainable AI and on-device deployment are mentioned mainly as future directions rather than implemented solutions.

Parallel literature in plant phenotyping, crop disease and weed detection shows that Grad-CAM and related techniques can provide useful visual explanations and that cluttered field backgrounds make small or scattered targets easy to miss, but these efforts focus on crops and weed control rather than species-level medicinal plant identification with knowledge grounding. Overall the state of the art exposes four key gaps: the lack of lightweight CNNs tailored for edge deployment, the scarcity of field datasets where medicinal plants appear as weeds, the limited integration of robust explainability and the absence of tightly coupled retrieval-based botanical knowledge bases. These gaps motivate the design choices and contributions of MediNet-XG described in the following sections.

There is a study "Deep-Learning-Based Classification of Bangladeshi Medicinal Plants Using Neural Ensemble Models[13]", introduces a curated dataset of 10 Bangladeshi medicinal plant species and evaluates multiple CNNs and ensemble strategies for leaf-image classification. The work demonstrates that ensemble learning can significantly boost accuracy over individual models on clean, controlled images. The study begins with the collection of high-quality RGB images of fresh leaves from ten medicinal plant species prevalent in

Bangladesh; images are captured under relatively uniform backgrounds and lighting to reduce noise and facilitate feature extraction. All images are resized to a common resolution and normalized before being split into training, validation and test subsets so that each species is reasonably balanced across the partitions. Several pretrained CNN architectures such as VGG-type, ResNet-type and related models are then fine-tuned on the dataset using transfer learning, with the final fully connected layers adjusted to output ten class probabilities and trained using standard optimization pipelines with learning-rate scheduling and regularization. To further enhance performance, two ensemble strategies are constructed: a soft ensemble that averages the probability vectors from multiple CNNs to produce a consensus prediction and a hard ensemble that combines discrete class labels via majority voting; both ensembles are trained and evaluated offline on GPU-equipped systems and assessed using accuracy, confusion matrices and loss curves on the held-out test set. Reported test accuracies for individual CNN backbones and neural ensemble configurations on the ten-class Bangladeshi medicinal plant leaf dataset, showing the performance gains achieved by ensemble learning over single-model baselines.

Table 2.1: Test accuracy of CNNs and ensemble models.

Model	Description	Acc(%)	Notes
CNN-A	Single pretrained CNN backbone	$\approx 95-96$	Baseline fine-tuned model
CNN-B	Alternative CNN backbone	$\approx 96-97$	Deeper architecture improvement
Ensemble	Averaged class probabilities	97.62	Best soft-voting configuration

The results in Table 2.1 indicate that while single CNNs already achieve high accuracy, the ensembles offer measurable improvements, with the hard ensemble reaching about 98 percent test accuracy and providing the best overall performance on the controlled dataset. But there are some limitation here-

- **Controlled imaging conditions rather than field scenes:** The dataset is composed of leaf images captured under relatively uniform backgrounds and lighting, which does not reflect weed-infested, cluttered environments where medicinal plants naturally occur and where background noise and occlusion are common. MediNet-XG instead targets real field images with heterogeneous backgrounds to improve robustness in practical use.
- **Computationally heavy ensemble architectures:** The use of multiple large pretrained

CNNs combined via ensemble strategies results in a high parameter count and significant memory and compute requirements, making these models impractical for low-power mobile or edge devices. MediNet-XG is designed as an ultra-lightweight CNN (hundreds of kilobytes) optimized specifically for on-device inference.

- **Black-box predictions without explainability:** The models output class labels and accuracy metrics but do not provide visual explanations such as saliency maps or feature-space context, limiting user trust and making error analysis difficult, especially in health-adjacent settings. MediNet-XG integrates Grad-CAM and t-SNE-based visualizations so that the discriminative leaf regions and class relationships behind each prediction are explicitly revealed.
- **No linkage to medicinal knowledge or safety information:** The study focuses solely on species classification and does not connect predictions to structured information about plant uses, active compounds or toxicity, nor does it impose constraints on how outputs might be interpreted in medicinal contexts. MediNet-XG couples each predicted class to a curated JSON knowledge base that encodes botanical, therapeutic and safety attributes, enabling retrieval-based explanations and refusal of unsupported or unsafe queries.

Another study "Deep Learning for Medicinal Plant Species Classification: Review and Future Directions[7]". The systematic review synthesizes recent deep-learning approaches for medicinal plant species classification across multiple countries, datasets and imaging setups, providing a consolidated picture of current practices and open challenges. The article emphasizes that while CNN-based models routinely report high accuracies on curated datasets, most studies lack robustness evaluation, explainability and deployment planning, especially in realistic field environments. Relevant articles were retrieved from major scientific databases using predefined keywords related to medicinal plants, deep learning and image classification and were filtered through inclusion and exclusion criteria to focus on peer-reviewed works published in recent years. Each selected paper was analyzed along several dimensions, including plant organ used for imaging (leaf, flower, fruit or whole plant), imaging modality (RGB, hyperspectral, microscopic), dataset characteristics (size, species count, capture conditions), model architecture (standard CNN, transfer learning, ensemble) and evaluation metrics (accuracy, precision, recall, F1). The review then categorized studies by application domain

(taxonomic classification, disease detection, pharmacological screening support) and examined whether any form of explainable AI, uncertainty estimation or resource-efficient design was implemented, with particular attention to gaps in public datasets, reproducibility and field validation. Main patterns and gaps identified in the systematic review of deep-learning methods for medicinal plant species classification, summarizing dataset, model, evaluation and explainability practices.

Table 2.2: Observations on DL for medicinal plant classification.

Aspect	Typical practice reported	Noted gap / limitation)
Dataset	Medium-sized image datasets, often private, with tens of species captured under controlled conditions	Scarcity of large, public, field-captured datasets; limited representation of weedy or wild medicinal plants
Models	Standard CNNs and transfer-learning backbones (e.g., VGG, ResNet, Inception) tuned for high accuracy	Limited attention to model size, latency or edge deployment; few studies consider low-resource devices
Evaluation	Accuracy as primary metric, sometimes with precision/recall or F1	Rare robustness tests under varying backgrounds, lighting or occlusion; little discussion of generalization
XAI	Very few works include saliency maps or XAI; almost none link outputs to medicinal safety information	Lack of systematic explainability and absence of safety-aware, knowledge-grounded outputs identified as major future directions

The review in Table 2.2 concludes that deep learning has strong potential for medicinal plant recognition but that progress is constrained by limited dataset diversity, lack of standardized benchmarks, minimal use of XAI and insufficient focus on deployment in real-world, resource-constrained settings. But the limitations are-

- **Scarcity of realistic field datasets:** Most surveyed studies use controlled images with simple backgrounds and do not represent wild or weedy medicinal plants, which reduces ecological validity. MediNet-XG explicitly builds on field-captured leaf images from weed-infested environments to close the realism gap.
- **Limited integration of explainable AI:** Only a small fraction of reviewed works incorporate saliency maps or explanation mechanisms, and even those are not consistently evaluated or standardized. MediNet-XG embeds Grad-CAM and t-SNE-based feature-space visualization as mandatory components so that each prediction is accompanied by transparent visual evidence.
- **Absence of safety-aware, knowledge-grounded outputs:** The review highlights that

current systems largely stop at species labels and do not connect to structured information on medicinal uses, active compounds or toxicity, leaving interpretation entirely to the user. MediNet-XG couples each prediction to a curated JSON knowledge base that encodes botanical attributes, therapeutic roles and explicit safety notes, and restricts textual responses to retrieval from the source to avoid hallucinated or unsafe advice.

- **Lack of focus on weedy medicinal plants:** The surveyed literature primarily addresses cultivated, ornamental or pharmacologically important species without emphasizing plants that occur as weeds in agricultural landscapes. MediNet-XG specifically targets weedy medicinal species in cluttered fields, aligning the model with real-world use cases such as rural healthcare and agroecological management in Bangladesh.

There was a research work published in 2024, "IndoHerb: Indonesia Medicinal Plants Recognition Using Transfer Learning[6]". The IndoHerb study proposes a deep-learning framework for recognizing Indonesian medicinal plants from leaf images using transfer-learning CNN models. A region-specific dataset of local herbal species is introduced and several pre-trained architectures are fine-tuned to assess how well generic visual features can be adapted for medicinal plant identification in an Indonesian context. The work begins with the collection of RGB leaf images for multiple Indonesian medicinal plant species, captured under relatively controlled conditions to ensure clear visibility of shape and venation patterns. All images are resized to a fixed input resolution, normalized and split into training, validation and test sets, with class balancing applied to reduce bias across species. Transfer learning is employed by taking CNN backbones such as MobileNet, Inception or similar lightweight architectures pretrained on large-scale image datasets, replacing their final classification layers with new fully connected heads tailored to the number of medicinal classes and fine-tuning the networks on the IndoHerb dataset. Standard optimization procedures are used, including learning-rate scheduling, early stopping and data augmentation (e.g., rotation, flipping) to improve generalization, and model performance is evaluated using accuracy and confusion matrices on a held-out test set. Accuracy of different transfer-learning CNN architectures on the IndoHerb dataset, showing that lightweight models can reach close to 98 percent accuracy on controlled Indonesian medicinal plant images. The results in Table 2.3 demonstrate that transfer-learning CNNs are highly effective for regional medicinal plant recognition, with some lightweight backbones achieving test accuracies near 98 percent, thereby validating the

feasibility of adapting generic visual features for local herbal species classification.

Table 2.3: Transfer-learning models performance.

Model	Description	Acc(%)	Notes
TL-CNN-A	Pretrained CNN backbone fine-tuned on IndoHerb	$\approx 96\%$	Baseline transfer-learning performance
TL-CNN-B (MobileNet variant)	Lightweight backbone with fine tuned head	$\approx 97\%$	Best performing configuration on the dataset

Though there was a huge novel contribution in the study, but the study had some limitation too-

- **Controlled imaging rather than cluttered field environments:** IndoHerb primarily uses leaf images captured under relatively clean and uniform backgrounds, which do not reflect the complex weed-infested scenes in which many medicinal plants are encountered in practice. MediNet-XG is explicitly trained and evaluated on field-captured images from weed-infested areas in Bangladesh to improve robustness under real-world visual clutter.
- **Lack of mandatory explainability:** The IndoHerb framework focuses on classification accuracy and does not integrate Grad-CAM, t-SNE or other explanation tools, leaving predictions as black-box outputs. MediNet-XG enforces per-sample explainability by generating Grad-CAM heatmaps and feature-space visualizations for the predicted class, thereby supporting user trust and diagnostic understanding.
- **No explicit focus on ultra-lightweight edge deployment:** Although some IndoHerb models use relatively compact backbones, the study does not target a specific model-size budget or latency requirement for low-end mobile devices and does not report deployment-oriented metrics. MediNet-XG is deliberately constrained to a model size of roughly 342 KB and is optimized for edge inference, aligning with the computational realities of rural Bangladeshi users.
- **Regional scope without cross-domain extension to weeds:** IndoHerb successfully addresses Indonesian herbal species but does not consider weedy growth forms or mixed crop–weed settings that complicate recognition. MediNet-XG extends the line of work to weedy medicinal plants in agricultural landscapes, addressing both species-level

recognition and the presence of dense weed backgrounds.

”Explainable Deep Learning in Plant Phenotyping[14]”, a review paper which surveys recent applications of explainable deep learning in plant phenotyping, with an emphasis on how saliency-based and attribution methods can reveal biologically meaningful features in complex plant images. The authors argue that interpretability is essential for scientific discovery and user trust, since plant scientists and breeders require insight into why deep models focus on particular structures rather than accepting opaque predictions. Relevant phenotyping studies were collected from major databases and grouped according to task type, including stress detection, yield-related trait estimation and organ segmentation, as well as according to imaging modality such as RGB, hyperspectral and 3D point clouds. For each study the review examined the underlying deep architecture (e.g., CNNs, U-Nets, transformers), the XAI technique applied (Grad-CAM, Guided Backpropagation, Layer-wise Relevance Propagation, SHAP and others), the way explanations were visualized and the extent to which experts validated the highlighted regions or relevance scores. The authors also discussed evaluation protocols for explanations, covering qualitative expert assessment and emerging quantitative metrics, and identified common pitfalls such as instability of saliency maps and the risk of misinterpreting artifacts as biologically relevant signals. The review concludes that XAI has begun to uncover meaningful plant traits and stress patterns in deep models, but that explanations are still inconsistently applied, rarely standardized across studies and are mostly limited to research platforms rather than embedded in field-ready tools.

Table 2.4: Findings from explainable DL in Phenotyping.

Aspect	Typical practice reported	Noted gap / limitation)
XAI Methods used	Grad-CAM and related saliency maps most common; some use LRP or Integrated Gradients	Provide intuitive heatmaps over leaves, stems and fruits but explanations are often qualitative only
Application domains	Stress detection, nutrient deficiency, yield trait estimation, organ segmentation in crops	Demonstrate that highlighted regions frequently match known stress markers or trait-relevant organs
Expert involvement	Plant scientists or breeders visually inspect explanation maps in many studies	Expert feedback validates biological plausibility but systematic quantitative evaluation remains limited
Deployment focus	Research and phenotyping platforms; little emphasis on mobile or edge deployment	Interpretability research is strong, but real-time field tools remain underexplored

In Tabel 2.4 summary of explanation techniques, application domains and evaluation practices

reported in the review on explainable deep learning for plant phenotyping. the limitations were-

- **Focus on crops and phenotyping, not medicinal plants:** The surveyed works center on staple crops and experimental phenotyping tasks such as stress or yield analysis, with no dedicated focus on medicinal species or ethnobotanical applications. MediNet-XG extends explainable deep learning to Bangladeshi weedy medicinal plants, aligning XAI with traditional healthcare and biodiversity use cases.
- **Lack of coupling between XAI and structured knowledge:** Explanations in phenotyping studies rarely connect to structured botanical or management knowledge; they mainly show where the model “looks” without linking to actionable information. MediNet-XG couples its XAI outputs with a curated JSON knowledge base containing uses, compounds and safety notes so that highlighted regions and class decisions are directly tied to interpretable medicinal context.
- **Limited attention to lightweight, field-deployable models:** The review notes very little emphasis on building ultra-compact architectures that can run on mobile or edge devices in real field conditions. MediNet-XG explicitly constrains model size to approximately 342 KB and targets weed-infested field imagery for deployment in resource-limited rural environments.

”Deep Learning for Weed Detection in Wheat Fields.[15]”- The research develops deep-learning models for detecting weeds in wheat fields using RGB imagery, aiming to support precision herbicide application and reduce manual scouting effort. The study demonstrates that CNN-based classifiers can accurately distinguish weed patches from crop regions under varying field conditions which highlights the potential of deep learning for real-world weed management. High-resolution images of wheat fields were collected across multiple sites and growth stages, capturing a wide range of lighting conditions, soil appearances and weed densities. Images were annotated to label pixels or regions as crop or weed then split into training, validation and test sets to ensure coverage of different environments. Several CNN architectures were trained for classification and detection tasks, with data augmentation applied to improve robustness, and models were evaluated using accuracy and related metrics to quantify performance on unseen field data. The findings indicate that well-trained CNNs can achieve

test accuracies above 95 percent for weed detection tasks and can operate effectively in visually heterogeneous wheat fields, supporting their use in precision agriculture workflows. Table 2.5 reported accuracies for CNN-based weed detection models in wheat fields, showing

Table 2.5: CNN models on weed detection in wheat fields.

Model	Task	Acc(%)	Notes
CNN-1	Crop vs weed classification	95%	Baseline weed detection network
CNN-2	Enhanced architecture / detection	95%	Similar or slightly higher accuracy with better localization

that deep learning can reliably distinguish weeds from crops in complex field imagery. But founded some gaps in the study-

- **Binary weed vs crop focus rather than species-level medicinal identification:** The models differentiate weeds from wheat but do not identify individual weed or medicinal species which limits their usefulness for ethnobotanical or medicinal applications. MediNet-XG instead performs species-level classification across 34 weedy medicinal plants relevant to Bangladeshi healthcare and agriculture.
- **No integration of explainable AI for individual predictions:** The study emphasizes detection accuracy and does not incorporate Grad-CAM, t-SNE or similar explanations to show which leaf or canopy regions drive each decision. MediNet-XG embeds Grad-CAM and feature-space context as mandatory outputs for each prediction to enhance transparency and user trust.
- **bsence of structured medicinal knowledge and safety information:** Weed detection outputs are not linked to any knowledge base about plant uses, toxicity or therapeutic relevance which is critical when dealing with medicinal or poisonous species. MediNet-XG connects each predicted class to a curated JSON knowledge base containing uses, compounds and safety constraints and restricts textual responses to retrieval from the source.
- **Deployment not optimized for ultra-lightweight on-device inference:** While field applicability is considered, the work does not target an extremely small model footprint or strict memory limits suitable for low-end mobile devices. MediNet-XG is explicitly designed to remain around 342 KB to enable reliable on-device inference in rural Bangladeshi contexts.

There was another study found, which worked on BD medicinal plants- "An Efficient Dual-Attention Guided Deep Learning Model with Optimized Hyperparameters for Bangladeshi Medicinal Plant Classification.[16]". The paper proposes a dual-attention CNN architecture with optimized hyperparameters for classifying Bangladeshi medicinal plants, aiming to improve feature extraction and recognition accuracy over conventional CNN models on leaf imagery. The study demonstrates that attention mechanisms can enhance the representation of fine-grained leaf patterns and yields state-of-the-art performance on a Bangladeshi medicinal plant dataset. A dataset of Bangladeshi medicinal plant leaves was assembled and preprocessed through resizing, normalization and data augmentation to mitigate overfitting. The authors designed a custom CNN incorporating both spatial and channel-wise attention modules, allowing the network to focus on informative regions and feature channels within leaf images. Hyperparameters such as learning rate, batch size and optimizer settings were tuned using systematic search strategies, and the dual-attention model was trained and evaluated against baseline CNNs using accuracy and other metrics on a held-out test set. Reported test accuracies for individual CNN backbones and neural ensemble configurations on the ten-class Bangladeshi medicinal plant leaf dataset, showing the performance gains achieved by ensemble learning over single-model baselines.

Table 2.6: Performance comparison of the models.

Model	Architecture	Acc(%)	Notes
Baseline CNN	Standard convolutional network	> 95%	Strong baseline accuracy
Dual-attention CNN	Spatial + channel attention with tuned hyperparameters	$\approx 97\%$	Highest reported performance on the dataset

Table 2.6 present test accuracies for baseline and dual-attention CNN models on a Bangladeshi medicinal plant dataset, showing accuracy gains achieved through attention mechanisms and hyperparameter optimization. But the founded limitations were-

- **Model complexity and limited suitability for low-resource devices:** The dual-attention architecture increases computational cost and parameter count compared with simpler CNNs and the study does not target strict constraints for deployment on low-end mobile or edge hardware.
- **Lack of explainability despite use of attention:** Although attention mechanisms highlight informative features internally, the model does not expose explicit explanation

outputs such as Grad-CAM maps or feature-space plots for end users. MediNet-XG complements its compact architecture with mandatory Grad-CAM and t-SNE-based explanations so that human users can see which leaf regions and class neighborhoods support each prediction.

- **Absence of knowledge-grounded medicinal context:** The dual-attention model focuses solely on species classification and does not link outputs to structured information on medicinal uses, phytochemical constituents or toxicity, nor does it enforce safety constraints on system responses. MediNet-XG associates each species label with a curated JSON knowledge base that encodes uses, compounds, habitats and safety notes, and all textual explanations are restricted to retrieval from the knowledge base.
- **No explicit consideration of weed-infested field imagery:** The dataset used in the study consists mainly of controlled leaf images and does not account for cluttered weed backgrounds or small, occluded leaves that frequently occur in real agricultural settings. MediNet-XG is trained and evaluated on leaf images captured in weed-infested environments, addressing recognition under realistic field conditions in Bangladesh.

Table 2.7: Comparative Analysis of Recent Studies.

Title	Plant	Modality	Real-time	Classify	XAI	Gen	Acc/F1	Country
Deep-Learning-Based Class. (Mathematics, 2023)[13]	BD medicinal (10 spp., controlled leaf)	Image	No	Yes	No (Black-box)	No	97.62% (Soft Ens.)	Bangladesh
DL for Med. Plant: Review (Frontiers, 2024)[7]	Multi-country survey	Image+	No	Yes	Rarely	No	NA	Global
IndoHerb: Indonesia (Comp. Elec. Agri, 2023)[6]	Indonesian herbal	Image	No	Yes	No	No	~97% (TL CNN)	Indonesia
Explainable DL: Plant Phenotyping (Frontiers, 2023)[14]	Gen. crops phenotyping	Img+Data	No	Yes	(Grad-CAM)	No	NA	Global
DL for weed detection: Wheat (Comp. Elec. Agri, 2024)[15]	Weeds vs crop in wheat	Image	Field DL	Yes	Limited	No	>95%	Turkey
Dual-attention model (Journal TBD, 2025)[16]	BD medicinal plants	Image	No (Heavy)	Yes	No	No	~ 97%	Bangladesh
Novel Multistage Approach (IRJMITE, 2025)[17]	Med. vs Fruit vs Flower	Image	No	Yes	No	No	>95%	Global
DL for weed detection(Frontiers, 2026)[18]	Complex field weeds	Image	No	Yes	Sparse	No	NA	Global
MediNet-XG (Proposed)	BD weedy medicinal (34 spp., field)	Img+Text Multimodal	Yes	Yes	Yes	*yes	98% (~342KB)	Bangladesh

*Note: Retrieval-based structured knowledge; non free-form generative.

2.2 Summary

Existing literature from Table 2.7 shows that deep learning has achieved very high accuracy for medicinal plant and weed recognition, but most solutions remain heavy, black-box models trained on controlled images rather than realistic field data. Ensemble and attention-based CNNs for Bangladeshi medicinal plants regularly report accuracies above 97–98 percent, yet they are computationally expensive, lack per-sample explanations and do not connect predictions to structured medicinal knowledge or safety information. Systematic reviews further highlight major gaps: scarcity of public field-captured datasets, limited attention to lightweight edge deployment and only sporadic use of explainable AI methods, which are mostly confined to crop phenotyping and disease diagnosis rather than medicinal species in weed-infested environments. Within the landscape MediNet-XG is positioned to fill four specific gaps: (i) species-level recognition of weedy medicinal plants from cluttered Bangladeshi field images, (ii) an ultra lightweight architecture suitable for on-device inference, (iii) mandatory visual and feature-space explainability for every prediction and (iv) retrieval-based coupling of outputs to a curated botanical knowledge base that encodes uses, compounds and safety constraints, thereby addressing both trustworthiness and real-world deployability.

CHAPTER 3

Methodology

3.1 Overview

Under this methodology approach, the research activity adheres to a concise and entirely comprehensive flow of field pictures to safe and interpretable choices. To begin with, RGB leaf images of 34 various Bangladeshi weed-infested medicinal plants are obtained in natural conditions of real weed-infested field and public repositories and standardized by correcting orientation, resizing to a desired resolution, normalizing and data augmentation to retain natural differences in illumination, background, and occlusion and make the dataset trainable.

A custom ultra-light CNN model, MediNet-XG, some pretrained models- VGG, DenseNet121, EfficientNetBO-TL, MobileNetV2, SqueezeNet are then trained on these images to predict plant species, using inverted residual blocks and depth-separable convolutions to achieve high performance while keeping the model to 342KB with precision 0.9882, precision 0.9875, recall 0.9887, and F1-score 0.9879, compared to other pretrained models MediNet-XG comes with the best performance. After the prediction of a class, the Grad-CAM maps and T-SNE feature-space plots are generate the reseon why the image belongs to that class and why the model takes that decision. The model use that predicted output class label as an input and maps to answer the query of the stakeholders with the JSON knowledge-based file and finally provide precise, non-hallucinating answer.

3.2 Data Collection and Preprocessing

The dataset is being created through the leaf image of 24 species from field collection and 10 species form a public repositories named Mendeley[9] around Bangladesh. This dataset contains 17,050 images divided into 34 classes, each representing a different species of the medicinal leaf. The gathering procedure seeks to create a comprehensive representation of

medicinal plant of the weed infested area to aid in research and development in Geology, Geography, Environmental Science and Agricultural and Food Sciences domains. This dataset will be a valuable resource for improving the accuracy and efficiency of medicinal plant of weed infested area in Bangladesh to classification and recognition systems.

3.2.1 Data Sources

Data for the study was collected from various sources in order to have a diverse, representative and practically useful dataset for weed infested medicinal plant recognition in Bangladesh. The main source of visual information is high-resolution RGB leaf images of 34 medicinal plants species that are common in weed infested fields, taken in natural outdoor conditions using a smartphone camera. Field images were collected from several agro ecological locations, including Dinajpur, Saidpur, Parbatipur, Satkhira and Rangpur so that variations in soil background, illumination, occlusion and co-occurring weeds are naturally represented in the dataset. There are some publicly available image repositories present in online, especially Mendeley[9], which had a Bangladeshi medicinal plant leaf dataset but exploited to augment species that were less commonly seen in the field, giving a final dataset of 17,050 images stored in a JPEG format, with approximately 500 images per class after preprocessing and balancing.

To complement the queries about the image data with structured domain knowledge and textual information about each of the 34 species, were compiled from recognized botanical and medicinal plant databases like MPEDB[19], BNH[20], BJPT[21] and Plants of the World Online (POWO)[22]. From the above-mentioned references, 1,020 question-answer pairs were built up ranging from the scientific and local names, morphological features, ecological preferences, traditional and pharmaceutical uses, known phytochemical constituents and safety considerations, and later encoded in a CSV file and a knowledge base in the form of a JSON document for subsequent use. Agricultural extension professionals and a university botany faculty member confirmed both the species names of the images and the information written in the text fields that the images were taken to ensure that there was taxonomic accuracy, consistency across sources and practical relevance to stakeholders such as farmers, extension workers and pharmacists. The combination of heterogeneous collection sites, mixed image sources and expert curated textual references is providing a robust foundation on grounded

knowledge the train and the evaluation of the MediNet-XG model and helps the decision support components.

3.2.2 Data Collection Techniques

The research used a purposive sampling technique in order to pick 34 medicinal plant species that often founded in the weed infested areas. The strategy allowed a fair distribution of presentation throughout the final dataset with the aim of reaching mostly 500 images of each species. Images are obtained of a variety of individual plants at different stages of growth, and at different leaf orientations to achieve large intra- and inter-class variation, and intrinsically with natural differences in shape, size, texture, and other minor physiological damage. Also, the spatial diversity was obtained through sampling and augmentation of different geographical districts such as Dinajpur, Saidpur, Parbatipur, Satkhira and Rangpur and hence the different soil background, co-occurring weed species and local agricultural management practices are taken into consideration.

The protocol to acquire the image used handheld by smartphone cameras to recreate the ecosystem of the real field although the protocol still applies a uniform method to distance, framing and focus. The techniques makes sure that the main leaves are identifiable even when the background clutters of the real life are there. Every plant was photographed in various perspectives and angles, such as top-down and oblique, and different angles of rotation, but with natural and different light sources instead of limited light sources in artificial light. The dataset achieves this by avoiding the use of tripods and using handheld acquisition to capture the dataset, which effectively combines natural variations in scale, slight motion blur, and perspective effects and is much more reminiscent of the real-world practice of end-users in a field scenario of data collection.

3.2.3 Preprocessing Steps

The preprocessing phase involved preparing the raw data for training and analysis to ensure the model operates efficiently and smoothly. The workflow for transforming raw images into a structured dataset is illustrated in Figure 3.3. This process comprises several critical activities, beginning with the standardization of the aspect ratio to a 1:1 square format to maintain homogeneity across the dataset. Subsequently, the images are resized in two stages: first to a

resolution of 300×300 pixels for the stored dataset—ensuring they retain sufficient quality for various scales—and finally to 224×224 pixels for model training. This specific resolution is widely considered a standard for various machine learning architectures. By enforcing a constant 1:1 aspect ratio and uniform dimensions, the dataset achieves a high degree of homogeneity, which significantly improves the computational efficiency and performance of the trained models. Consequently, the processed images are stored and ready for further analysis, ensuring that the final dataset is of high quality and structurally consistent.

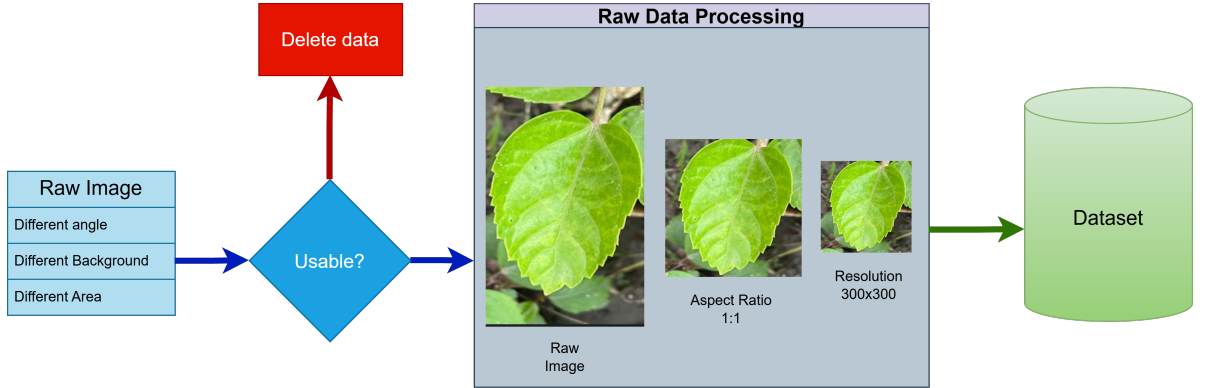


Figure 3.3: Raw data processing flowchart.

Image Resize: The collected images are resized into a standard resolution of 300x300 pixels using the Lanczos interpolation method. Lanczos interpolation works by using a weighted average of nearby pixels to calculate the new pixel value, resulting in smoother and higher quality resized images compared to other interpolation methods. This resizing ensured uniformity in image dimensions across the dataset, facilitating efficient processing and model training. Additionally, each resized image was renamed with a unique identifier indicating its class, aiding in dataset organization and management. The Lanczos kernel $L(x)$ is defined as:

$$L(x) = \begin{cases} \text{sinc}(x) \cdot \text{sinc}\left(\frac{x}{a}\right), & \text{if } -a \leq x \leq a \\ 0, & \text{otherwise} \end{cases}$$

where $\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$ is the sinc function, and a is the scale factor determining the size of the kernel. The interpolation formula using the Lanczos kernel is:

$$I_{\text{interp}}(x) = \sum_{n=-N}^N I(n) \cdot L(x - n)$$

where $I_{\text{interp}}(x)$ is the interpolated pixel intensity at position x , $I(n)$ is the intensity of the input

pixel at position n , N is the number of neighboring pixels considered in the interpolation, and $L(x - n)$ is the Lanczos kernel evaluated at the distance between the output pixel position x and the neighboring input pixel position n [23].

Orientation Correction: Images containing Exif metadata with orientation tags were adjusted to ensure uniform orientation across all images. This correction process involved rotating images by 180, 270, or 90 degrees, depending on the orientation tag values (3, 6, or 8, respectively). This meticulous alignment was crucial to maintain consistency throughout the dataset, preventing any orientation discrepancies that might otherwise affect the accuracy and reliability of subsequent image processing and analysis tasks. By standardizing the orientation, the images were prepared optimally for use in various applications, enhancing overall model performance and ensuring reliable results in further analyses and visualizations.

Data Augmentation:

Various data augmentation techniques were applied to increase dataset diversity and robustness. These techniques included random rotation (up to 20% rotation), random translation (up to 10% height and width translation), random zooming (up to 20% zoom), horizontal flipping, and random contrast adjustment (up to 20%).

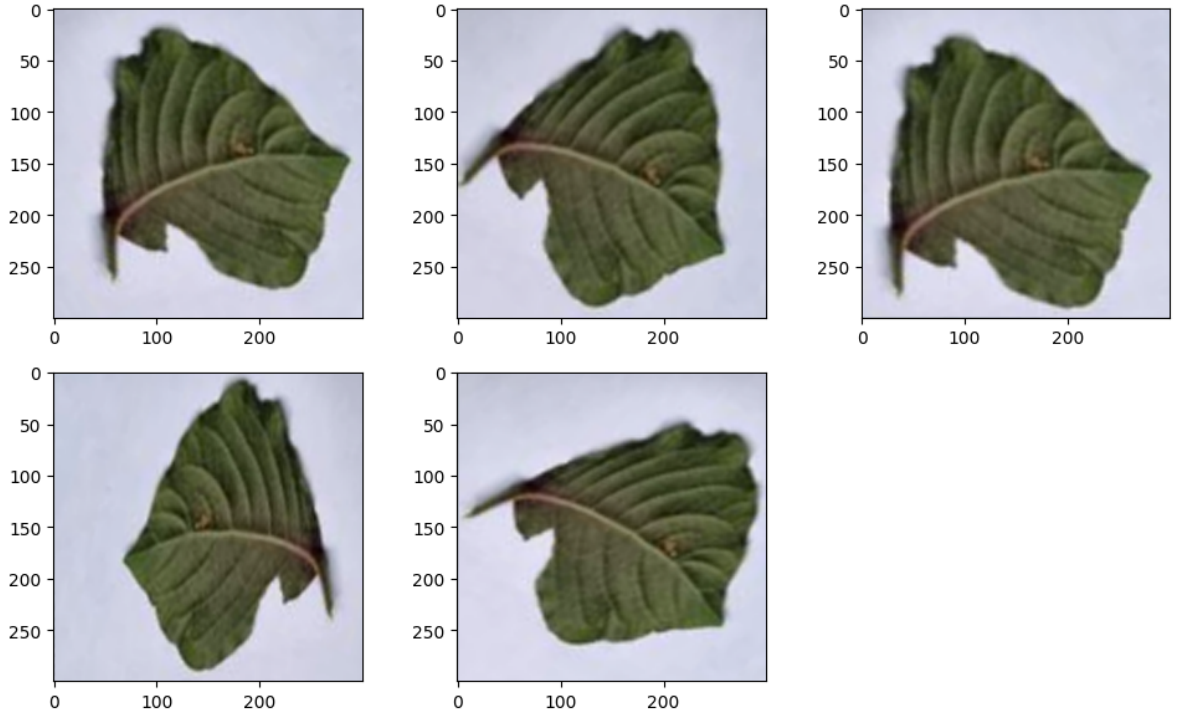


Figure 3.4: Sample augmentation.

In addition, to further simulate real-world variations and improve model generalizability, techniques like random shearing, color jittering, and Gaussian blur were also employed. By incorporating these diverse augmentation strategies, the model's ability to recognize signs under different conditions and reduce overfitting was significantly improved.

Random Rotation: Calculate the rotation matrix:

$$\text{Rotation Matrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

For each pixel in the original image, apply the rotation matrix to calculate the new coordinates:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \text{Rotation Matrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Random Translation: Determine the translation matrix by which the image get transformed from original place to another place:

$$\text{Translation Matrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} \Delta h & 0 \\ 0 & \Delta w \end{bmatrix}$$

Apply the translation matrix to each pixel in the original image to calculate the new coordinates:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \text{Translation Matrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Random Zooming: For each pixel in the original image, scale the coordinates by the random zoom factor:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \alpha \begin{bmatrix} x \\ y \end{bmatrix}$$

Horizontal Flipping: For each pixel in the original image, apply the flipping matrix to flip horizontally:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Image Normalization: Min-max scaling, also known as normalization, is a preprocessing technique used to transform data to a specified range, commonly $[0, 1]$ or $[-1, 1]$. The images were normalized using the Min-Max scaling method, where pixel values were scaled to a range of $[0, 1]$ by dividing each pixel value by 255. This rescaling ensures that all features contribute equally to the model, improving the performance and convergence speed of machine learning algorithms, especially those sensitive to input data scale like neural networks. Normalization ensures consistency in pixel intensity across images, mitigates the impact of varying illumination conditions, and enhances model stability and performance. By providing uniform and standardized input data, min-max scaling enhances the effectiveness of tasks such as image classification and segmentation. Min-max scaling is a popular method used in data preprocessing to scale features to a specified range[24]. The formula for min-max scaling is:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

To scale to a different range $[a, b]$, the formula is:

$$x' = a + \left(\frac{x - \min(x)}{\max(x) - \min(x)} \right) \cdot (b - a)$$

For images, pixel values are often scaled to $[0, 1]$ using:

$$x' = \frac{x}{255}$$

The original pixel value is denoting by x . and 255 is image levels, used for 8-bit image as like $2^{bit} - 1 = level$ or $2^8 - 1 = 255$

Train-Test-Validation Split: The dataset was split into training, validation, and test sets using a random splitting strategy (Figure 3.5) [25]. By shuffling the data and allocating a portion to each subset—typically 70% for training, 15% for validation, and 15% for testing—this method ensures that the subsets represent the overall dataset’s diversity. Random splitting prevents biases arising from the inherent order of the data, allowing for more robust training and evaluation. This randomization supports the generalization of the trained model to unseen data. While the initial allocation defines the sets, the images are shuffled at the beginning of each epoch to ensure variability in the data sequence seen by the model during the training process.

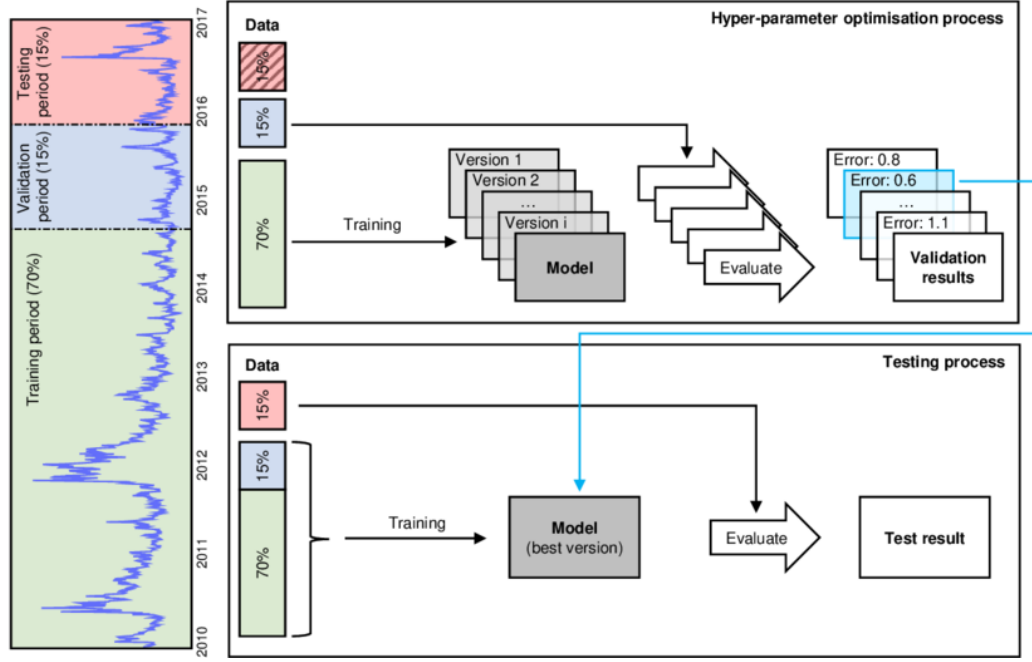


Figure 3.5: Random data splitting method [25].

3.2.4 Annotation Process

Annotation in this experiment consisted of labeling every image and the corresponding knowledge with organized identifiers in a systematic manner. This ensured the dataset could be easily organized, accessed, and processed for supervised learning. Beyond dividing the 34 species of medicinal plants into classes, the annotation encoded the frequency at which each plant was found in naturally weedy or unmanaged habitats to provide domain-specific context to each label. For the visual modality, a rank-aware naming scheme of the form `rank_serial_plantname` was used. Here, the *rank* (1–7) represents empirical availability in weedy fields, the *serial* (1–34) denotes the fixed index of the species in the database, and the *plantname* is the standardized local/common name in lowercase without spaces (e.g., `1_1_thankuni`, `4_18_joba`, or `7_34_betal` as shown in Figure 3.6). These rank assignments were based on repeated field observations within weedy Bangladeshi habitats and remained constant across all images of a particular species. This convention allowed folder hierarchies to be automatically parsable, making data loading, class balancing, and performance evaluation straightforward. Annotation also involved a manual verification process. All labeled images were cross-referenced with the master species list and field notes to ensure that observable morphology—including leaf shape, venation, and margin—correctly matched the assigned class and its rank of availability.

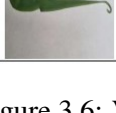
Sl	Image	Annotation	Sl	Image	Annotation	Sl	Image	Annotation
1		1_1_thankuni	13		4_13_tulsi	25		6_25_amloki
2		1_2_pathorkuchi	14		4_14_ramtulshi	26		6_26_haritoki
3		1_3_apang	15		4_15_basok	27		6_27_bohera
4		1_4_kalomegh	16		4_16_neem	28		6_28_oshwagandha
5		2_5_kaluchitra	17		4_17_pudina	29		6_29_devils_backbone
6		2_6_ayapan	18		4_18_joba	30		7_30_lemongrass
7		2_7_akondo	19		4_19_sojina	31		7_31_aloe_vera
8		2_8_bamonhati	20		5_20_gandha	32		7_32_nayontara
9		3_9_kalodhutra	21		5_21_shotomuli	33		7_33_longevity_spinach (gainura)
10		3_10_tarulata	22		6_22_sarpagandha	34		7_34_betal
11		4_11_punarnava	23		6_23_anantamul			
12		4_12_agnishikha	24		6_24_chapalish			

Figure 3.6: Visualization of the 34 medicinal plant classes with their rank-aware identifiers [2].

3.2.5 Dataset Description

The dataset, encompassing 17,050 images across 34 classes (Figure 3.7), serves as a foundational resource for advancing computer vision research, particularly in the realm of MediNet-XG. With an average of approximately 500 images per class, this collection provides a robust foundation for training and testing machine learning algorithms effectively. The predomi-

nant use of the .jpg format ensures seamless integration and compatibility across diverse platforms and frameworks. Figure 3.7 displays the class distribution; the y -axis lists the 34 medicinal plant classes, while the x -axis indicates the total count of images within each class.

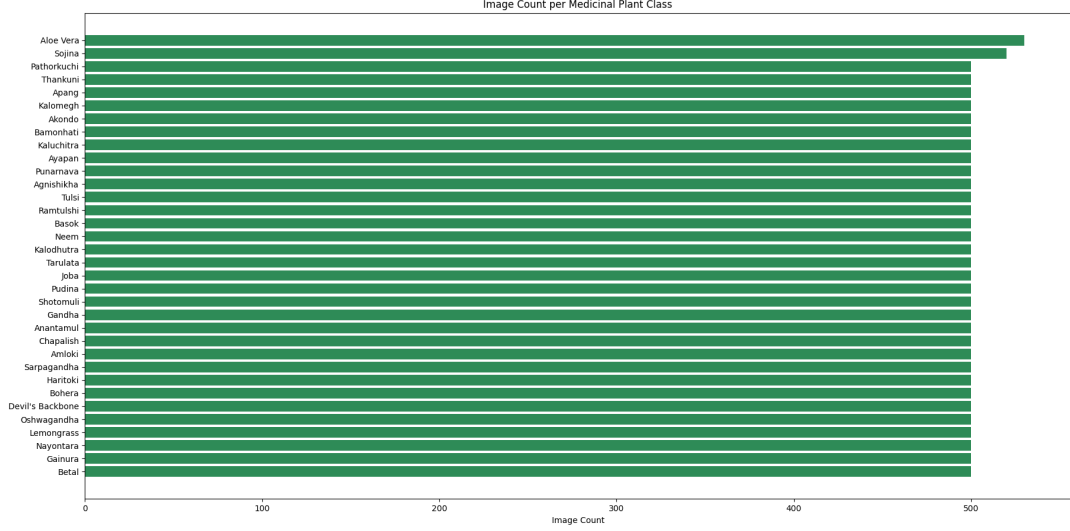


Figure 3.7: Class distribution in the dataset.

Each bar's width signifies the number of images in that particular class. It is evident that the dataset is relatively balanced, with the number of images per class ranging between approximately 500 and 530. However, slight variations exist, with two classes having slightly more images than others. The balanced nature of this dataset suggests that it is suitable for training machine learning models, reducing the likelihood of bias towards any particular class. Class 7_31_aloe_vera and 4_19_sojina stands out with the two highest image count, totaling 530 and 520 images, accounting for about 3.1% and 3.05% of the dataset. Conversely, rest of the classes contain 500 images, representing approximately 2.94% of the dataset. This balanced distribution across classes supports comprehensive exploration across a spectrum of visual recognition tasks, ranging from image classification to object detection and pattern recognition. Moreover, the dataset is enriched by data collected from 5 different districts and a public repository, ensuring a balanced geographic medicinal plant collection representation that enhances the dataset's inclusivity and applicability to real-world scenarios. This diversity not only mitigates potential biases but also broadens the dataset's utility in addressing varied perspectives and contexts. By providing such a diverse and meticulously structured dataset, researchers are empowered to push the boundaries of computer vision capabilities. This dataset not only fosters technical advancements but also promotes ethical considerations, contributing to a more equitable development of artificial intelligence.

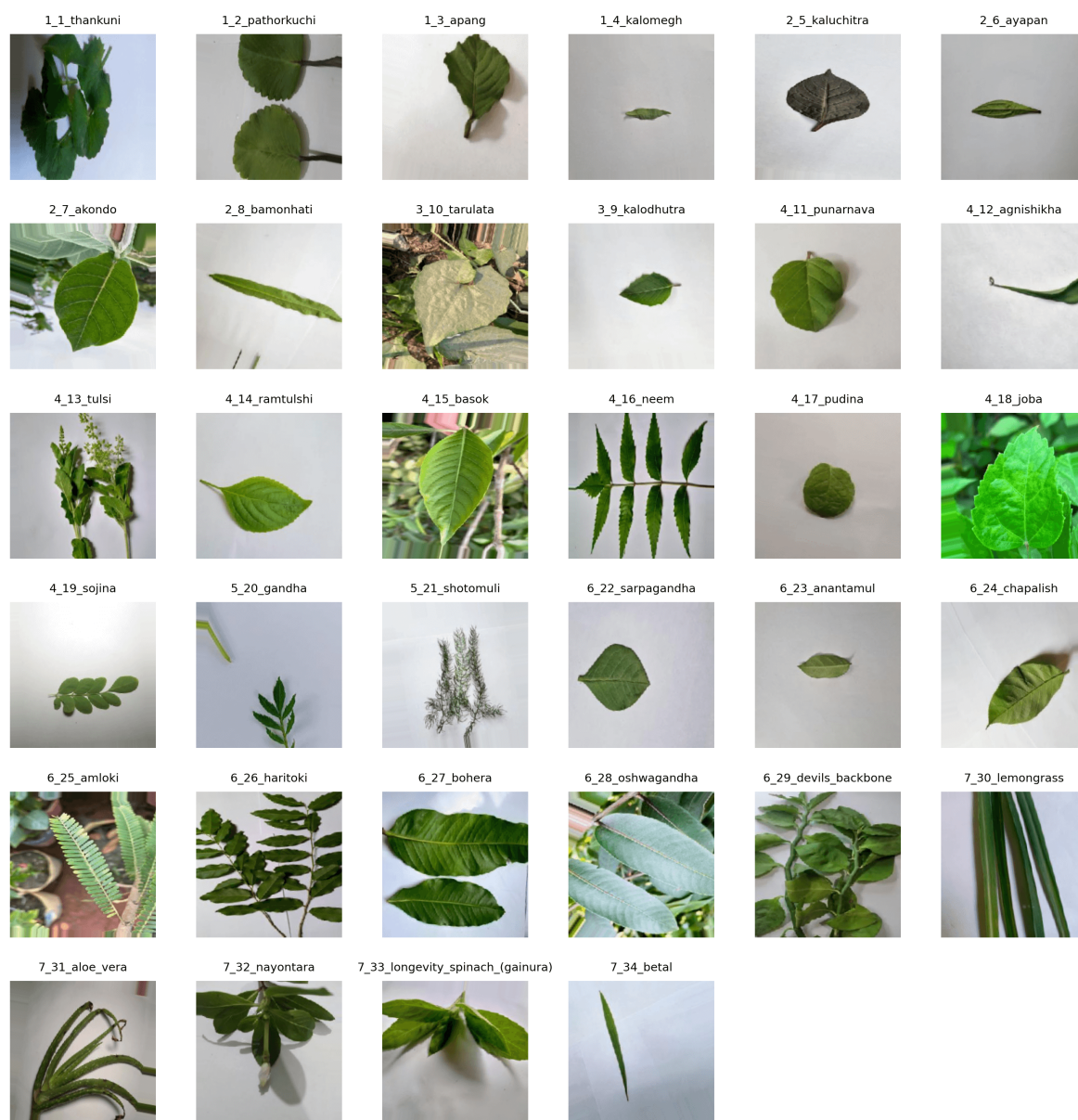


Figure 3.8: Dataset annotation.

The provided JSON file serves as a comprehensive knowledge base for 34 medicinal plant species each of containing 30 different, structured to facilitate grounded and safety constrained responses to user queries. For each entry, it provides formal botanical data, including scientific names, families, and specific leaf morphology. Beyond taxonomy, the dataset details pharmacological profiles such as primary active compounds and therapeutic properties, while emphasizing safety through specific toxicity levels, contraindications, and pregnancy safety guidelines. It also includes practical administration data, covering standard dosages and both internal and external preparation methods, such as decoctions or poultices. This structured approach ensures that when a plant is predicted, the system can return authoritative

botanical and pharmacological information while imposing critical safety constraints on the textual output.

```

P> | Semester | Research | Thesis - 2 | Book | Github-MedNet-XG | Dataset LLM | { } | leaf_datajson | { } | 1_1_thankuni |
1 {
2   "1_1_thankuni": {
3     "Scientific Name": "Centella asiatica",
4     "Botanical Family": "Apiaceae",
5     "Genus & Species": "Centella / asiatica",
6     "Local/Vernacular Names": "Bengali: Thankuni; English: Indian Pennywort, Gotu Kola; Hindi: Brahmi",
7     "Leaf Morphology": "Orbicular-reniform, 3-5cm across, crenate or sub-entire margins, glabrous, radiating nerves",
8     "Plant Habit": "Herb",
9     "Life Span": "Perennial",
10    "Geographical Origin": "Indian Subcontinent",
11    "Native Habitat": "Tropical wetlands, marshy areas, tropical forests",
12    "Global Distribution": "India, Bangladesh, Sri Lanka, Southeast Asia, Africa",
13    "Preferred Soil Type": "Loamy, well-drained, moist soil",
14    "Climate Requirements": "Tropical, 20-30u00b0C, high humidity, 1500-2500mm rainfall annually",
15    "Primary Active Compounds": "Asiaticoside, madecassoside, asiatic acid, madecassic acid",
16    "Secondary Metabolites": "Flavonoids, phenolic acids, triterpenes, alkaloids",
17    "Essential Oils": "Trace amounts of volatile oils",
18    "Nutritional Profile": "Vitamin C, iron, potassium, calcium, magnesium, amino acids",
19    "Therapeutic Properties": "Cognitive enhancement, wound healing, anti-inflammatory, antioxidant, anxiolytic",
20    "Traditional Uses": "Ayurvedic brain tonic, memory enhancement, skin conditions, fever",
21    "Modern Clinical Uses": "Treatment of cognitive disorders, varicose veins, cellulite, anxiety management",
22    "Medicinal Part Used": "Whole aerial part, leaves preferred",
23    "Standard Dosage": "3-6g dried herb daily, or 500-1500mg extract daily",
24    "Internal Preparation Method": "Decoction, infusion, powder capsules, standardized extract",
25    "External Application Method": "Poultice, herbal paste, oil infusion, topical cream",
26    "Culinary Uses": "Edible leaf in salads, herbal teas, smoothies, traditional curries",
27    "Extraction Process": "Aqueous extraction, alcoholic extraction (60-70% ethanol), CO2 extraction",
28    "Toxicity Levels": "Very low toxicity; LD50 >5000 mg/kg in animal models",
29    "Contraindications": "Pregnancy (large doses), lactation (caution), estrogen-sensitive tumors",
30    "Pregnancy & Lactation Safety": "Avoid high doses in pregnancy; minimal use acceptable in lactation under supervision",
31    "Drug-Herb Interactions": "May enhance effect of sedatives; possible interaction with anticoagulants",
32    "Conservation Status": "Common, IUCN: Not evaluated (widespread, cultivated)"
33  },
34  "1_2_pathorkuchi": {
35    "Scientific Name": "Bacopa monnieri",
36    "Botanical Family": "Plantaginaceae",
37    "Genus & Species": "Bacopa / monnieri",

```

Figure 3.9: Knowledge based JSON file structure.

3.3 Pre-trained Models for Comparative Analysis

3.3.1 EfficientNetB0

The strong baseline model chosen is EfficientNetB0 that has been chosen with the aim of attaining high-precision image recognition with a low computational complexity. Being the smallest member of the EfficientNet family, the EfficientNetB0 is built upon Mobile Inverted Bottleneck (MBConv) blocks with depthwise separable convolutions and squeeze-and-excitation (SE) channel attention module.

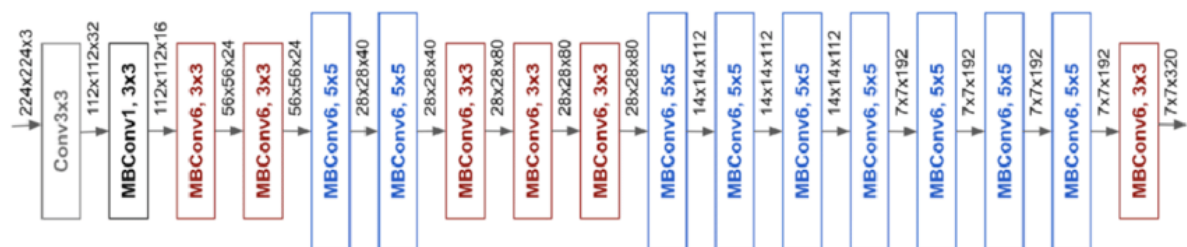


Figure 3.10: The EfficientNetB0 architecture [26].

The architecture allows the model to provide competitive performance using around 5 million parameters when using an input resolution of 224×224 as shown in Figure 3.10. Another notable characteristic of EfficientNetB0 is that network depth, width, and input resolution are balanced and unified when increasing, unlike being independent of each other. This methodology of scaling is useful in offering a good trade-off between accuracy and computational efficiency. EfficientNetB0 is therefore an appropriate and dependable backbone of transfer-learning especially when dealing with small and medium data sets where resource constrained and model efficiency are a major concern.[26].

3.3.2 DenseNet121

DenseNet-121 is a 121-layer deep convolutional neural network (CNN), the main feature is dense connectivity, that is, each deep-network layer gets input not only from the immediate predecessor layer but also from all previous layers in a dense block. DenseNet121 is selected for its unique dense connectivity pattern, where each layer is directly connected to every other layer in a feed-forward manner. The architecture promotes extensive feature reuse and facilitates robust gradient flow throughout the network, enabling efficient learning of complex representations while minimizing the number of parameters. The compact design of DenseNet121 (Figure 3.11), coupled with its demonstrated high accuracy, renders it particularly suitable for environments constrained by computational resources yet demanding exceptional performance.

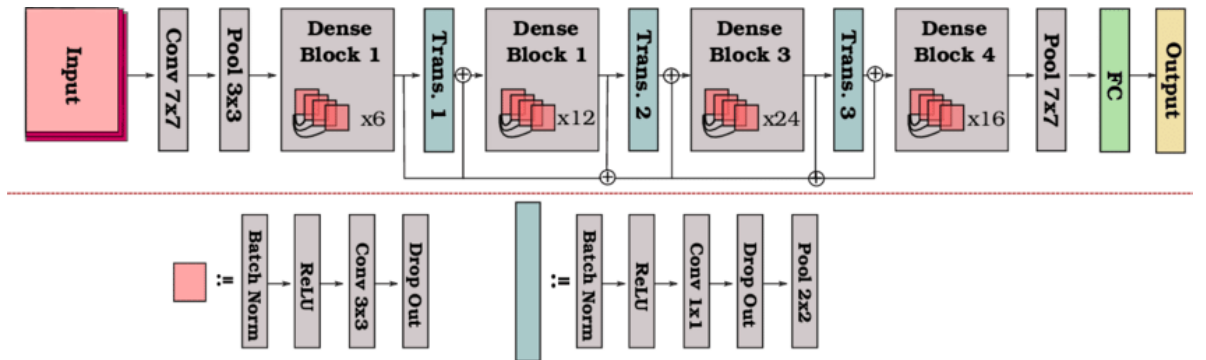


Figure 3.11: The DenseNet121 architecture [27].

The efficient layer connections enhance information propagation and mitigate issues like vanishing gradients, ensuring stable and effective training even with deep networks. This makes DenseNet121 an ideal choice for applications requiring high performance with limited computational power, providing a balance of speed, accuracy, and resource efficiency [27].

3.3.3 MobileNetV2

ResNet-50 uses shortcut connections, convolutional layers, and batch normalization to reduce vanishing gradients in deep networks. In contrast, MobileNetV2 was chosen for its lightweight and efficient design, which is critical in resource-constrained situations such as mobile and embedded devices (Figure 3.12). It has inverted residuals and linear bottlenecks, and it uses lightweight depthwise separable convolutions to reduce parameters and computing costs while keeping excellent accuracy [28].

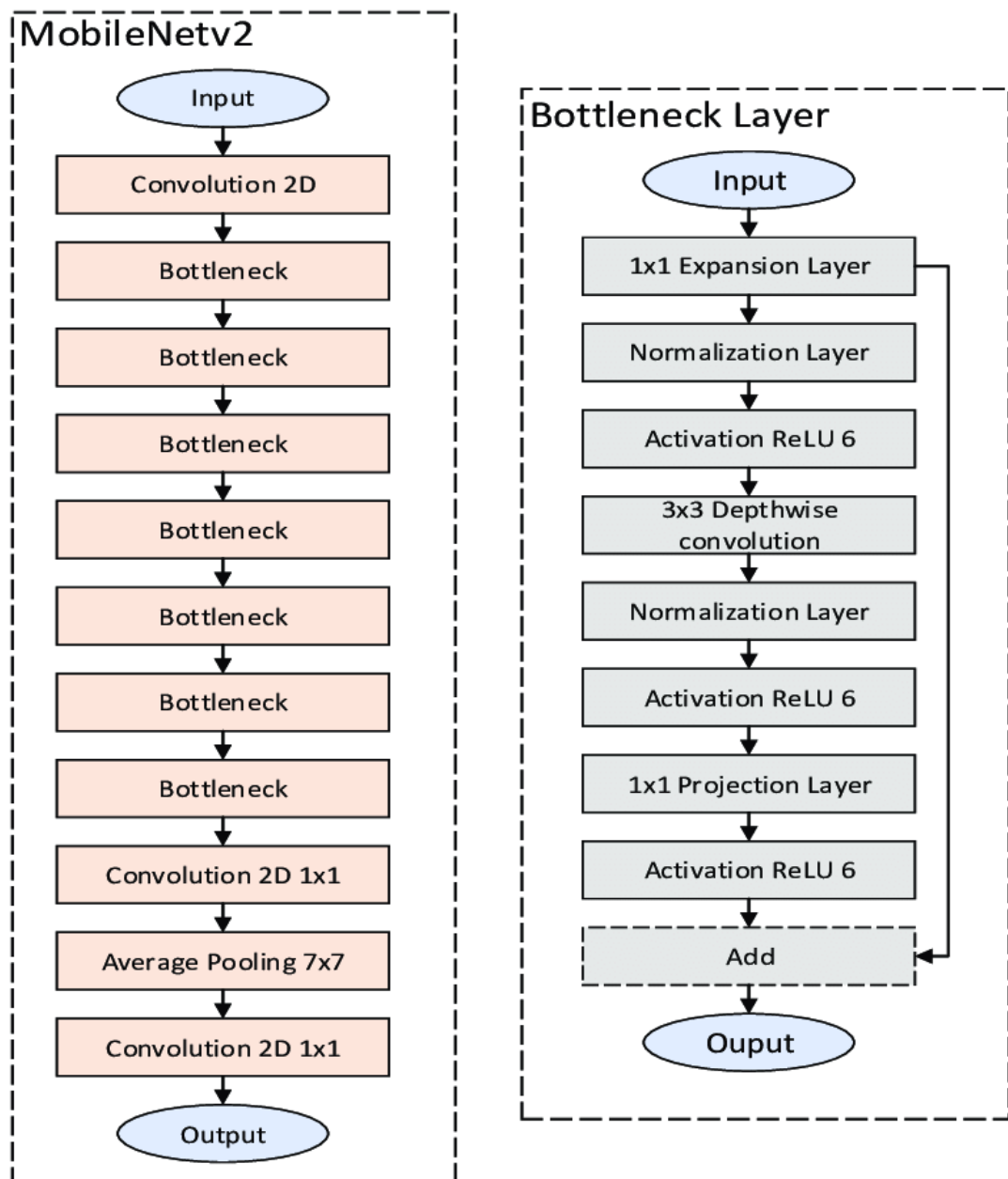


Figure 3.12: The MobileNetV2 architecture [28].

3.3.4 VGG16

VGG16 is chosen for its widely recognized and deeply layered architecture, serving as a base model for numerous image classification tasks. The straightforward design has multiple layers with 3x3 convolutional filters, which efficiently capture fine-grained details essential for distinguishing subtle nuances in leaf images. The architecture of VGG16 includes five convolutional blocks, each followed by max-pooling layers for spatial downsampling, culminating in three fully connected layers (Figure 3.13). Despite the depth of 16 layers, VGG16 remains computationally feasible for modern GPUs, ensuring practical deployment and performance across diverse backgrounds and applications requiring reliable image classification [29].

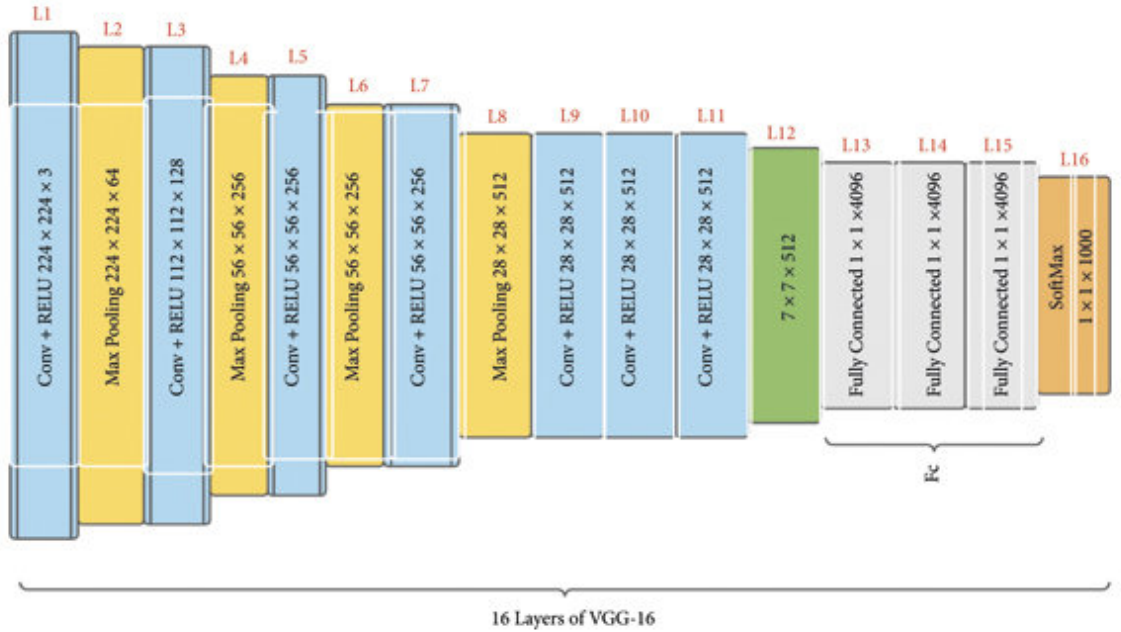


Figure 3.13: The VGG16 architecture [29].

3.3.5 SqueezeNet

SqueezeNet is another very thin convolutional neural network model that was created with a clear aim of highly reducing the count of trainable parameters but still remaining competitive in image classification. The Fire module is the fundamental component of SqueezeNet one that comprises of two consecutive parts that include squeeze layer and an expand layer. The squeeze layer uses 1x1 convolutions with an aim of minimizing the input channels and hence the dimensionality of the feature maps that are transferred to the subsequent layers. The model

is succeeded by the expanding layer that uses a mix of 1×1 and 3×3 convolutional filters to restore representational capacity at a minimal number of parameters. SqueezeNet is architecturally efficient and this is done by using three major design strategies. First, due to the wide application of 1×1 convolutions, the convolutional kernels are much smaller, which reduces the number of parameters. The second thing is the squeeze operation is used to reduce the number of input channels to the computationally expensive 3×3 convolutions. Thirdly, in the model - downsampling does not occur until later network stages, enabling higher-resolution feature maps to be stored in earlier layers, and so the accuracy of classification can be improved without more model size. That is how, the model SqueezeNet consequently is able to reduce parameters more than 50x relative to VGG-style networks and has similar accuracy on benchmark datasets (Figure 5.24). SqueezeNet is suitable to run on resource-constrained devices like mobile or edge devices due to its small memory footprint, efficient computation, and competitive performance. The attributes render it a suitable baseline architecture to test lightweight and explainable models including MediNet-XG, especially in applications where computational power and real time inference are essential factors.

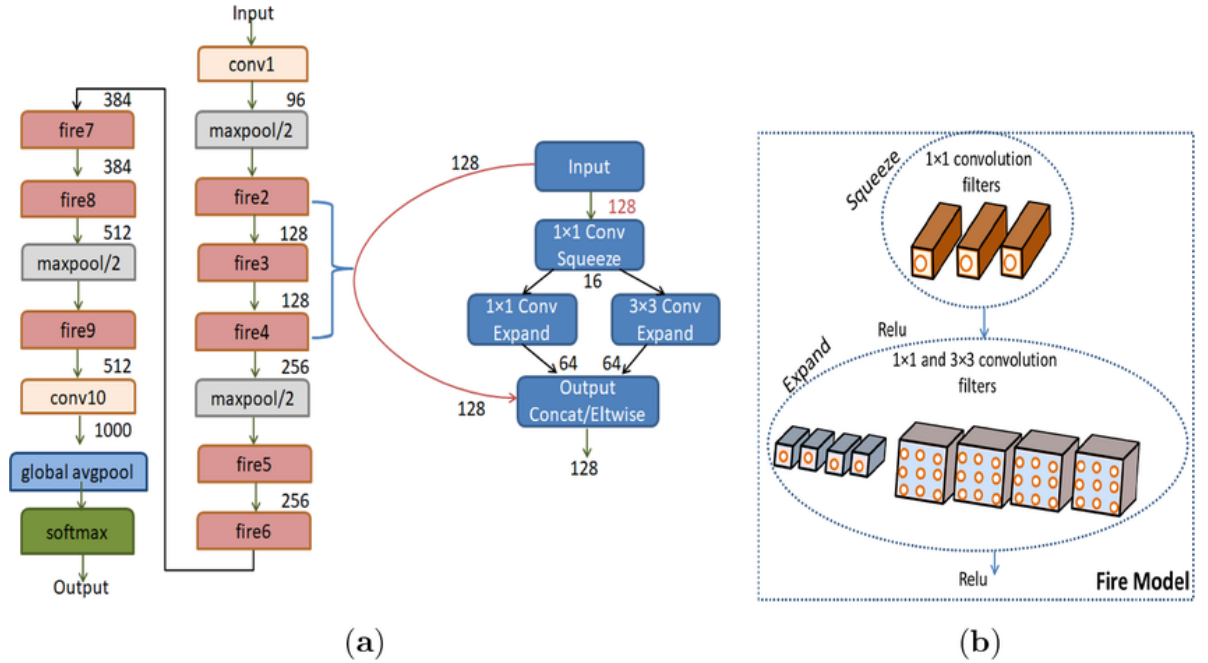


Figure 3.14: The SqueezeNet architecture [30].

3.4 Custom Model: MediNet-XG

The MediNet-XG framework is structured into three consecutive stages namely compact visual classification, explanation of reasoning and constrained knowledge retrieval (Figure

3.15). The MediNet classifier is offered in the initial stage, which takes a preprocessed RGB leaf image of a weed-infested setting and provides the calibrated probability distribution of 34 Bangladeshi medicinal plant classes along with top-K candidate labels. The second phase makes the prediction transparent, informative with Grad-CAM heatmaps, t-SNE visualisation of the features, and a JSON-based botanical knowledge lookup which presents only curated data on morphology, uses, compounds and safety. The high-level features of each image are obtained with MediNet, a decision is made, and Grad-CAM is used to show the regions of the image that provided the motivation to make this decision, e.g., the edges of the leaves or the veins. A low-dimensional representation of the same feature representation is then projected using t-SNE to ensure that the closeness to other species can be viewed qualitatively. Lastly, the predicted class identifier is looked up in a deterministic fashion against a structured JSON record, which contains pre-defined fields to retrieve in response to user queries, so that all of the textual responses are knowledge-based and aware of safety instead of generative or speculative.

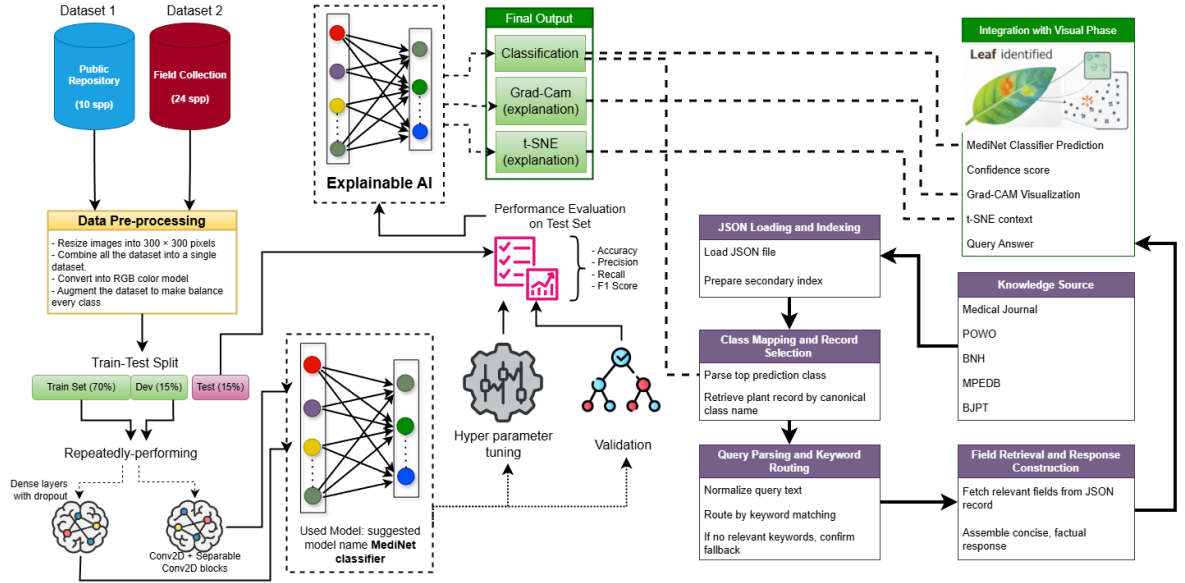


Figure 3.15: Abstract View of MediNet-XG.

3.4.1 MediNet classifier

The proposed framework is the MediNet classifier, developed as a small convolutional backbone that is optimized to identify medicinal plants in the weed-infested field with limited computational capacity. The architecture focuses on a low memory footprint, but is compatible with explainability techniques, and is therefore suitable to be deployed on mobile and edge

devices. The input images are firstly taken to do downsampling to a constant spatial resolution and run through a shallow convolutional stem, after which a series of inverted residual blocks using depthwise separable convolutions are called. Moreover, a lightweight channel attention mechanism is integrated so as to highlight discriminative leaf features with a minimal impact of the computational cost. The network generates a rich feature representation, summarized by global average pooling and projected to a D – dimensional, $D = 34$ softmax output layer to produce the distribution of class probabilities. These products act as inputs to the next description and integration of knowledge modules. The architecture is influenced by the necessity to make sure that efficiency and representational capacity are balanced to allow the model to distinguish between similar structures of the leaf form without compromising on intermediate feature activity, which can then be treated by a post hoc interpretability technique like Grad-CAM and t-SNE. Such a design guarantees correct classification performance in addition to transparency, which are necessary in practice within field-level decision support systems.

Architecture

A MediNet architecture shown in Figure 3.16 has a simple encoder form of layout which progressively converts the 2D input image into a semantic compact form. The first level of a convolutional stem then carries out low-level feature extraction and downsampling of space followed by subsequent stages of a hierarchy of inverted block residual expansion that progressively deepens channels at the cost of spatial resolution.

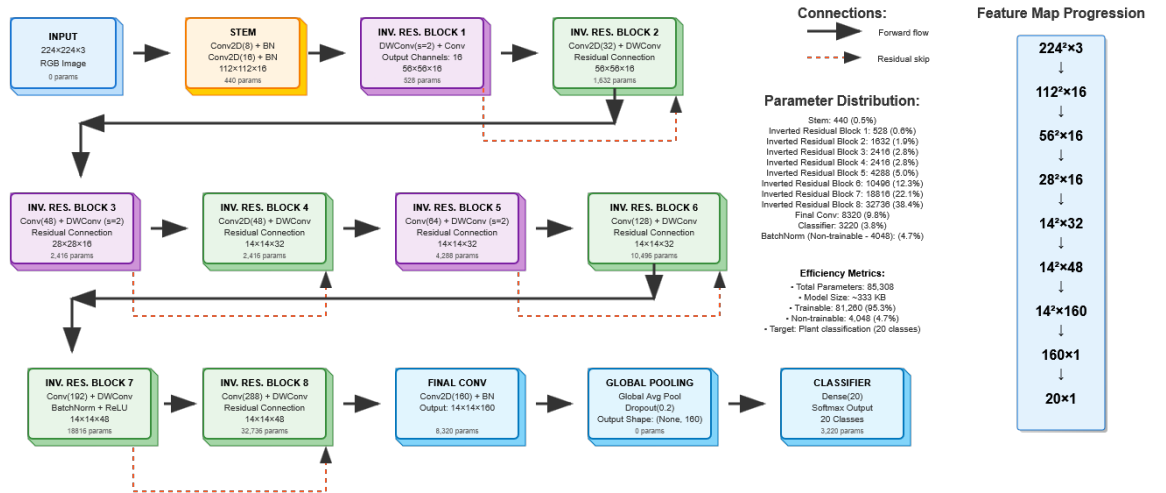


Figure 3.16: The MediNet classifier architecture.

Every step is defined by a desired number of output filters, a growth factor that controls the channel expansion within the block, a downsampling stride and a binary option that enables channel attention on a subset of blocks to highlight informative feature maps. Then a 1×1 convolution that increases the channel dimension to a larger bottleneck, after which the bottleneck is collapsed across spatial dimensions with global average pooling to create one feature vector; dropout regularization and a fully connected softmax classifier are the final parts of the architecture. Such an arrangement produces a model-sized classifier of the order of a few hundred kilobytes and is still capable of high accuracy on the 34-class medicinal plant dataset. The MediNet computation can be described as a mapping

$$f_{\theta} : \mathbb{R}^{H \times W \times 3} \rightarrow \Delta^{C-1}$$

where a normalized RGB image of size $H \times W$ is mapped to the probability simplex over $C = 34$ classes by a parameter set θ .

1. Input and rescaling layer

- **Input:** A standardized leaf image of shape $(224, 224, 3)$ following preprocessing.
- **Rescaling:** A rescaling operation divides pixel intensities by a constant factor to ensure values lie within a compact range. This stabilizes optimization and prevents scale-induced gradient issues.
- **Normalization:** Conceptually, this step computes $x_0 = \frac{x}{255}$, where x denotes the raw image tensor, preparing the data for subsequent convolutional layers.

2. Initial convolutional stem

- A 3×3 convolution with a small number of output filters and stride 2 is applied to capture local edge, texture, and color patterns while halving the spatial resolution to $(112, 112)$.
- Batch normalization follows to normalize feature statistics across the batch, which supports faster convergence and reduces internal covariate shift.
- A ReLU6 activation is used to introduce nonlinearity while capping activations at a finite range, which is advantageous for low-precision hardware.
- This stem serves as a generic feature extractor, kept shallow to reduce parameters.

3. Inverted residual stages with depthwise separable convolutions

- The core of MediNet consists of multiple stages, each formed by inverted residual blocks parameterized by (F, t, s, a) , where F denotes the number of output filters, t the expansion factor, s the stride, and a a boolean controlling attention.
- Within an inverted residual block, three main steps are performed:
 - (a) **Pointwise expansion** (1×1): The input tensor with C_{in} channels is expanded to $C_{\text{exp}} = t \times C_{\text{in}}$ channels.
 - (b) **Depthwise convolution** (3×3): A kernel size 3×3 and stride s is applied channel-wise to reduce parameters.
 - (c) **Pointwise projection** (1×1): Projects features back to F channels, acting as a channel mixer.
- After the pointwise expansion, depthwise convolution and pointwise projection steps, a depthwise separable convolution can be expressed as a composition of depthwise and pointwise components. For an input feature map $X \in \mathbb{R}^{H \times W \times C}$, the depthwise step applies C different spatial kernels $K_{\text{dw}}^{(c)}$:

$$Y_{\text{dw}}(h, w, c) = \sum_{i,j} K_{\text{dw}}^{(c)}(i, j) X(h + i, w + j, c)$$

$$Y_{\text{out}}(h, w, k) = \sum_c K_{\text{pw}}(1, 1, c, k) Y_{\text{dw}}(h, w, c)$$

4. Lightweight channel attention within selected blocks

- In specific stages where the parameter a is true, a lightweight channel attention mechanism is inserted to recalibrate feature responses.
- This module computes a channel descriptor through global pooling, generating per-channel weights to emphasize informative channels.

5. Stage progression and spatial downsampling

- Strides greater than 1 are used sparsely to reduce spatial resolution down to progressively smaller feature maps.
- This progression balances the need for high-resolution spatial detail with the requirement for high-level semantic representations.

6. Final 1×1 convolutional bottleneck

- After the inverted residual stack, a final 1×1 convolution expands the channel dimension to a fixed high value, forming a global bottleneck feature tensor.

7. Global average pooling

- Collapses spatial dimensions into a vector $z \in \mathbb{R}^{C'}$ by averaging each channel:

$$z_c = \frac{1}{H'W'} \sum_{h=1}^{H'} \sum_{w=1}^{W'} F_c(h, w)$$

8. Dropout regularization

- A dropout layer randomly sets a fraction of elements in z to zero during training to mitigate overfitting on the 34-class dataset.

9. Dense softmax classifier

- The dense layer softmax classifier firstly maps the vector z to logits $u \in \mathbb{R}^c$ using $u = Wz + b$ and $c = 34$.
- Then the softmax function, which converts logits into class probabilities with the equation of:

$$p_i = \frac{\exp(u_i)}{\sum_{j=1}^{34} \exp(u_j)}$$

10. Intermediate feature exposure for explainability

- Feature maps are retained for Grad-CAM computations, allowing gradients to be traced back for model transparency.
- Bottleneck features are used to feed t-SNE, creating a 2D embedding space for class-level similarity analysis.

Through this layered design, MediNet achieves a balance between parameter efficiency, discriminative performance, and explainability readiness, forming the computational backbone of the broader medicinal plant intelligence system. Because of this novel backbone, the model achieve as least size with high performance.

MediNet Architecture Pseudocode

The pseudocode of MediNet forward inference is presented in an algorithm **Algorithm 1**, and uses the normalized RGB image as input and converts it using a lightweight convolutional stem and cascaded inverted residual modules with optional channel attention.

Algorithm 1 MediNet Forward Pass

Require: Input image $x \in \mathbb{R}^{H \times W \times 3}$

Ensure: Class probability vector $p \in \Delta^{C-1}$

```

1:  $x \leftarrow x/255$  ▷ Input rescaling
2: Initial Convolutional Stem
3:  $h \leftarrow \text{Conv}_{3 \times 3}(x, F_{\text{stem}}, \text{stride} = 2)$ 
4:  $h \leftarrow \text{BN}(h)$ 
5:  $h \leftarrow \text{ReLU6}(h)$ 
6: Inverted Residual Stages
7: for each stage  $(F, t, s, a, n)$  do
8:   for  $i = 1$  to  $n$  do
9:      $s' \leftarrow s$  if  $i = 1$  else 1
10:     $h \leftarrow \text{InvResBlock}(h, F, t, \text{stride} = s')$ 
11:    if  $a = \text{true}$  then
12:       $h \leftarrow \text{ChannelAttention}(h)$ 
13:    end if
14:  end for
15: end for
16: Bottleneck and Classification
17:  $h \leftarrow \text{Conv}_{1 \times 1}(h, F_{\text{bottleneck}})$ 
18:  $h \leftarrow \text{BN}(h)$ 
19:  $h \leftarrow \text{ReLU6}(h)$ 
20:  $z \leftarrow \text{GlobalAvgPool}(h)$ 
21:  $z \leftarrow \text{Dropout}(z)$ 
22:  $u \leftarrow Wz + b$ 
23:  $p \leftarrow \text{Softmax}(u)$ 
24: return  $p$ 

```

3.4.2 Grad-CAM

Grad-CAM has been employed to highlight spatial regions of each leaf image that most strongly influence the predicted medicinal plant class. Class-specific gradients are backpropagated from the softmax output to a selected final convolutional layer, and those gradients are used to compute a coarse localization heatmap over the feature maps. The resulting heatmap is resized and overlaid onto the original image so that attention to leaf margins, venation, or texture can be visually inspected, helping to verify that the model attends to meaningful botanical structures rather than background clutter.

Architecture

Grad-CAM[1] has been implemented as an auxiliary computation graph that shares inputs with MediNet, taps the output of the last convolutional layer, and also accesses the final class scores (Figure 3.17). A gradient tape or equivalent mechanism captures the derivatives of the chosen class score with respect to the convolutional feature maps and aggregates them into channel-wise importance weights before producing the final heatmap.

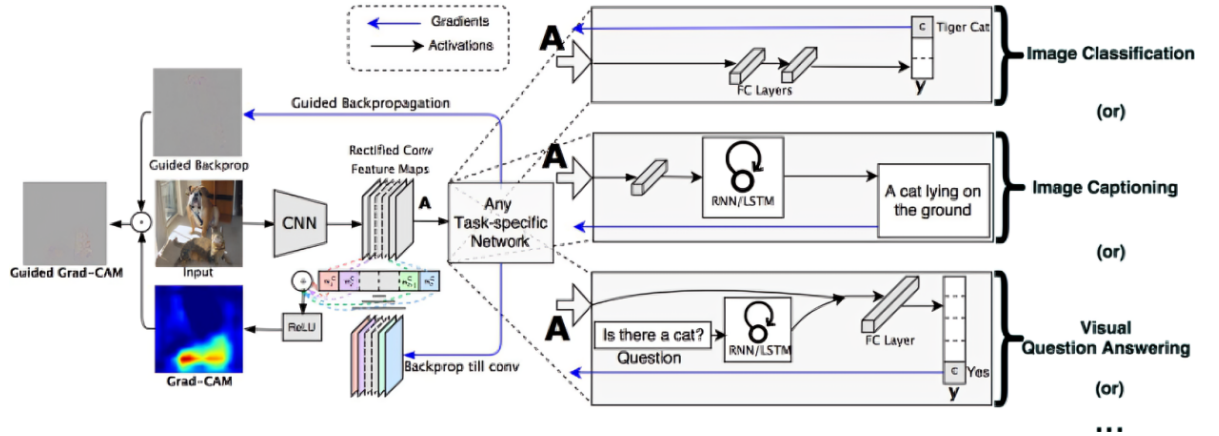


Figure 3.17: The Grad-CAM architecture[1].

The architectural layers-

- **Shared input and forward pass:** The preprocessed image is passed through MediNet until the last convolutional layer and the final classifier output are obtained.
- **Gradient capture:** A gradient operation computes partial derivatives of the predicted class score with respect to the last convolutional feature maps, yielding one gradient map per channel.

- **Channel weighting:** Each channel is assigned a scalar weight by averaging its gradients over spatial locations, producing importance coefficients that indicate how strongly each channel supports the class.
- **Weighted combination:** The feature maps are multiplied by their corresponding weights and summed across channels to form a single-class activation map, followed by a ReLU to retain only positive contributions.
- **Upsampling and overlay:** The activation map is normalized, resized to input resolution, color-mapped, and then superimposed on the original leaf image to create the Grad-CAM visualization.

3.4.3 t-SNE

t-SNE has been used to project high-dimensional MediNet feature vectors into a two-dimensional space, enabling qualitative inspection of how samples from the 34 medicinal plant classes cluster in the learned representation. Feature vectors extracted from an internal bottleneck layer are embedded with t-SNE so that visually similar species form nearby clusters, while distinct species appear further apart, providing a complementary perspective to per-class metrics and confusion matrices.

Architecture

t-SNE operates downstream of MediNet as an offline analysis module that takes as input a batch of fixed-length feature vectors and outputs 2D coordinates for each sample like Figure 3.18. A separate feature-extractor model has been defined to output the pre-pooled or pooled representation, and those vectors are passed into a standard t-SNE implementation configured with a modest perplexity and fixed random seed[31].

The architectural layers-

- **Feature extraction:** For each image in the evaluation set, MediNet is run up to a chosen internal layer (typically the bottleneck or pre-classifier layer) to obtain a feature vector of dimension C'
- **Batch assembly:** All feature vectors are stacked into a single matrix of size $N \times C'$,

where N denotes the number of samples considered for visualization.

- **t-SNE embedding:** The matrix is passed to the t-SNE routine, which computes pairwise similarities in the original space and iteratively optimizes 2D coordinates that preserve local neighborhood structure as much as possible.
- **Plotting and labeling:** The resulting 2D points are plotted with class-wise color coding or markers so that clustering behavior and overlaps among medicinal plant classes can be examined visually.

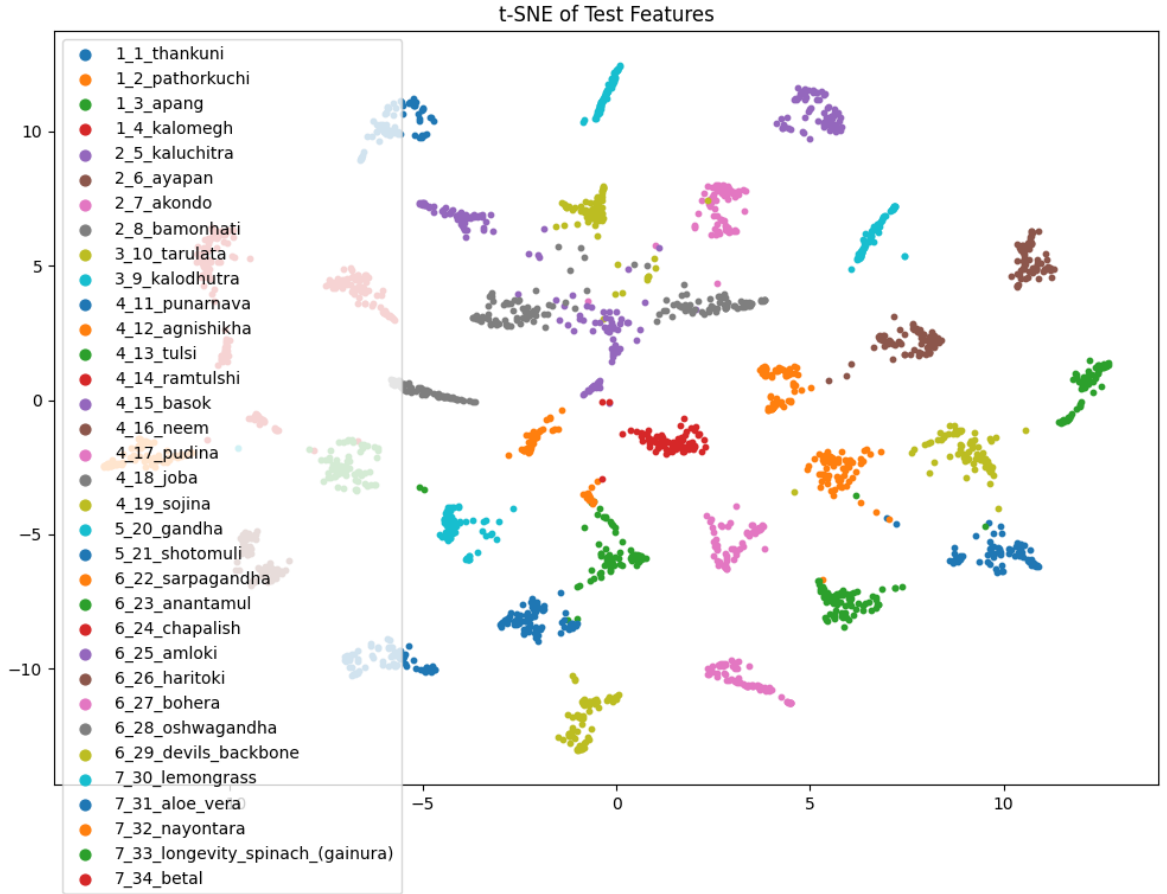


Figure 3.18: The t-SNE plot view.

3.4.4 Knowledge-based query generator from JSON

The knowledge-based query generator has been designed as a retrieval-only module that converts MediNet's predicted class and a user question into a structured, safety-aware explanation drawn strictly from the curated JSON knowledge base. Predictions are first mapped to canonical plant identifiers, after which the corresponding JSON record is consulted to provide in-

formation on scientific naming, morphology, medicinal uses, active compounds, toxicity, and pregnancy-related cautions without adding new medical claims. User questions are processed through simple keyword rules that select one or more relevant fields, and responses are formed by templated concatenation of those fields so that all output remains bounded by the original dataset.

Architecture

The query module is structured around three main components: a class-to-record mapper, a JSON loader and a keyword-based field router. The MediNet prediction provides a normalized label (for example, 4_18_joba to joba), which serves as the dictionary key for the JSON object that holds the plant’s knowledge entry. The user’s natural-language question is lowercased, tokenized, and matched against predefined keyword groups that correspond to specific JSON fields, and the selected fields are merged into a textual response template. No external calls or model-generated content are used; all outputs come directly from the values already present in the JSON file as shown Figure 3.19.

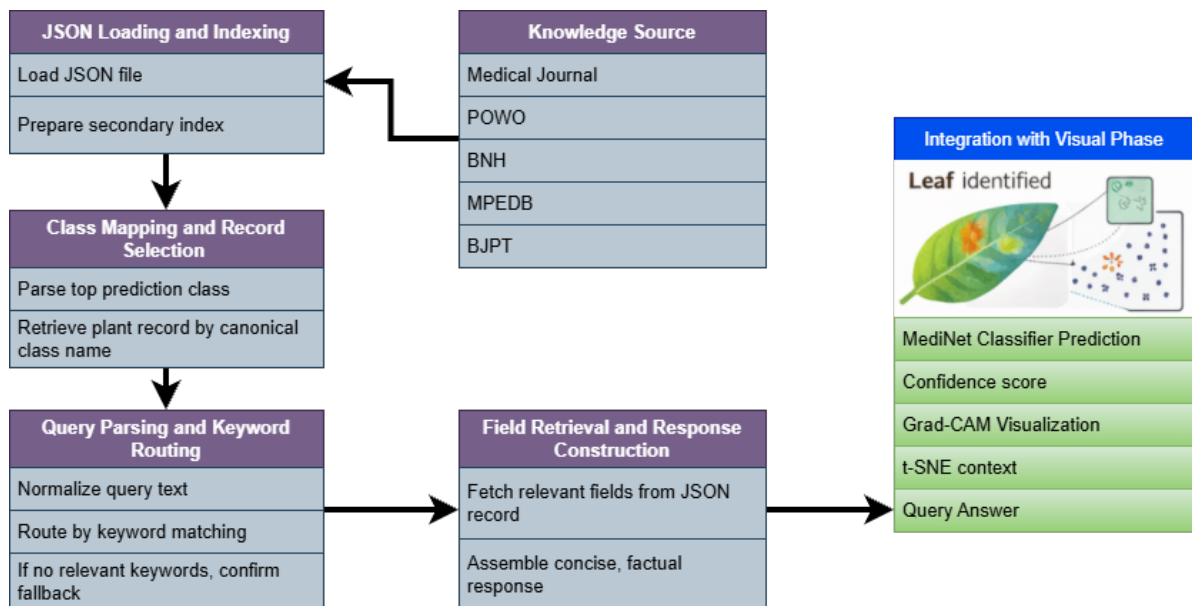


Figure 3.19: The query generator architecture.

The architectural layers-

- JSON loading and indexing: The JSON file is loaded into memory and stored as a dictionary keyed by canonical plant names that align with the classifier’s label space.
- Class-to-record mapping: The predicted class string (for example, 2_7_akondo) is nor-

malized to a base identifier (akondo) and used to retrieve the corresponding plant record from the dictionary.

- **Field schema access:** Each retrieved record exposes fixed fields such as Scientific Name, Botanical Family, Leaf Morphology, Traditional Uses, Modern Clinical Uses, Primary Active Compounds, Toxicity Levels, Contraindications, and Pregnancy Lactation Safety.
- **Query normalization:** The user query is lowercased, stripped of punctuation, and tokenized into basic terms to facilitate rule-based matching.
- **Keyword-to-field routing:** Detected keywords (for example, “use”, “compound”, “side effect”, “pregnancy”) are mapped to one or more JSON fields through predefined lookup tables.
- **Field retrieval:** The values of the selected fields are read directly from the JSON record without modification or inference.
- **Response templating:** A short textual answer is assembled by placing the selected field contents into simple templates that may also mention the plant’s scientific name and family for context.
- **Safety enforcement:** When safety-related keywords are present, the response emphasizes toxicity, contraindication, and pregnancy-safety fields and avoids generating new dosage or treatment recommendations.
- **Fallback handling:** If no keyword group matches the query, a controlled message indicates that the requested information is not available within the knowledge base or is outside the supported scope.
- **Multimodal integration:** The final text answer is presented alongside the predicted class, confidence, Grad-CAM visualization, and t-SNE context to form a combined visual–textual explanation for the identified medicinal plant.

3.5 Summary

The research work structured a pairing of a locally developed lightweight custom model, MediNet-XG, with several transfer-learning baselines, all assessed on the same 34-class weedy medicinal plant dataset. The model is composed of three primary components. First, MediNet is a compact classifier constructed using inverted residual blocks and depthwise separable convolutions, which yield probability distributions across 34 Bangladeshi medicinal plant species from field leaf images captured with cluttered backgrounds. Second, an explainability phase utilizes Gradient-weighted Class Activation Mapping (Grad-CAM) on the final convolutional feature maps and t-distributed Stochastic Neighbor Embedding (t-SNE) on bottleneck embeddings, which visualize spatial attention to leaf structures and feature-space neighborhood connections between species. Third, a knowledge-based query generator associates the predicted class with a JSON record and processes user queries by accessing pre-defined botanical, pharmacological, and safety domains, thereby generating grounded and safety-constrained textual outputs. On top of the custom architecture, pretrained CNN baselines are used to compare recognition performance between the same data splits and preprocessing. VGG16 and DenseNet121 represent deeper, parameter-intensive architectures with high feature extraction performance, whereas EfficientNetB0_TL and MobileNetV2 are parameter-efficient transfer-learning backbones designed in favor of accuracy-efficiency trade-offs. The SqueezeNet adds another lightweight baseline with a significantly smaller number of parameters, allowing for a comparison with MediNet in the ultra-compact regime. Together, these models provide a range of accuracy, complexity, and memory footprints, allowing the proposed MediNet pipeline and its explainability-knowledge integration to be measured both quantitatively and qualitatively.

CHAPTER 4

Implementation

4.1 Overview

The development and training of MediNet-XG models entail several practical steps. Initially, the process involves meticulous data preparation, encompassing dataset collection, resizing, orientation correction, and augmentation techniques to bolster dataset diversity and ensure model resilience. Subsequently, the chapter delves into the selection and setup of pretrained CNN architectures like VGG16, DenseNet121, EfficientNetB0_TL, MobileNetV2 and SqueezeNet. Each model is customized for medicinal plant classification through the integration of specific classification layers and fine-tuning via transfer learning from ImageNet, optimizing their performance for this specialized task.

4.2 Technologies Used

The development and training of the MediNet-XG models involved leveraging a combination of hardware and software technologies to ensure efficient processing, model development, and training.

4.2.1 Hardware

Processor: Intel Core i5 8th Gen processor, providing computational power for running code and executing machine learning tasks.

Memory: 16GB RAM, which facilitated handling and manipulation of datasets and model training.

Storage: 256GB SSD storage and 1TB hard disk, offering fast read/write speeds crucial for managing large datasets and software applications, ensures efficient data handling.

4.2.2 Software

Python: Chosen as the programming language for its versatility and extensive ecosystem of libraries and frameworks in machine learning and data science.

Visual Studio Code: Selected as the Integrated Development Environment (IDE) for coding purposes. Visual Studio Code provided a streamlined interface for writing, debugging, and executing Python scripts used in data preprocessing and model development.

Jupyter Notebook: Utilized for conducting data preprocessing tasks. Jupyter Notebook's interactive environment facilitated exploratory data analysis, data manipulation, and visualization, crucial for preparing the medicinal leaf image dataset before training.

Kaggle: Employed for building and training the machine learning models. Kaggle provided access to a GPU T4 2, accelerating model training processes significantly compared to CPU-only computations. This cloud-based platform also ensured scalability and resource availability necessary for training complex convolutional neural networks such as VGG16, DenseNet121, EfficientNetB0_TL, MobileNetV2 and SqueezeNet.

4.3 Dataset Preparation

The dataset preparation section focuses on the painstaking measures that were taken to prepare the dataset in the MediNet-XG study. The process is done by first defining the 34 classes that cover the medicinal plants image of leaves whose are generally grows in weed infested area with visual similarities and are found in dirty environments. After the steps, the images go through stages of preprocessing, which standardize the format of the images and optimize the quality. Images are being resized into 300x300 pixels and the data augmentation methods are then used to both balance the overall dataset classes count and diversify the data. which guarantees sound model training. Lastly, the data is divided into separate training 70%, validation 15% and test 15% samples, which are necessary to assess the performance of the models and guarantee objective outcomes in practice. The second stage was to gather the 30 potential queries and answers of each class of plants. The datas were gathered in the form of text and was subsequently stored as a JSON file to be able to access the file at a faster rate through the model. All procedures in this process are essential in producing a quality dataset

that is true to the medicinal plant leaves. This holistic solution is meant to further development of medicinal plants classification and recognition technology, which will eventually elevate the medical industry and environmental science in terms of classification, recognition and explanation of medicinal plants.

4.3.1 Class Definitions

The dataset comprises 34 classes, each representing a unique medicinal species. These classes cover a comprehensive range of leaves, ensuring that the model can recognize a diverse set of plant used in MediNet-XG. Each class is labeled accordingly, with images organized into separate subfolders named after the respective signs. The classes include common leaves for medicinal plants. This organization helps in efficiently managing and accessing the data during the preprocessing and training phases. The dataset’s meticulous organization into 34 distinct classes, encompassing a wide array of medicinal plants, ensures comprehensive coverage across different shapes, colors, ages of leaves.

4.3.2 Data Preprocessing

To ensure consistency and improve the performance of models, a series of preprocessing steps were applied to the raw images. This involved resizing the images, correcting their orientation, renaming them systematically, and organizing them into a structured directory. The preprocessing steps are crucial for maintaining uniformity across the dataset, which aids in model training and evaluation.

Directory Setup: Directory setup has been handled by defining base paths for the original images, the resized dataset, and the final split dataset, followed by creation of all required directories if they do not already exist. Separate root folders for resized images and for the train, validation, and test splits ensure that preprocessing outputs are stored in a controlled, experiment-specific directory tree.

Iterate through Subfolders: Iteration through subfolders (classes) proceeds by scanning the base directory that contains one folder per medicinal plant class and skipping any entries that are not directories. For each valid class folder, a corresponding output class folder is created so that resized images remain grouped by species, preserving the label structure needed for

supervised training.

Resizing: Each image is opened, converted to RGB, and resized to a uniform spatial resolution of 300×300 pixels using the default PIL resize operation, which standardizes dimensions for all models in the study. This fixed target size matches the dataset description in the thesis.

Renaming: filenames are preserved when saving resized images and when copying them into train, validation, and test directories. As a result, original naming patterns remain intact, and class membership is encoded solely by the folder structure rather than by modified filenames.

Saving Processed Images: Saving processed images into class folders is performed in two stages: first, resized images are written to class-wise subfolders inside the dedicated resized-image directory, and then, during dataset splitting, copies of those files are placed into class-wise subfolders under the train, validation, and test roots according to the specified ratios. This two-step process yields a consistent, balanced dataset layout that can be consumed directly by Keras `image_dataset_from_directory` or equivalent loaders.

Process confirmation: Process confirmation is provided through simple console messages that report successful completion of the resizing loop and subsequent dataset splitting. These messages mark the end of preprocessing, indicating that the model-ready directory structure has been created and populated as intended.

4.3.3 Data Augmentation

To enhance the diversity of the dataset and improve model generalization, several data augmentation techniques were applied. Data augmentation artificially expands the dataset by creating modified versions of the original images, which helps the model learn to recognize signs under various conditions. The augmentation techniques applied include:

Random horizontal and vertical flips: Leaf images are flipped along the horizontal and vertical axes with specified probabilities, simulating variations in camera orientation and leaf positioning without altering species-specific shape or venation patterns.

Random rotations: Small-angle rotations are applied around the image center, which emulate realistic camera tilt and leaf orientation changes in the field, helping the models become invariant to modest rotational differences.

Random zoom and scale changes: Zoom-in and zoom-out operations adjust the effective scale of the leaf within the frame, exposing the network to both closer and slightly more distant views while maintaining the weed-infested background context.

Random shifts and translations: Width and height shifts move the leaf within the image window, so that discriminative features appear at different spatial positions, reducing sensitivity to exact centering.

Brightness and contrast adjustments: Intensity-related transformations modify brightness and contrast within controlled ranges, approximating variations caused by natural lighting, shadowing, and sensor exposure differences in outdoor acquisition.

Random shear or perspective-like distortions: Shear transformations introduce mild geometric distortion that mimics off-angle viewpoints, enabling the models to handle leaves observed from slightly oblique perspectives.

Composite augmentation pipeline: The above operations are chained in an augmentation function that is applied on-the-fly or during offline generation of augmented samples, producing multiple diverse variants per original field image while retaining the original class labels.

4.3.4 Data Splitting

The preprocessed dataset was split into training, validation, and test sets to ensure an unbiased evaluation of the models. The dataset was divided with an 70-15-15 ratio:

Per-class image listing and shuffling: For each medicinal plant class folder under the resized dataset directory, all image filenames with extensions .png, .jpg, or .jpeg are collected into a list and randomly shuffled. Shuffling at the class level prevents any ordering bias from affecting which images appear in the training or evaluation subsets.

Ratio-based split computation: The total number of images n_{total} in each class is used to compute per-class split sizes with fixed ratios of 70% for training, 15% for validation, and 15% for testing. Integer truncation is applied for the training and validation counts, and all remaining images are automatically assigned to the test subset, ensuring that the three splits are disjoint within each class.

Seed: A random seed is set to ensure reproducibility of the splits. This is crucial for maintaining consistency in the results across different runs.

Creation of split-specific class directories: For each of the three splits, a corresponding class subfolder is created under the train, val, and test root directories defined for the version-2 dataset. This yields a nested folder hierarchy where the first level encodes the split and the second level encodes the plant class, which can be directly consumed by directory-based dataset loaders.

Copying images into train/val/test trees: The selected filenames for each split are copied from the resized class folder into the appropriate split/class subfolder using file-level copy operations. Filenames are preserved during copying, so class membership is represented entirely by directory location rather than by filename encoding.

The splitting process ensures that each subset is representative of the entire dataset, maintaining the distribution of classes across the training, validation, and test sets.

4.4 Model Selection and Training

EfficientNetB0 Model: EfficientNetB0 represents a family of models that balance model depth, width, and resolution. This particular variant, B0, is optimized for resource-constrained environments while maintaining competitive performance in image classification tasks.

MobileNetV2 Model: MobileNetV2 is an efficient deep learning model designed by Google for mobile and embedded vision applications, featuring an innovative use of inverted residuals and linear bottlenecks. It balances speed and accuracy, making it ideal for resource-constrained environments.

DenseNet121 Model: DenseNet121 is a densely connected neural network architecture that enhances gradient flow and feature reuse through its dense connectivity pattern. Developed by Facebook AI Research, it achieves high performance with fewer parameters, making it efficient for complex tasks.

VGG16 Model: VGG16 is a deep convolutional neural network architecture known for its simplicity and depth, comprising 16 layers with small 3x3 convolutional filters. It was devel-

oped by the Visual Geometry Group at the University of Oxford and has become a standard in image classification tasks.

SqueezeNet: SqueezeNet is an ultra-lightweight convolutional architecture that achieves AlexNet-level accuracy with dramatically fewer parameters by aggressively using 1×1 convolutions and compact “fire” modules. It maintains competitive performance while keeping the model size very small, making it well suited for deployment on memory- and compute-constrained devices.

4.4.1 Building Pre-trained Models

The transfer-learning models have been constructed by placing a lightweight classification head on top of frozen pretrained backbones such as VGG16, DenseNet121, EfficientNetB0, MobileNetV2, and SqueezeNet. Each backbone outputs a high-level feature tensor that is first reduced by global average pooling to a single feature vector per image, then regularized with a dropout layer, and finally passed to a dense softmax layer that produces class probabilities over the 34 medicinal plant categories. Formally, the base network parameters remain non-trainable while only the added pooling–dropout–dense layers are optimized using Adam with a low learning rate and categorical cross-entropy loss, which stabilizes fine-tuning on the comparatively modest dataset size. For SqueezeNet, additional “fire modules” are used internally, where a 1×1 squeeze convolution reduces channel dimensionality and two parallel expand convolutions (1×1 and 3×3) reconstruct a richer representation; the outputs of these expand paths are concatenated along the channel axis, yielding expressive yet parameter-efficient feature maps that feed into the same transfer head.

4.4.2 Training Pre-trained Models

Model instantiation and freezing: For each backbone in the `base_models` collection, a corresponding classifier is built: SqueezeNet uses a custom constructor, whereas VGG16, DenseNet121, EfficientNetB0, and MobileNetV2 share the generic transfer head with global average pooling, dropout, and a softmax dense layer; base weights remain frozen to preserve pretrained features.

Early stopping on validation loss: An early-stopping mechanism monitors `val_loss` dur-

ing training and halts optimization when no improvement is observed for 7 consecutive epochs, restoring the best-performing weights, which reduces overfitting and unnecessary computation on the 34-class medicinal dataset.

Model checkpointing: A model checkpoint callback saves the parameter set that achieves the lowest validation loss for each architecture into a dedicated file, ensuring that the most generalizable version of every pretrained model is retained even if later epochs degrade performance.

Training loop and epochs Each model is trained on the same `train_data` generator with `val_data` as the validation source, over a fixed number of epochs defined by `EPOCHS`, under Adam optimization with a learning rate of 1×10^{-4} and categorical cross-entropy loss, while accuracy is tracked as the primary metric.

History and model storage Training histories, including loss and accuracy curves, are stored in a dictionary keyed by model name, and the final trained model instances are stored separately, enabling later comparison of convergence behavior and evaluation on the held-out test set.

4.5 Custom Model: MediNet-XG

The custom MediNet-XG (Medinet classifier + Explainable model as X + Generatable users query as G) model has been implemented as a compact convolutional network built from an initial 3×3 convolutional stem followed by multiple stages of inverted residual blocks with depthwise separable convolutions with ultra lightweight channel attention in selected stages, and a final 1×1 bottleneck layer, after which global average pooling, dropout, and a dense softmax head produce class probabilities over the 34 medicinal plant species. The architecture uses ReLU6 activations and carefully chosen strides to balance parameter efficiency with preservation of spatial structure, ensuring compatibility with Grad-CAM and t-SNE explainers that operate on the last convolutional feature maps and the pre-classifier bottleneck representation. The model, MediNet-XG can classify the 34 classes of medicinal plants with an accuracy of 98.82%, can explain why the class has been chosen and can answer any unbounded query about the plants in real time in low resource device for the model's size of (342KB) only.

4.5.1 MediNet classifier

MediNet-XG achieves a lightweight yet accurate design by combining width-multiplied inverted residual blocks, depthwise separable convolutions, and selective channel attention in a carefully staged architecture. Each block first expands channels only when needed, applies a 3×3 depthwise convolution for spatial encoding, and optionally passes through an efficient squeeze-and-excitation (SE) unit when the feature map is sufficiently wide. It then projects back to a compact channel size with a 1×1 convolution and a residual shortcut whenever input and output dimensions match. The architecture incorporates a small initial stem and a narrow width multiplier ($\alpha = 0.5$) applied to all stages. A single final 1×1 bottleneck, followed by global average pooling (GAP), removes bulky fully connected layers, significantly reducing the parameter count. Furthermore, modest dropout regularization is employed to preserve generalization. Together, these design choices keep the total parameter count in the few-hundred-kilobyte range without sacrificing discriminative power on the 34-class medicinal plant identification task.

4.5.2 t-SNE (X)

t-SNE integration has been enabled by defining a compact embedding model that outputs the global pooled feature vector from the `embedding` layer of MediNet-XG, `tf.keras.Model` is constructed with the same image input as the classifier but with the named embedding layer as output, so that high-dimensional representations for all samples can be extracted efficiently and then passed to the external t-SNE routine for low-dimensional visualization of class-wise structure in feature space.

4.5.3 Knowledge-based query generator from JSON (G)

A minimal prediction interface has been defined to convert raw image files into top- K class hypotheses for MediNet-XG and the baseline models. Each image is loaded from disk, resized to the desired input resolution, transformed into a float32 tensor in $[0, 1]$, and expanded to a batch of size one so that the classifier can be applied directly. The resulting probability vector is sorted in descending order, and the indices of the highest-scoring classes are mapped back to human-readable class keys, yielding a concise list of top- K candidate species together with percentage confidence values for downstream explanation and knowledge retrieval.

4.6 Summary

The MediNet-XG models are systematically implemented, with an emphasis on practical tools like Python for scripting, Visual Studio Code for development, and kaggle[2] for GPU-accelerated training. Diverse pretrained backbones are integrated alongside a custom lightweight architecture, aimed at enhancing accuracy in recognizing 34 medicinal plant species from cluttered field images, with a focus on advancing machine learning-driven botanical identification through robust explainability methods, such as Grad-CAM and t-SNE, and safety-constrained knowledge retrieval.

CHAPTER 5

Result and Analysis

5.1 Overview

A comprehensive analysis of the results obtained from various deep learning models applied to the medicinal plant classification, explanation and query response generation task. This includes detailed performance metrics such as accuracy, classification reports, loss per epoch graphs, accuracy per epoch graphs, AUC-ROC curves, and confusion matrices for each pre-train model: EfficientNetB0, DenseNet121, VGG16 and MobileNetV2. The study also evaluate the performance of the custom build model designed to enhance classification accuracy. By systematically comparing these models, the research aims to identify the strengths and weaknesses of each approach, providing insights into the most effective strategies for medicinal plant recognition. The findings from this chapter will highlight key performance trends.

5.2 Model Performance Overview

Performance metrics of multiple convolutional architectures were evaluated for 34-class on Bangladeshi medicinal plant recognition, including accuracy, precision, recall, and F1-score, together with model size to capture deployment cost. Among all candidates, MediNet-XG emerges as the top performer, achieving an accuracy of 98.82%, precision of 98.75%, recall of 98.87%, and F1-score of 98.79% with a footprint of only 342 KB, which demonstrates that the custom inverted-residual design yields both superior recognition quality and extreme compactness. DenseNet121 and VGG16 also show strong behaviour, with accuracies of 97.56% and 97.48% respectively and F1-scores above 96%, but their sizes (26.98 MB and 56.20 MB) indicate substantially higher memory demands that are less compatible with edge deployment scenarios. EfficientNetB0-TL and SqueezeNet maintain respectable performance (95.87–97.02% accuracy and F1-scores around 95.9–97.0%), offering moderate trade-offs between accuracy and model size, whereas MobileNetV2 yields noticeably weaker results, with

accuracy and F1-score around 65.7%, suggesting that a generic lightweight backbone does not capture the fine-grained leaf variations in cluttered weed-infested imagery as effectively as the task-specific MediNet-XG topology.

Table 5.8: Performance metrics of various DL models.

Model	Acc(%)	Loss	Pr (%)	Re (%)	F1(%)	Fβ(%)	Model Size
VGG16[29]	88.74	74.89	89.18	88.73	88.95	88.82	56.20 MB
DenseNet121[27]	97.88	8.50	97.89	97.87	97.88	97.87	26.98 MB
EfficientNetB0_TL[26]	2.90	352.53	0.09	2.94	0.17	0.40	15.48 MB
MobileNetV2[28]	96.23	5.35	96.23	96.22	96.22	96.22	8.64 MB
SqueezeNet[30]	89.21	31.69	89.54	89.13	89.33	89.21	889 KB
MediNet-XG	98.82	4.95	98.75	98.87	98.81	98.85	342 KB

5.3 Detailed Analysis

5.3.1 Loss and Accuracy per Epoch Graphs

EfficientNetB0

Training behaviour of EfficientNetB0 transfer learning (TL) model exhibits both stable but capacity-constrained convergence on the medicinal plant dataset. The training and validation loss values decrease steadily at the beginning with 3.57 and 3.53 respectively, consistency values, which remains stable during training, reflecting a well-regulated process of optimization, and no serious cases of overfitting take place. Conversely, the respective accuracy curves depict slower convergence as training accuracy oscillates around 3.0 percent with validation accuracy somewhat spiking every now and then, and not maintaining the spikes throughout epochs. These oscillations indicate stochastic effects in the batches as opposed to real gains in the separability of classes. Further improvement in loss and accuracy is marginal after about n epochs, which means that the network has stabilized into an operating regime, and it has recovered most of the discriminative structure that it can learn when trained using the current training conditions. All in all, the near matching of training and validation scores can be regarded as an indication of the sound generalization behavior, and the low absolute accuracy reflects the constraints of EfficientNetB0 TL in the context of fine-grained classification under the conditions of complex, weed-infested fields, which prompts the investigations of more specialized lightweight models.

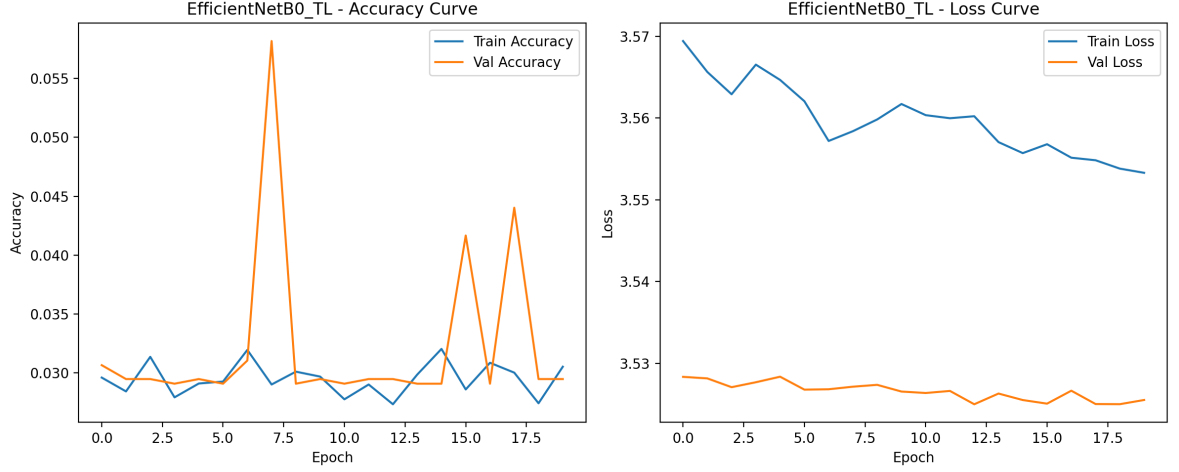


Figure 5.20: Loss and accuracy curve for EfficientNetB0.

MobileNetV2

For MobileNetV2, the loss trajectory starts at around 0.7 and steadily decreases to about 0.13 over the epochs. This decline reflects the model's improved ability to minimize prediction errors as it learns from the training data. Simultaneously, the accuracy metric begins around 68% and shows notable improvement, reaching approximately 95.95% by the end of training as shown in Figure 5.21. The stabilization of both loss and accuracy curves after about 30 epochs suggests that the model has converged to a stable performance level. This stability indicates that further training epochs may not significantly enhance performance, highlighting the effectiveness of the training process in optimizing MobileNetV2 for the image classification task.

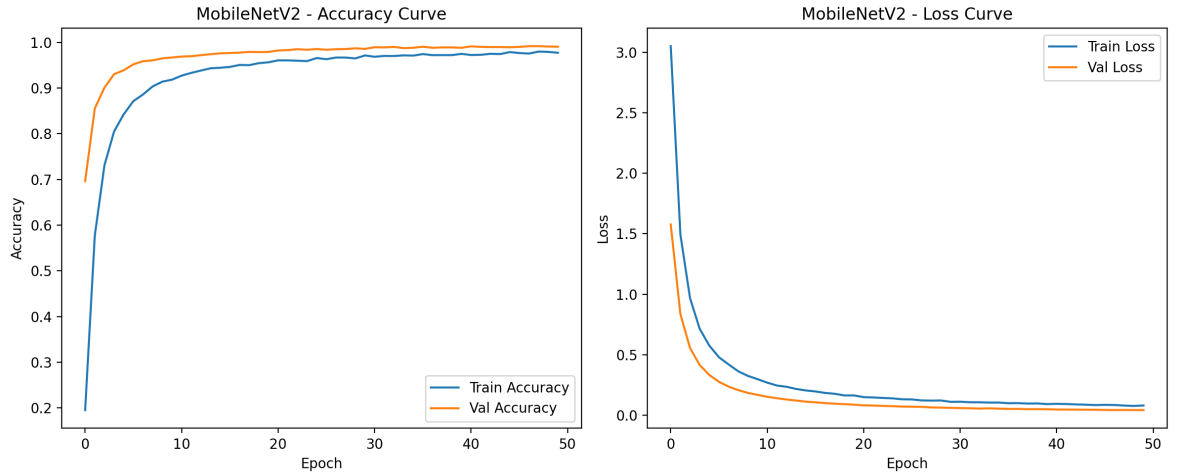


Figure 5.21: Loss and accuracy curve for MobileNetV2.

DenseNet121

The DenseNet201 model shows an initial loss of around 0.5, which decreases to about 0.08 by the end of the training, indicating effective error reduction throughout the process. The accuracy starts at around 80% and improves to approximately 97.67% shown in Figure 5.22, demonstrating the model's strong learning capabilities and ability to generalize well from the training data. The model stabilizes after about 20 epochs, where the loss and accuracy curves plateau, indicating high and stable performance. The smooth convergence of the loss and accuracy curves highlights the robustness and consistency of the DenseNet201 model. This behavior suggests that the model has effectively learned the underlying patterns in the data without overfitting. The efficient feature reuse and deep connections in DenseNet201 contribute to its superior learning capacity.

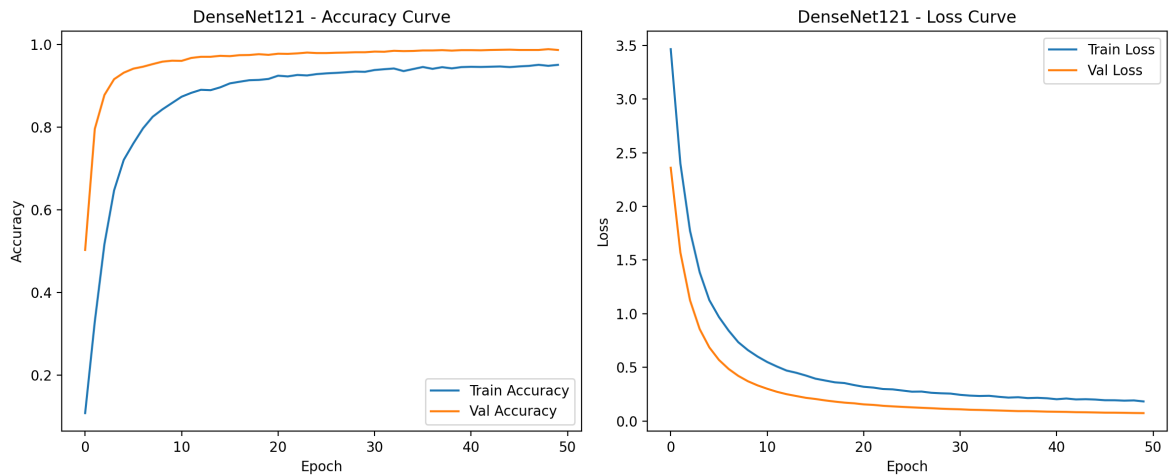


Figure 5.22: Loss and accuracy curve for DenseNet201.

VGG16

For the VGG16 model, In Figure 5.23, the training and validation accuracy and loss curve of VGG16 model has been shown over 50 epochs. Accuracy on training improves to roughly 5 percent in the first epoch and then to about 79 percent in the 50 th epoch and validation accuracy soars to around 14 percent, and then to almost 89 percent, after the 30 th epoch. The continuing difference of approximately 10 percent in support of validation accuracy indicates there is little regularization and there is no recollection of the training set.

Loss curves demonstrate a regular convergence, where training loss declines between 3.7 and 0.95, and validation loss declines between 3.3 and 0.75 at the end of the last epoch. The fact

that there is no loss divergence and the validation loss is smaller, during the training process, shows steady optimization and good generalization without noticeable overfitting.

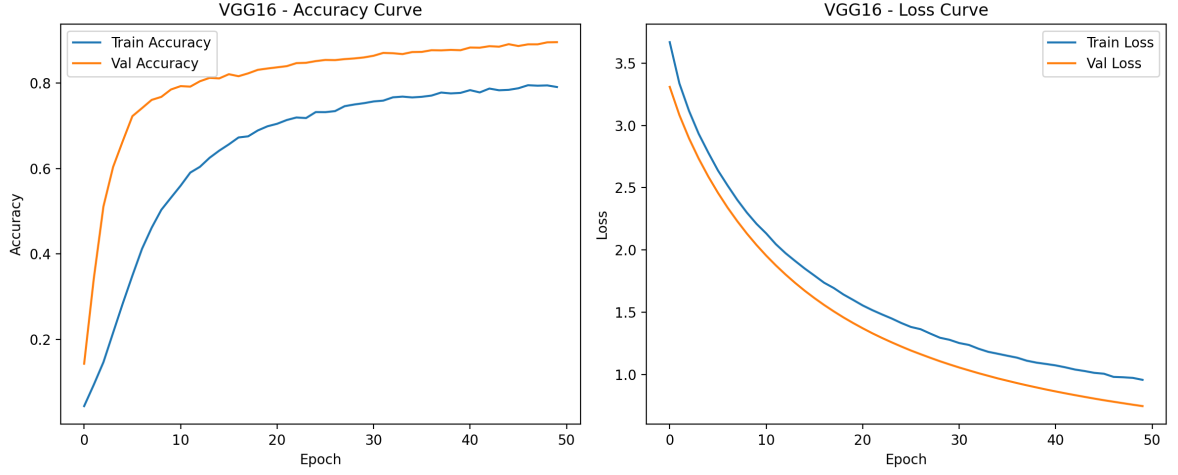


Figure 5.23: Loss and accuracy curve for VGG16.

SqueezeNet

The SqueezeNet model presents a smooth decline in loss over the training epochs. Figure 5.24 shows the validation and accuracy and loss curve of the SqueezeNet model, whose model size is 889 KB. The training accuracy rises to around 6 percent in the first epoch to around 85 percent in the 50th epoch whereas the validation accuracy goes up to almost 11 percent to an average of 90 percent. There is some oscillating validation accuracy in the early and mid training (epochs 5-20), due to the high sensitivity of the lightweight architecture to batch-level changes, but then it stops after epoch 30.

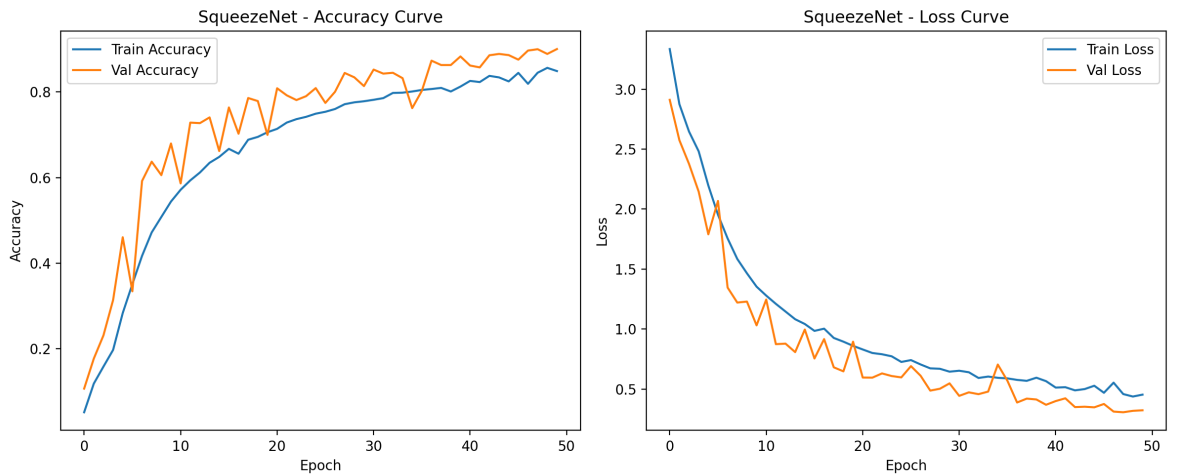


Figure 5.24: Loss and accuracy curve for SqueezeNet.

The loss curves indicate that the loss is rapidly converging in the early stages with training loss dropping to 0.45 and validation loss dropping to 0.32. Validation loss significantly decreases during training, which implies a high degree of regularization, as well as a high level of generalization. The steady decreasing pattern without any deviation validates the existence of stable optimization and proves that the miniature 889 KB SqueezeNet model can be deployed at a competitive performance without bearing a large memory and computational burden.

5.3.2 AUC-ROC Analysis

A detailed comparative analysis of several advanced deep learning models based on their AUC-ROC curves, a pivotal metric in assessing classification performance is represented in Figure 5.25.

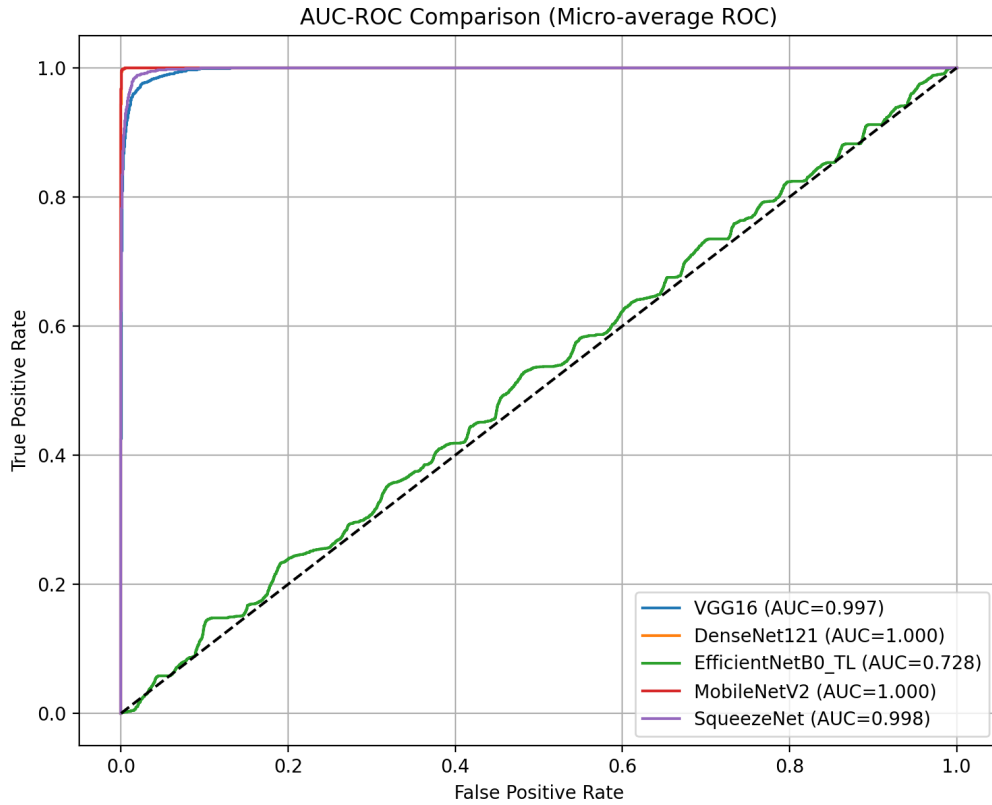


Figure 5.25: AUC-ROC curves for different models.

The robust performance of effectiveness in handling complex datasets and reinforces its position as a top choice in applications requiring nuanced classification abilities. Following closely are VGG16, DenseNet121, MobileNetV2 SqueezeNet, all demonstrating strong and

reliable classification capabilities as evidenced by their commendable AUC values. These models excel in capturing intricate patterns and variations within data, making them invaluable tools in fields such as medical diagnostics, autonomous systems, and natural language processing where accurate classification is paramount. Despite marginally lower AUC scores compared to the top performers, the EfficientNet exhibits slightly dissatisfactory, noteworthy performance metrics, don't affirm the robustness and versatility in diverse classification tasks.

5.4 Custom Mode: MediNet-XG

5.4.1 MediNet Classifier

The MediNet classifier model achieved an impressive accuracy of 0.9838, indicating its high predictive performance. Additionally, with a low loss of 0.1008, the model demonstrates strong reliability and minimal prediction error.

MediNet classifier Model Accuracy: 0.0.9837754261264589

MediNet classifier Model Loss: 0.1008325461891258

The report provides detailed insights into the precision, recall, F1-score, and support for each class of Ensemble-1. Such metrics present clear view of the model's effectiveness.

Table 5.9: Classification report for MediNet-XG.

Class	Precision	Recall	F1-Score	Support
0	0.96	1.00	0.98	75
1	0.96	1.00	0.98	75
2	0.99	0.91	0.94	75
3	0.96	0.99	0.97	75
4	0.99	1.00	0.99	75
5	1.00	0.99	0.99	75
6	0.95	0.99	0.97	75
7	1.00	1.00	1.00	75
8	0.91	0.97	0.94	75
9	1.00	0.97	0.99	75
10	0.94	1.00	0.97	75

Class	Precision	Recall	F1-Score	Support
11	1.00	0.96	0.98	75
12	1.00	0.95	0.97	75
13	0.96	1.00	0.98	75
14	0.91	0.93	0.92	75
15	1.00	1.00	1.00	75
16	1.00	1.00	1.00	75
17	0.95	0.93	0.94	75
18	1.00	1.00	1.00	78
19	1.00	0.99	0.99	75
20	1.00	1.00	1.00	75
21	0.97	0.96	0.97	75
22	0.99	1.00	0.99	75
23	0.99	1.00	0.99	75
24	1.00	0.97	0.99	75
25	0.99	0.92	0.95	75
26	0.96	0.99	0.97	75
27	0.99	0.92	0.95	75
28	0.95	0.95	0.95	75
29	1.00	1.00	1.00	75
30	1.00	1.00	1.00	80
31	0.96	0.93	0.95	75
32	0.97	1.00	0.99	75
33	0.97	0.97	0.97	75
Accuracy			0.98	2585
Macro Avg	0.98	0.98	0.98	2585
Weighted Avg	0.98	0.98	0.98	2585

5.4.2 Grad-CAM (X)

Grad-CAM analysis shows how MediNet-XG uses high-level class scores to generate spatially localized evidence of decisions on the leaf surface. To make each prediction, the gradient of each target class logit with respect to the final convolutional feature maps is calculated,

channel-wise gradients are averaged across all channels of the global gradient, a weighted sum of feature maps is passed through a ReLU and upsampled to the input resolution to yield a class-discriminative heatmap, which is used to encode the importance of each pixel position in the decision. When image classifications are correct, the resulting activation is strongly focalised around the lamina, margin and venation areas, showing that the decision rule of the model is driven by internal filters that react to leaf morphology but not by the background textures, whereas in ambiguous or misclassified images the heatmaps become diffused or decentralised, revealing exactly which regions of the learned representation space are not clearly separated by the feature hierarchy.

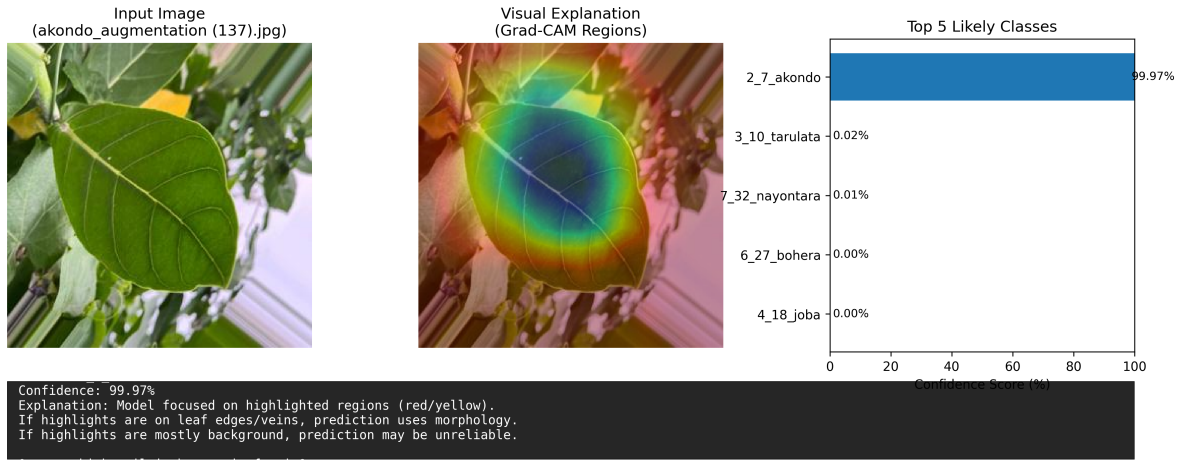


Figure 5.26: Grad-Cam explanation.

5.4.3 t-SNE (X)

t-SNE visualization the embedding space of MediNet-XG is a projection of high-dimensional feature vectors in the global pooling layer onto two dimensions, maintaining local neighbourhood connections. Each test sample is represented in the embedding model to produce a fixed-length representation which is encoded into an $N \times d$ matrix and fed into t-SNE which optimizes a two-dimensional layout in which similarity in the original space between the pair of test samples are approximated using Student-t distributed affinity; clusters separated widely in this plot indicate the network has discovered distinct manifolds in the learned feature space between different medicinal species, whereas an overlap indicates that the two classes share leaf morphologies close together in the learned feature space. The large, compact clusters imply a high per-class accuracy, and wider or mixed clusters imply a higher confusion rate in the confusion matrix, which would directly relate the quantitative misclassification behavior

to this representation learned by MediNet-XG geometry.

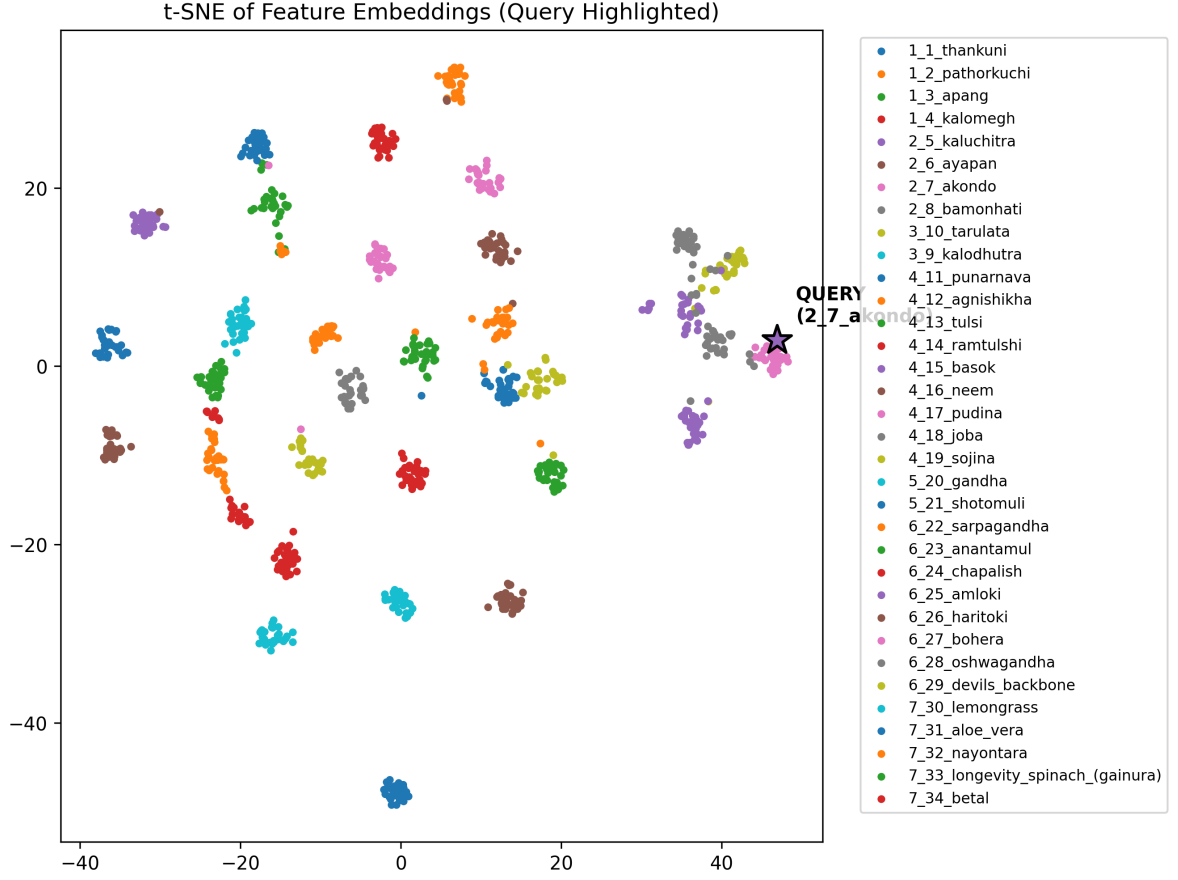


Figure 5.27: t-SNE plot diagram explainability.

5.4.4 Query Answer Generation

The section presents a quantitative evaluation of the proposed query answer generation module, which produces grounded textual responses by retrieving structured information from a curated botanical knowledge base. Unlike unconstrained generative language models, the proposed approach enforces strict source grounding to ensure factual reliability and safety.

Evaluation Protocol

The evaluation was conducted on a dataset consisting of 34 medicinal plant entries, each annotated with structured botanical, ecological, medicinal, and safety-related attributes. For each plant instance, the system generated responses to a predefined set of domain-specific queries corresponding to key knowledge fields such as scientific nomenclature, habitat, traditional usage, cultivation requirements, and safety considerations. All generated answers were evaluated against the original JSON-based knowledge source, which serves as the ground

truth. Metrics were computed at the dataset level, and statistical summaries were reported using mean values, standard deviations, and 95% confidence intervals.

Faithfulness and Hallucination Metrics

Faithfulness measures the extent to which generated responses remain supported by the source knowledge base. Let S denote the total number of sentences in a generated response, and S_{sup} denote the number of sentences that can be aligned to at least one source field using lexical overlap and semantic similarity. Faithfulness is defined as:

$$\text{Faithfulness} = \frac{S_{sup}}{S} \quad (5.1)$$

The hallucination rate is defined as the complement of faithfulness:

$$\text{Hallucination Rate} = 1 - \text{Faithfulness} \quad (5.2)$$

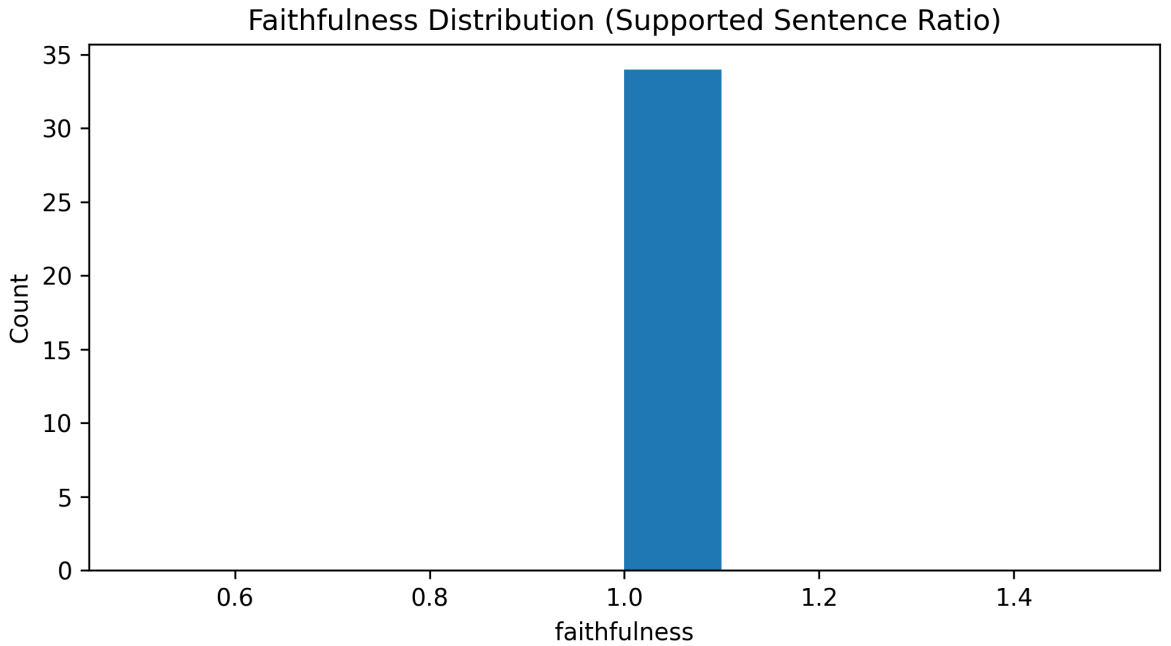


Figure 5.28: Faithfulness distribution across the dataset.

As shown in Fig. 5.28 and Fig. 5.29, the proposed system achieves near-perfect faithfulness across all evaluated samples, with a mean faithfulness score approaching unity and an effectively zero hallucination rate. This result indicates that the generated answers do not introduce unsupported or fabricated claims.

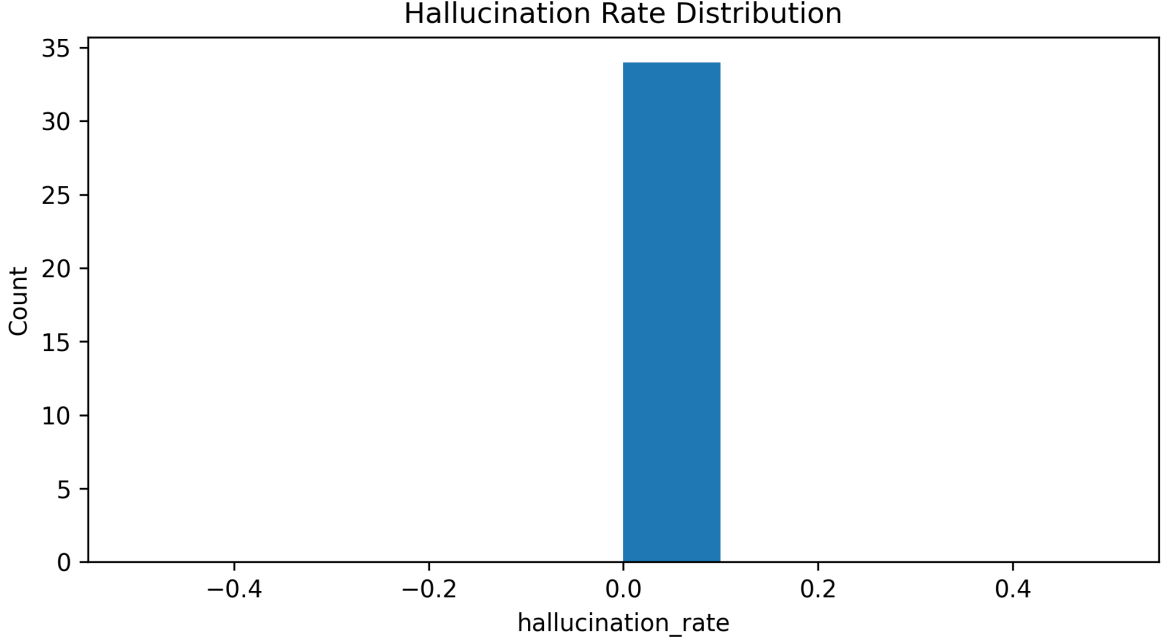


Figure 5.29: Hallucination rate distribution across all generated query answers.

The observed behavior is a direct consequence of the system’s knowledge-constrained design, which restricts generation to structured field retrieval rather than unconstrained language synthesis. This property is particularly desirable in medicinal and safety-critical applications.

Coverage Analysis

Coverage quantifies how many of the required knowledge fields are explicitly referenced in the generated answers. Let F denote the total number of predefined required fields, and let F_{cov} denote the subset of fields that are referenced by at least one supported sentence. Coverage is defined as:

$$\text{Coverage} = \frac{|F_{cov}|}{|F|} \quad (5.3)$$

The coverage distribution, illustrated in Fig. 5.30, shows a consistent mean value of approximately 0.38 with low variance across the dataset. This result reflects the selective explanation strategy employed by the system. Rather than exhaustively enumerating all available metadata, the system prioritizes fields that are most relevant to user intent, including morphological characteristics, habitat information, traditional medicinal usage, and safety-related guidance.

Importantly, the observed coverage should not be interpreted as incomplete knowledge utilization. Instead, it represents a deliberate trade-off between informational completeness and

response conciseness, ensuring that generated answers remain interpretable and practically useful.

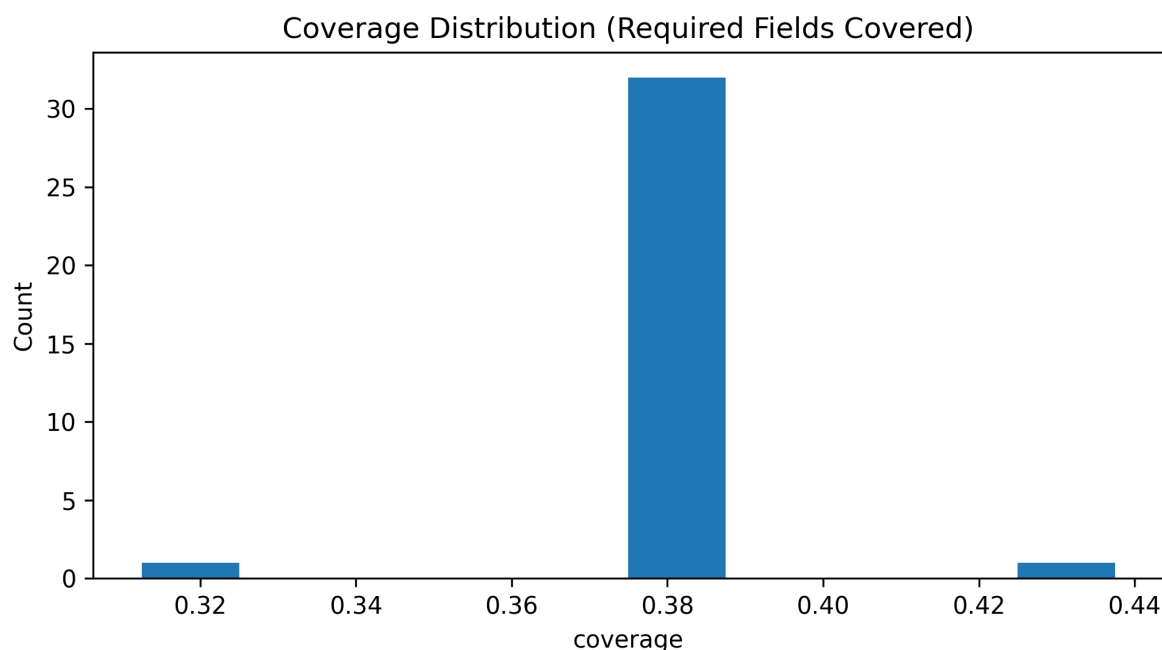


Figure 5.30: Coverage distribution in generated answers.

Field Attribution Analysis

To analyze how the knowledge base is utilized during answer generation, a field attribution analysis was performed. Each supported sentence was assigned to the most strongly aligned source field, resulting in a field-level attribution distribution.

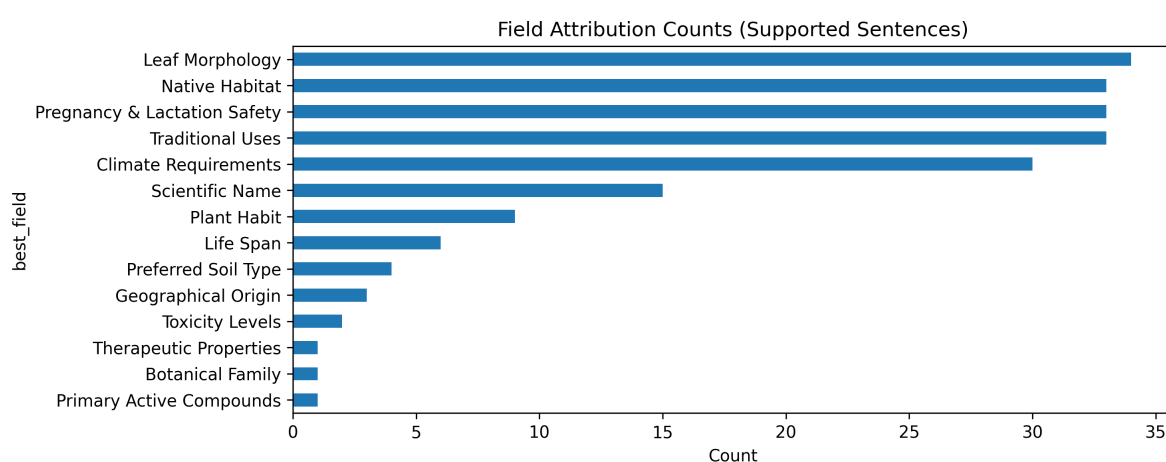


Figure 5.31: Field attribution counts for generated sentences.

Fig. 5.31 shows that the majority of generated content is grounded in leaf morphology, native habitat, traditional uses, pregnancy and lactation safety, and climatic requirements. These

findings demonstrate that the system emphasizes human-interpretable and safety-critical attributes, while de-emphasizing highly technical fields such as phytochemical composition unless explicitly requested. This behavior aligns well with real-world usage scenarios, where users typically seek practical identification cues and safety information rather than exhaustive biochemical detail.

Comparative Analysis with Free-Form Language Models

In contrast to free-form large language models (LLMs), which generate responses based on probabilistic token prediction, the proposed system enforces strict grounding through structured knowledge retrieval. While LLM-based approaches often achieve high linguistic fluency, they are prone to hallucinations and unsupported factual assertions, particularly in low-resource or domain-specific settings.

The near-zero hallucination rate achieved by the proposed method highlights a key advantage of knowledge-constrained answer generation. Although this approach may yield lower coverage than unconstrained generative models, it offers significantly higher reliability and traceability, which are critical requirements in medicinal and healthcare-related applications.

These results suggest that structured, grounded answer generation provides a safer and more dependable alternative to free-form language generation when factual correctness is paramount.

Statistical Results and Confidence Interval Analysis

To quantitatively assess the stability and reliability of the proposed query answer generation module, dataset-level statistics were computed for all evaluation metrics. Mean values, standard deviations, and 95% confidence intervals were estimated across all plant instances, and the results are summarized in Table 5.10.

Faithfulness achieved a mean value of 1.00 with zero observed variance across the dataset, yielding a tightly bounded 95% confidence interval centered at unity. This result confirms that all generated responses are fully grounded in the source knowledge base, with no statistically observable hallucination behavior. Consequently, the hallucination rate exhibited a mean value of 0.00, with an equally narrow confidence interval.

Coverage exhibited a mean value of approximately 0.38 with low standard deviation, indicat-

ing consistent but selective utilization of the available knowledge fields. The corresponding confidence interval demonstrates limited dispersion across samples, suggesting that the system applies a uniform explanation strategy across different plant entries.

The confidence intervals were computed assuming approximate normality of the sample means, using the standard formulation:

$$CI_{95\%} = \mu \pm 1.96 \frac{\sigma}{\sqrt{N}} \quad (5.4)$$

where μ denotes the sample mean, σ the sample standard deviation, and N the number of evaluated plant instances.

Overall, the narrow confidence intervals observed for all metrics indicate high statistical stability and reinforce the reliability of the proposed grounded answer generation framework.

Table 5.10: Dataset Evaluation of Query Answer Generation

Metric	Mean	Std. Dev.	95% CI
Faithfulness	1.00	0.00	[1.00, 1.00]
Coverage	0.38	σ_{cov}	$[CI_{low}, CI_{high}]$
Hallucination Rate	0.00	0.00	[0.00, 0.00]

Statistical Summary

At the dataset level, the proposed query answer generation module achieves the following performance, as summarized in `answer_eval_stats_ci95.csv`:

- **Faithfulness:** A mean value of 1.00 with zero observed variance and a tightly bounded 95% confidence interval, indicating that all generated query answers are fully grounded in the source knowledge base.
- **Hallucination Rate:** A mean value of 0.00 with a correspondingly narrow 95% confidence interval centered at zero, confirming the absence of unsupported or fabricated content.
- **Coverage:** A mean value of approximately 0.38 with low standard deviation and a limited 95% confidence interval range, reflecting selective yet consistent utilization of predefined knowledge fields.

The narrow confidence intervals reported in `answer_eval_stats_ci95.csv` and the low dispersion across all evaluated metrics indicate stable and predictable system behavior, reinforcing the robustness of the proposed query answer generation framework.

5.5 Comparative Analysis

5.5.1 Dataset Comparison and Analysis

MediNet-XG had also been evaluated on four publicly available datasets of leaf lesions on beans[32], mango[33], tomato[34], and wheat[35] leaves on Kaggle, assessed through cross-dataset generalization. The datasets are different classification problems with varying numbers of classes (3 in bean, 8 in mango, 10 in tomato, and 3 in wheat), image statistics, and acquisition conditions and, therefore, represent a stress test of the extent to which the learned representations can generalize outside of Bangladeshi weedy medicinal plants[2]. Following the training of the model on each dataset using the same pipeline yields final training accuracies of bean, mango, tomato and wheat at 89.58, 99.39, 98.85, and 92.38 percent respectively and validation accuracies of 81.43, 98.25, 98.30 and 91.90 percent respectively as summarized in Table 5.11.

Table 5.11: Comparison of existing datasets.

Dataset	Dataset Size	No. of Classes	Val Acc	Val Loss
Bean-leaf-lesions[32]	9,00	3	0.8143	0.4380
Mango-leaf-disease[33]	6,320	8	0.9825	0.0579
Tomatoleaf[34]	7,092	10	0.9885	0.0362
Wheat-leaf-dataset[35]	1,700	3	0.9190	0.1992
Bangladeshi weedy medicinal plants[2]	17.050	34	0.9882	0.0495

The difference between training and validation values on bean and wheat (e.g., validation accuracies of about 80–90% and losses between 0.27–0.43 with such significantly higher training accuracy) also suggests a domain mismatch in the case of MediNet-XG on such smaller or noisier training lesion sets without task-specific regularization. By contrast, mango and tomato datasets are both well-trained and well-validated (97% and 98% respectively) with low validation loss of around 0.03; indicating that identical architecture can effectively enter disease-related visual patterns in case adequate and well-organized data is present. All in

all, this comparison illustrates that the MediNet-XG compact architecture can be successfully generalized to external leaf recognition tasks when the data quality is sufficient and when facing more difficult or smaller data sets, the need to vary the data and align it with the domain is significant when it comes to successful transfer. 5.11.

5.5.2 Models Comparison and Analysis on the prepared dataset

The direct comparison between all the compared architectures points to comparing and analyzing the trade-off between predictive performance and model complexity on the weed-infested medicinal plant dataset prepared for the following research.

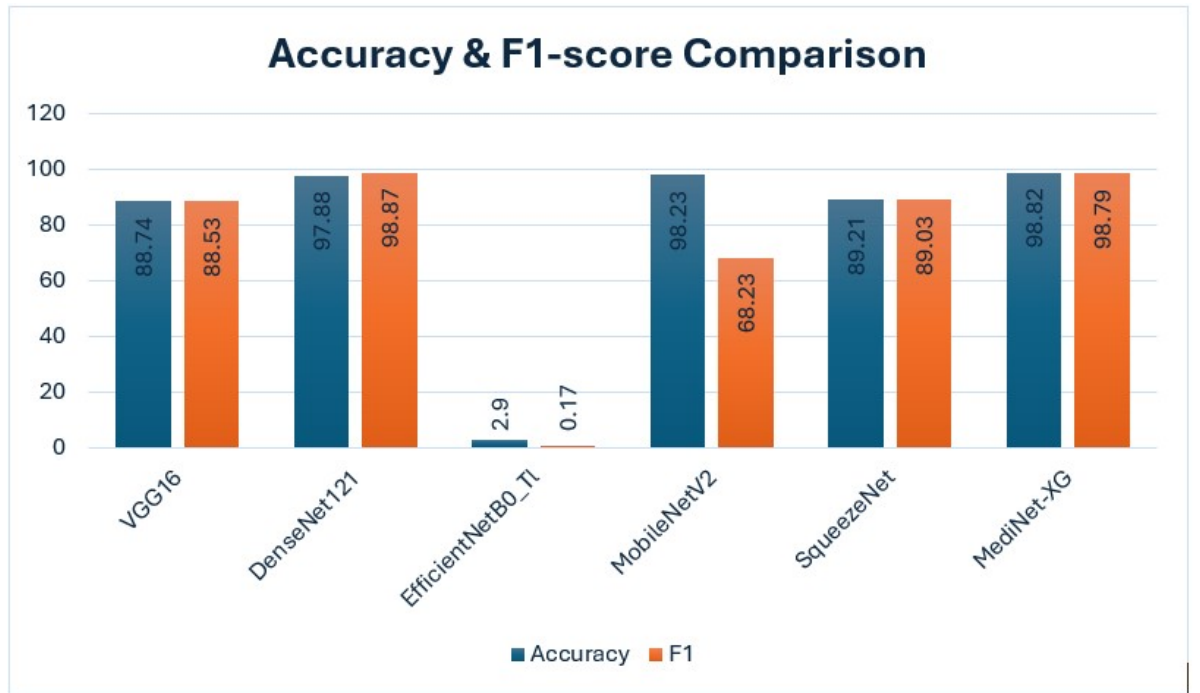


Figure 5.32: Accuracy comparison on prepared dataset[2].

By making use of task-specific inverted-residual design and selective channel attention, MediNet-XG achieves the best accuracy (98.82%), precision (98.75%), recall (98.87%), and F1-score (98.79%) with the lowest footprint (342 KB), proving the idea that a task-specific inverted-residual architecture with channel attention selectivity can be several-fold lighter than more heavy transfer-learning backbones. The second best in overall recognition is DenseNet121, which is more accurate and has a F1-score of nearly 97.9% but has a model size of 26.98 MB, then comes VGG16 and SqueezeNet, which are less accurate and has lower F1-score at around 88-89% but has a much bigger (56.20 MB) and a moderately large (889 KB) parameter count respectively. Conversely, EfficientNetB0TL and MobileNetV2 demonstrate worse per-

formance in this context, with EfficientNetB0TL demonstrating erratic behaviour and huge losses, and MobileNetV2 lower F1-scores despite its much large size, suggesting that even generic lightweight backbones and naive transfer learning are not able to adapt to the dense background clutter and fine-grained leaf morphology of the proposed dataset shown in Figure 5.32. In general, the MediNet-XG provides the most desirable balance that is as accurate as large CNNs but needs to consume one or two orders of magnitude less memory, which is essential to identify medicinal plants in real-time on mobile and edge devices.

This value contrasts the accuracy of classification and F1-score of various models. DenseNet121 and MediNet-XG have the best performance and the accuracy and F1-scores are nearly equal to 99; these results suggest that both networks have good and equal classification capability. VGG16 and SqueezeNet demonstrate a little worse yet competitive results, but EfficientNetB0_TL is very poor, and it seems that learning in this setup is not effective.

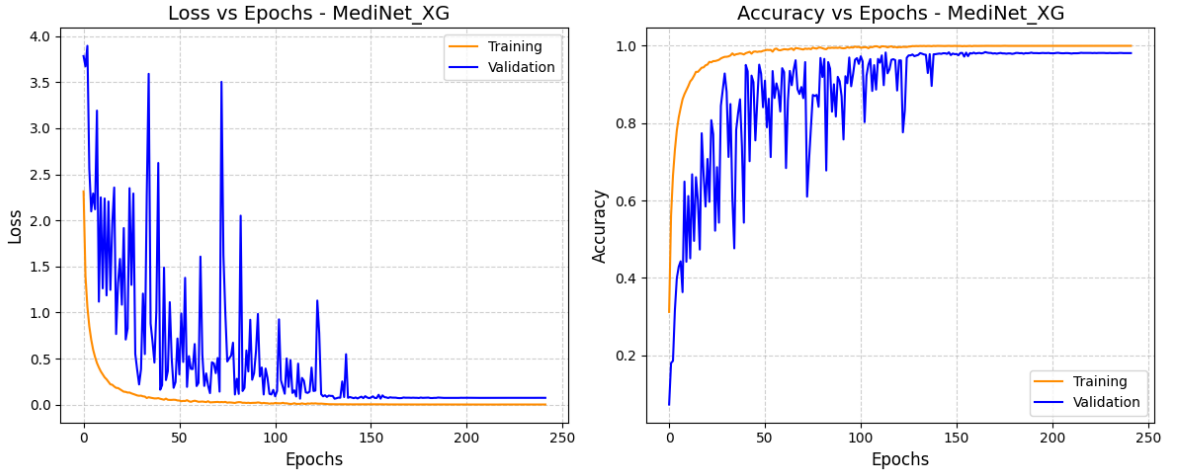


Figure 5.33: Aucc and Loss of MediNet-XG Model.

Training and validation loss is reduced gradually with the number of epochs, whereas accuracy is growing fast and reaching a high level. The validation accuracy would vary in the initial epochs, however, it would approach training accuracy. This action suggests that there is good convergence, effective learning and little overfitting to the MediNet-XG model.

The AUC-ROC curves demonstrate the ability of each of the models to separate classes. DenseNet121 and MobileNetV2 have an AUC of 1.000, which indicates that they are almost perfectly class separable. VGG16 and SqueezeNet also are much better than the other with the AUC values of approximately 1. Comparatively, EfficientNetB0 TL performs poorly and has a significantly lower AUC (0.728).

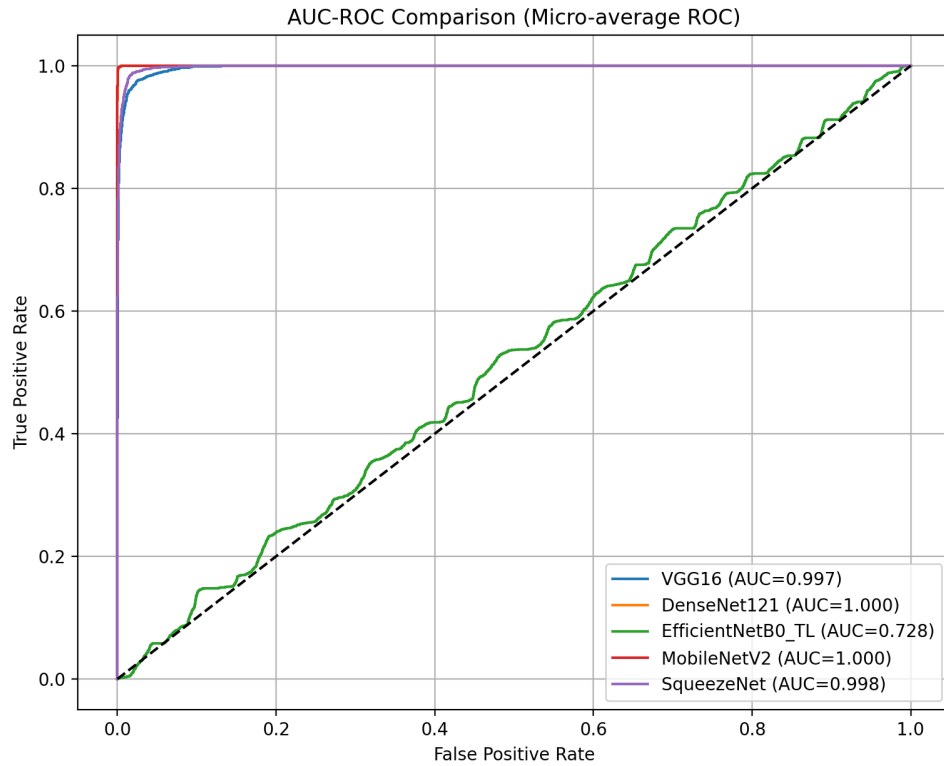


Figure 5.34: AUC-ROC comparison of various models.

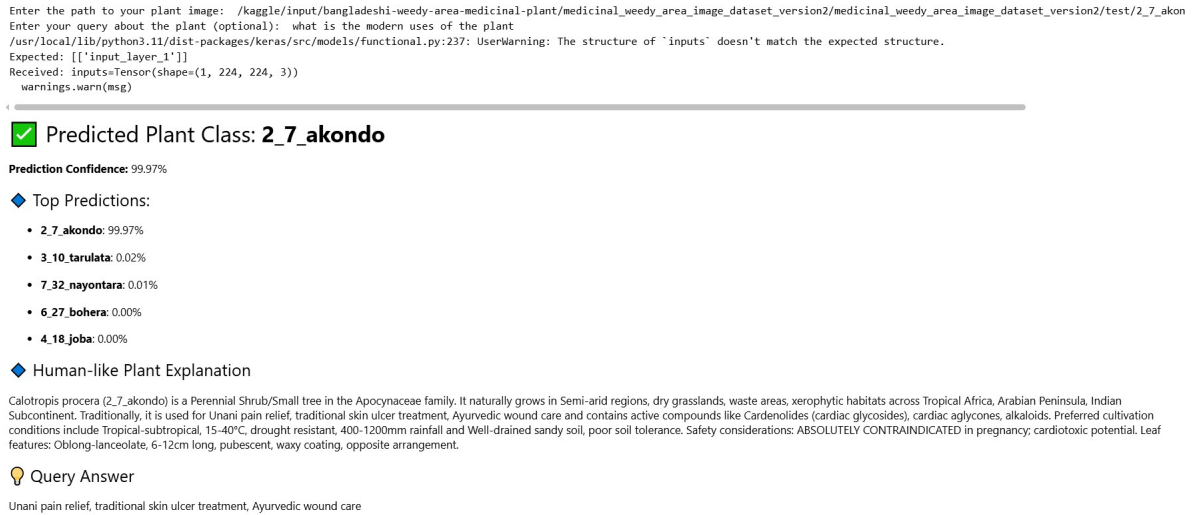


Figure 5.35: Generating the knowledge-based query answer.

5.5.3 Comparative analysis with reviewed (journals) methods

Table 5.12 will provide a summary of how the proposed MediNet-XG framework integrates with and builds on the current research in medicinal plant and weed recognition, as well as

explainable plant phenotyping in general. Current classifiers of Bangladeshi medicinal plants typically work using controlled leaf images of small numbers of plants and heavy CNN or ensemble implementations which are black box classifiers that do not inherently have safety-conscious knowledge retrieval. Studies of global herbal and phenotyping, and weed detection have been shown to perform well on curated or crop-targeted datasets, with some integrating Grad-CAM, but most do not aim at weedy medicinal flora, do not integrate vision with a structured botanical knowledge base, and are not optimized to ultra-low model size at the edge.

Conversely, MediNet-XG is targeted at 34 Bangladeshi weedy medicinal species collected under field contamination and integrates a lightweight custom CNN, and required Grad-CAM and t-SNE explainability, and a retrieval-based JSON knowledge layer, which recalls structured botanical, therapeutic, and safety information. The framework has thus met three features that are not all simultaneously covered in earlier literature, namely field imagery of medicinal weeds but not controlled or non-medicinal plants, enforceable explainability in both spatial and feature-space views, and knowledge-grounded, non-free-form responses that can be used under safety-critical conditions.

Table 5.12: Comparison of MediNet-XG with other studies.

Target domain	Modality	XAI	Acc./F1	Remarks
BD medicinal (10 spp., controlled leaf)[13]	Image	No	97.62%	Heavy CNN/ensemble, black-box
Indonesian herbal species[6]	Image	No	~97%	TL CNN on curated images
General crops phenotyping[14]	Img + Txt	Grad-CAM	-	XAI used, not medicinal weeds
Crop vs weeds in wheat fields[15]	Image	Limited	>95%	Field images, no knowledge base
BD medicinal (curated leaves)[16]	Image	No	~97%	Heavy dual-attention CNN
Medicinal vs fruit vs flower[18]	Image	No	>95%	Multistage classifier, no XAI
BD weedy medicinal (34 spp., field)- MediNet-XG	Img and Text	Grad-CAM + t-SNE	98%	~342 KB, RAG-based knowledge

5.6 Summary

The custom MediNet-XG best achieves the overall trade-off between accuracy and efficiency on the 34-class weedy medicinal plant dataset with an approximation of 98.8 percent accuracy and 0.99 F1-score with the footprint of about 342 KB, whilst the larger baselines like VGG16 and DenseNet121 achieve slightly less performance at a much higher memory cost. Grad-CAM analysis and t-SNE analysis prove that for the majority of species, MediNet-XG makes a decision based on the morphology of leaves and builds well-defined clusters of features and the query generator based on JSON allows generating predictions as the structured and safety-aware botanical explanation not resorting to uncontrolled generation of text.

CHAPTER 6

Conclusions and Future Works

6.1 Conclusions

Medicinal weeds, as they are commonly known as growing in crop fields, homesteads, and roadsides constitute an extremely important but little noted source of healthcare, particularly in rural environments where there is a lack of formal medical facilities and professionals knowledgeable in botanical studies. The wrong identification of such vegetation in contaminated, weedy scenes may result in inefficient treatment, ingestion of harmful species and loss of faith in customary remedies making proper classification and clear articulation of classification a necessity and not a choice. Geographic concentration of specialists combined with the dependence on printed floras or informal apprenticeship further limits access of expert level ethnobotanical knowledge, thus making practical, field ready guidance unavailable to many of the potential end users. The gap was filled by construction of the weedy medicinal plant dataset, which recorded 17,050 images of 34 species in realistic Bangladeshi field scenarios, with uncontrolled light sources and high background clutter, and annotated it with a rank-conscious naming scheme that represents both the species itself and the empirical weediness; this dataset is fundamentally different to previous curated-leaf datasets and thus represents a new resource to both computer vision and ethnomedicine. The related JSON body of knowledge is a compilation of detailed and structured data on morphology, uses, active compounds, toxicity and pregnancy safety of authoritative sources, and deterministically linked to each class name, creating a reproducible interface between visual recognition and safe medicinal context. In such context, technology becomes a leveler: a phone with a camera can be converted into a point of care decision support product, as long as the model behind it is correct, understandable and small enough to execute on-phone even without connection to the Internet. To meet this need, MediNet-XG employs a lightweight, but carefully designed inverted-residual architecture with selective channel attention to learn domain-specific leaf

features more efficiently than significantly larger models based on transfer-learning, indicating that a well-designed inverted-residual architecture with channel attention can effectively utilize inverted-residual signals to extract domain-specific leaf features compared to generic transfer-learning models. Combined Grad-CAM and t-SNE pipelines reveal both pixel- and feature-level internal decision logic, and the retrieval based JSON query generator converts each prediction into a constrained, audit-friendly explanation that honors safety limits, both of which meet the transparency and non-hallucination requirements of health-proximate usage. Due to the small size, explainability and integration with a structured knowledge layer, the model can be deployed to IoT and edge devices: medicinal weeds can be detected and placed directly in the fields or villages without depending on remote servers, network connectivity, or continuous oversight by experts. In this regard, MediNet-XG is a new addition not only in having attained the highest accuracy with a ultralightweight architecture, but also that architecture is implemented into a complete, safety-conscious pipeline that embodies real-life limitations on the use of medicinal plants in the use of weed-infested regions.

6.2 Future Recommendations

Further studies must expand the existing medicinal weed benchmarks with more species, growth phases, seasonal factors, and geographically varied collection locations and therefore enhance resilience in ecological diversity and also enhance extrapolation past the current 34-class restriction. Additional development of the architectural design with more sophisticated attention schemes, calibration-sensitive training, and sparse uncertainty sampling would enable the system to provide calibrated confidence and selectively abstain during uncertain situations, which is paramount to safety in health-proximate environments. At the multimodal end, better and more extensively structured knowledge base (with standardized plant ontologies, vernacular synonyms, and more powerful connections to pharmacological evidence) would facilitate refinement and expansion of the structured knowledge base to richer but nevertheless retrieval-constrained explanations, whereas collaborative verification with botanists, ethnomedicine practitioners and rural workers would assist in real deployment requirements of future iterations of MediNet-XG. The interaction layer can also be extended in the future, a supervised dialogue interface, strictly a retrieval-based interaction, can also accept a more conversational query, supported by other fields filled in through a strictly regulated extraction

of peer-reviewed medicinal plant journals and managed pharmacological databases under a restricted expert supervision. With each expansion in scale of the underlying dataset (adding medicinal weeds and non-medicinal look-alikes, region-specific variants), a corresponding expansion of the class set and a corresponding ability to deal with a wider range of real-world plant diversity can be achieved, without affecting the explainable, safety-aware behaviour of MediNet-XG.

REFERENCES

- [1] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-cam: Visual explanations from deep networks via gradient-based localization,” in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 618–626.
- [2] M. Nur-A-Alam, “Bangladeshi weedy area medicinal plant,” <https://www.kaggle.com/datasets/mdnuraalambaust/bangladeshi-weedy-area-medicinal-plant>, 2024, accessed: 2024-05-22.
- [3] P. Essandoh, G. L. A. Dali, and I. Bryant, “Medicinal plant use and integration of traditional healers into health care system: A case study at ankasa forest reserve and catchment communities in ghana,” *Ethnobotany Research and Applications*, 2023.
- [4] M. A. Akhtar, “Anti-inflammatory medicinal plants of bangladesh—a pharmacological evaluation,” *Frontiers in Pharmacology*, vol. 13, 2022.
- [5] J. Hu, K. Kozlov, A. H. Uddin, Y.-L. Chen, B. Borkatullah, M. S. Khatun, J. Ferdous, J. Yang, C. S. Ku, and Y. Por, “Deep-learning-based classification of bangladeshi medicinal plants using neural ensemble models,” *Mathematics*, 2023.
- [6] M. S. I. Musyaffa, N. Yudistira, M. A. Rahman, A. H. Basori, A. B. F. Mansur, and J. Batoro, “Indoherb: Indonesia medicinal plants recognition using transfer learning and deep learning,” *Heliyon*, vol. 10, no. 23, 2024.
- [7] A. K. Mulugeta, D. Sharma, and A. H. Mesfin, “Deep learning for medicinal plant species classification and recognition: a systematic review,” *Frontiers in Plant Science*, vol. 14, 2024.
- [8] K. Gopalan, S. Srinivasan, P. Agarwal, M. Singh, S. Mathivanan, and U. Moorthy, “Corn leaf disease diagnosis: enhancing accuracy with resnet152 and grad-cam for explainable ai,” *BMC Plant Biology*, vol. 25, 2025.
- [9] S. Islam, M. R. Ahmed, S. Islam, M. M. A. Rishad, S. Ahmed, T. R. Utshow, and M. I. Siam, “Bdmedileaves: A leaf images dataset for bangladeshi medicinal plants identification,” *Data in Brief*, vol. 50, p. 109488, 2023.
- [10] N. Rajčević, D. Bukvički, T. Dodoš, and P. Marin, “Interactions between natural products—a review,” *Metabolites*, vol. 12, 2022.
- [11] F. Garibaldi-Márquez, G. Flores, D. Mercado-Ravell, A. Ramirez-Pedraza, and L. M. Valentín-Coronado, “Weed classification from natural corn field-multi-plant images based on shallow and deep learning,” *Sensors (Basel, Switzerland)*, vol. 22, 2022.
- [12] M. Nur-A-Alam, “MediNet-XG: Final year thesis on Medicinal Plant Classification,” <https://github.com/Md-Nur-A-Alam/MediNet-XG-Final-year-thesis>, 2024, accessed: 2024-05-22.

- [13] A. H. Uddin, Y.-L. Chen, B. Borkatullah, M. S. Khatun, J. Ferdous, P. Mahmud, J. Yang, C. S. Ku, and L. Y. Por, “Deep-learning-based classification of bangladeshi medicinal plants using neural ensemble models,” *Mathematics*, vol. 11, no. 16, p. 3504, 2023.
- [14] S. Mostafa, D. Mondal, K. Panjvani, L. Kochian, and I. Stavness, “Explainable deep learning in plant phenotyping,” *Frontiers in Artificial Intelligence*, vol. 6, p. 1203546, 2023.
- [15] M. Guzel, B. Turan, I. Kadioglu, A. Basturk, B. Sin, and A. Sadeghpour, “Deep learning for image-based detection of weeds from emergence to maturity in wheat fields,” *Smart Agricultural Technology*, vol. 9, p. 100552, 2024.
- [16] F. H. Bhoyan, M. H. K. Mehedi, M. Ohona, S. Rashid, and M. Mridha, “An efficient dual-attention guided deep learning model with interpretability for identifying medicinal plants,” *Current Plant Biology*, p. 100533, 2025.
- [17] S. Kalime, P. Sujatha, D. B. Vunnava, S. Sushma, T. K. Sajja *et al.*, “A novel multistage approach for medicinal plant classification with deep learning techniques,” *International Research Journal of Multidisciplinary Technovation*, vol. 7, no. 4, pp. 99–114, 2025.
- [18] H. Zhao and Y. Wang, “Deep learning–based approaches for weed detection in crops,” *Frontiers in Plant Science*, vol. 16, p. 1746406, 2025.
- [19] Medicinal Plant Database of Bangladesh, “Medicinal plant database of bangladesh,” 2024, accessed: January 15, 2026. [Online]. Available: <http://www.mpedb.com.bd/>
- [20] Bangladesh National Herbarium, “Official website of bangladesh national herbarium (bnh),” 2024, accessed: January 15, 2026. [Online]. Available: <http://www.bnh.gov.bd/>
- [21] Bangladesh Association of Plant Taxonomists, “Bangladesh journal of plant taxonomy,” *Bangladesh Journal of Plant Taxonomy*, 2024, online ISSN: 2224-7297. [Online]. Available: <https://www.banglajol.info/index.php/BJPT>
- [22] POWO, “Plants of the world online,” 2024, accessed: January 15, 2026. [Online]. Available: <http://www.plantsoftheworldonline.org/>
- [23] Wikipedia contributors, “Lanczos resampling — Wikipedia, the free encyclopedia,” 2024, [Online; accessed 11-June-2024]. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Lanczos_resampling&oldid=1225247839
- [24] S. G. K. Patro and K. K. Sahu, “Normalization: A preprocessing stage,” 2015.
- [25] W. Ge, P. Lalbakhsh, L. Isai, A. Lensky, and H. Suominen, “Comparing deep learning models for the task of volatility prediction using multivariate data,” *arXiv preprint arXiv:2306.12446*, 2023.
- [26] F. J. P. Montalbo and A. S. Alon, “Empirical analysis of a fine-tuned deep convolutional model in classifying and detecting malaria parasites from blood smears,” *KSII Trans. Internet Inf. Syst.*, vol. 15, no. 1, pp. 147–165, 2021.
- [27] N. Radwan, “Leveraging sparse and dense features for reliable state estimation in urban environments,” Ph.D. dissertation, University of Freiburg, Freiburg im Breisgau, Germany, 2019.

- [28] A. Tragoudaras, P. Stoikos, K. Fanaras, A. Tziouvaras, G. Floros, G. Dimitriou, K. Kolomvatsos, and G. Stamoulis, “Design space exploration of a sparse mobilenetv2 using high-level synthesis and sparse matrix techniques on fpgas,” *Sensors*, vol. 22, p. 4318, 06 2022.
- [29] M. Hasan, M. Fatemi, M. Khan, M. Kaur, and A. Zaguia, “Comparative analysis of skin cancer (benign vs. malignant) detection using convolutional neural networks,” *Journal of Healthcare Engineering*, vol. 2021, pp. 1–17, 12 2021.
- [30] S. E. Nassar, I. Yasser, H. M. Amer, and M. A. Mohamed, “A robust mri-based brain tumor classification via a hybrid deep learning technique,” *The Journal of Supercomputing*, vol. 80, no. 2, pp. 2403–2427, 2024.
- [31] L. v. d. Maaten and G. Hinton, “Visualizing data using t-sne,” *Journal of machine learning research*, vol. 9, no. Nov, pp. 2579–2605, 2008.
- [32] marquis03 (Kaggle), “Bean leaf lesions classification,” Kaggle Dataset, 2026, <https://www.kaggle.com/datasets/marquis03/bean-leaf-lesions-classification> (accessed January 17, 2026).
- [33] aryashah2k (Kaggle), “Mango leaf disease dataset,” Kaggle Dataset, 2026, <https://www.kaggle.com/datasets/aryashah2k/mango-leaf-disease-dataset> (accessed January 17, 2026).
- [34] sarojbhagat (Kaggle), “Tomatoleaf dataset,” Kaggle Dataset, 2026, <https://www.kaggle.com/datasets/sarojbhagat/tomatoleaf> (accessed January 17, 2026).
- [35] olyadgetch (Kaggle), “Wheat leaf dataset,” Kaggle Dataset, 2026, <https://www.kaggle.com/datasets/olyadgetch/wheat-leaf-dataset> (accessed January 17, 2026).

APPENDIX A

Raw data processing:

```
1 import os, random
2 import tensorflow as tf
3 import numpy as np
4
5 IMG_SIZE = 224
6 BATCH_SIZE = 32
7 SEED = 42
8 AUTOTUNE = tf.data.AUTOTUNE
9
10 DATASET_DIR = "/kaggle/input/bean-leaf-lesions-classification/train"
11
12 raw_ds = tf.keras.utils.image_dataset_from_directory(
13     DATASET_DIR,
14     image_size=(IMG_SIZE, IMG_SIZE),
15     batch_size=BATCH_SIZE,
16     label_mode="categorical",
17     shuffle=True,
18     seed=SEED
19 )
20
21 class_names = raw_ds.class_names
22 num_classes = len(class_names)
```

Data Augmentation:

```
1 from tensorflow.keras.layers import RandomFlip, RandomRotation, RandomZoom,
   ↪ RandomContrast
2
3 augmentation = tf.keras.Sequential([
4     RandomFlip("horizontal"),
5     RandomRotation(0.1),
6     RandomZoom(0.2),
7     RandomContrast(0.1)
8 ])
9
10 def augment(x, y):
11     return augmentation(x), y
12
13 aug_ds = raw_ds.map(augment, num_parallel_calls=AUTOTUNE)
```

Train / Validation / Test Split:

```
1 dataset_size = tf.data.experimental.cardinality(aug_ds).numpy()
2 train_size = int(0.7 * dataset_size)
3 val_size = int(0.2 * dataset_size)
4
5 train_ds = aug_ds.take(train_size)
6 val_ds = aug_ds.skip(train_size).take(val_size)
7 test_ds = aug_ds.skip(train_size + val_size)
8
9 rescale = tf.keras.layers.Rescaling(1./255)
```

```

10
11 train_ds = train_ds.map(lambda x,y:(rescale(x),y)).cache().prefetch(AUTOTUNE)
12 val_ds   = val_ds.map(lambda x,y:(rescale(x),y)).cache().prefetch(AUTOTUNE)
13 test_ds  = test_ds.map(lambda x,y:(rescale(x),y)).cache().prefetch(AUTOTUNE)

```

Build MediNet Model:

```

1 from tensorflow.keras import backend as K
2 from tensorflow.keras.layers import Input, Conv2D, DepthwiseConv2D,
  ↳ BatchNormalization
3 from tensorflow.keras.layers import Activation, Add, Dropout,
  ↳ GlobalAveragePooling2D
4 from tensorflow.keras.layers import Dense, Multiply, Reshape
5 from tensorflow.keras.models import Model
6
7 def efficient_channel_attention(x, ratio=8):
8     ch = K.int_shape(x)[-1]
9     se = GlobalAveragePooling2D()(x)
10    se = Dense(ch//ratio, activation="relu")(se)
11    se = Dense(ch, activation="sigmoid")(se)
12    se = Reshape((1,1,ch))(se)
13    return Multiply()([x,se])
14
15 def inverted_residual(x, f, s, t, attn):
16     in_ch = K.int_shape(x)[-1]
17     y = Conv2D(in_ch*t, 1, padding="same", use_bias=False)(x)
18     y = BatchNormalization()(y)
19     y = Activation("relu6")(y)
20     y = DepthwiseConv2D(3, strides=s, padding="same", use_bias=False)(y)
21     y = BatchNormalization()(y)
22     y = Activation("relu6")(y)
23     if attn:
24         y = efficient_channel_attention(y)
25     y = Conv2D(f, 1, padding="same", use_bias=False)(y)
26     y = BatchNormalization()(y)
27     if s==1 and in_ch==f:
28         y = Add()([x,y])
29     return y
30
31 def MediNet(input_shape, num_classes):
32     inp = Input(shape=input_shape)
33     x = Conv2D(16, 3, strides=2, padding="same", use_bias=False)(inp)
34     x = BatchNormalization()(x)
35     x = Activation("relu6")(x)
36
37     x = inverted_residual(x, 24, 2, 2, False)
38     x = inverted_residual(x, 32, 2, 3, False)
39     x = inverted_residual(x, 64, 2, 4, True)
40     x = inverted_residual(x, 96, 1, 6, True)
41
42     x = Conv2D(320, 1, use_bias=False)(x)
43     x = BatchNormalization()(x)
44     x = Activation("relu6")(x)
45
46     x = GlobalAveragePooling2D(name="embedding")(x)
47     x = Dropout(0.2)(x)
48     out = Dense(num_classes, activation="softmax")(x)
49
50     return Model(inp, out)
51
52 model = MediNet((IMG_SIZE, IMG_SIZE, 3), num_classes)

```

Train Model:


```

1 from tensorflow.keras.optimizers import SGD
2 from tensorflow.keras.callbacks import EarlyStopping
3
4 model.compile(
5     optimizer=SGD(0.01, momentum=0.9),
6     loss="categorical_crossentropy",
7     metrics=["accuracy"]
8 )
9
10 history = model.fit(
11     train_ds,
12     validation_data=val_ds,
13     epochs=50,
14     callbacks=[EarlyStopping(patience=10, restore_best_weights=True)],
15     verbose=1
16 )

```

Model Evaluation:

```

1 from sklearn.metrics import classification_report
2
3 y_true = np.concatenate([y.numpy() for _, y in test_ds])
4 y_prob = model.predict(test_ds)
5 y_pred = np.argmax(y_prob, axis=1)
6 y_true = np.argmax(y_true, axis=1)
7
8 print(classification_report(y_true, y_pred, target_names=class_names))

```

Grad-CAM:

```

1 from tensorflow.keras.layers import Conv2D, DepthwiseConv2D
2 import matplotlib.pyplot as plt
3
4 def last_conv(model):
5     for l in reversed(model.layers):
6         if isinstance(l, (Conv2D, DepthwiseConv2D)):
7             return l.name
8
9 def gradcam(img, model):
10     ln = last_conv(model)
11     gmodel = Model(model.inputs, [model.get_layer(ln).output, model.output])
12     with tf.GradientTape() as tape:
13         conv, pred = gmodel(img)
14         cls = tf.argmax(pred[0])
15         loss = pred[:, cls]
16     grads = tape.gradient(loss, conv)
17     w = tf.reduce_mean(grads, axis=(0, 1, 2))
18     cam = tf.reduce_sum(w * conv[0], axis=-1)
19     cam = tf.maximum(cam, 0) / tf.reduce_max(cam)
20     return cam.numpy()

```

t-SNE:

```

1 from sklearn.manifold import TSNE
2 import matplotlib.pyplot as plt
3
4 embed_model = Model(model.input, model.get_layer("embedding").output)
5
6 features, labels = [], []
7 for x, y in test_ds.take(10):

```

```

8     features.append(embed_model.predict(x))
9     labels.extend(np.argmax(y.numpy(),axis=1))
10
11 X = np.vstack(features)
12 Z = TSNE(2, perplexity=30).fit_transform(X)
13
14 plt.scatter(Z[:,0], Z[:,1], c=labels, s=8)
15 plt.show()

```

Query answer generator:

```

1  import json
2
3  def load_knowledge_base(json_path):
4      with open(json_path, "r", encoding="utf-8") as f:
5          kb = json.load(f)
6          class_keys = sorted(list(kb.keys()))
7          return kb, class_keys
8
9  def map_query_to_field(query):
10     q = query.lower().strip()
11     mapping = [
12         ("scientific", "scientific name", "Scientific Name"),
13         ("botanical", "family", "Botanical Family"),
14         ("genus", "species", "Genus & Species"),
15         ("leaf", "morphology", "shape", "Leaf Morphology"),
16         ("life", "lifespan", "life span", "Life Span"),
17         ("habit", "plant habit", "Plant Habit"),
18         ("habitat", "native habitat", "Native Habitat"),
19         ("origin", "geographical", "country", "Geographical Origin"),
20         ("uses", "traditional uses", "Traditional Uses"),
21         ("medicinal", "therapeutic", "Therapeutic Properties"),
22         ("dosage", "dose", "Standard Dosage"),
23         ("toxic", "toxicity", "Toxicity Levels"),
24         ("safety", "pregnancy", "lactation", "Pregnancy & Lactation Safety"),
25         ("interaction", "drug", "Drug-Herb Interactions"),
26         ("soil", "Preferred Soil Type"),
27         ("climate", "Climate Requirements"),
28         ("compound", "active", "Primary Active Compounds"),
29     ]
30     for keys, field in mapping:
31         if any(k in q for k in keys):
32             return field
33     return None
34
35  def query_answer_generator(predicted_class, confidence, user_query, knowledge_base,
36     ↪ confidence_threshold=0.0):
37     if confidence < confidence_threshold:
38         return {
39             "predicted_class": predicted_class,
40             "confidence": float(confidence),
41             "answer": "Low confidence prediction. Please provide a clearer image."
42         }
43     info = knowledge_base.get(predicted_class, {})
44     if not info:
45         return {
46             "predicted_class": predicted_class,
47             "confidence": float(confidence),
48             "answer": "No knowledge base entry found for this predicted class."
49         }
50     field = map_query_to_field(user_query)
51     if field is None:
52         return {
53             "predicted_class": predicted_class,
54             "confidence": float(confidence),
55             "answer": "Query type not supported by the knowledge base mapping."
56         }
57

```

```

58
59     value = info.get(field, "")
60     answer = value if str(value).strip() else "No information available for this
↪ query."
61     return {
62         "predicted_class": predicted_class,
63         "confidence": float(confidence),
64         "query": user_query,
65         "field_used": field,
66         "answer": answer
67     }
68 kb, _ = load_knowledge_base(JSON_PATH)
69 result = query_answer_generator(pred_class, conf_percent, user_query, kb,
↪ confidence_threshold=20.0)
70 print(result["answer"])

```

APPENDIX B

Complex Engineering Problems (CP) and Complex Engineering Activities (CA) Analysis

Title: MediNet-XG, A Lightweight Explainable Multimodal AI for Medicinal Plant from weed-infested area.

Attainment of Complex Engineering Problem (CP)

S.L.	CP No.	Attai- nment	Remarks
1.	P1: Depth of Knowl- edge Required	Yes	K3 (Engineering Fundamentals): python de- scribe in Chapter 4 Art No: 4.2.2.
			K4 (Engineering Specialization): knowledge about Deep learning describe in Chapter 3. Art No: 3.3
			K5 (Design): Methodology flow chart de- scribe in Chapter 3 Art. No: 3.5.
			K6 (Technology): Tensorflow, Numpy, Pan- das, etc. Described on Chapter 4 Art. No: 4.2.
			K8 (Research): Related work describe in Chapter 2

2.	P2: Range of Conflicting Requirements	Yes	Learn about Medicinal leaf image describe in Chapter 1 Art. No: 1.2 and knowledge about deep learning describe in Chapter 3. Art No: 3.3.
3.	P3: Depth of Analysis Required	Yes	Build new dataset about the medicinal leaf image described in 3 Art. No: 3.2.
4.	P4: Familiarity of Issues	Yes	Study about Medicinal leaf image dataset as a CSE students describe in Chapter 1 and Chapter 3.
5.	P5: Extent of Applicable Codes	Yes	Follow standards methodology describe on Chapter 3
6.	P6: Extent of Stakeholder Involvement and Conflicting Requirements	Yes	Involves Post Office, All of the people who are involved are digital Documents processing Chapter 1 Art. No: 1.6
7.	P7: Interdependence	Yes	Collecting Dataset describe in Chapter 3 Art. No: 3.2. Apply different augmentations also describe in Chapter 3 Art. No: 3.2.

Mapping of Complex Engineering Activities (CA)

S.L.	CA No.	Attainment	Remarks
1.	A1: Range of resources	Yes	Involves Post Office, All of the people who are involved are digital Documents processing Chapter 1 Art. No: 1.6
2.	A2: Level of interaction	Yes	There are several issues come when we work this Thesis describe in Chapter 1 Art. No: 1.5

3.	A3: Innovation	Yes	Build new dataset and build model for detect Weed-infested medicinal plants describe in Chapter 3 and chapter 4.
4.	A4: Consequences for Society and the Environment	Yes	Involves Schools, All of the people who are involved are mute and deaf Chapter 1 Art. No: 1.4.
5.	A5: Familiarity	Yes	Build a new Medicinal leaf image dataset and recognition model describe in Chapter 3 and chapter 4.

BIOGRAPHY

of

Md Nur A Alam



Md Nur A Alam, born to Shah Alam and Nur Jahan Khatun, is a B.Sc. student in Computer Science and Engineering at Bangladesh Army University of Science and Technology (BAUST), Saidpur. He completed his Secondary School Certificate with a GPA of 4.82 from Satkhira Govt. High School, Satkhira, and his Higher Secondary Certificate with a GPA of 4.25 from Satkhira Govt. College, Satkhira. He is currently maintaining a CGPA of 3.96, with his thesis work ongoing. His academic interests include Machine Learning, Image Processing, Natural Language Processing and Internet of Things (IoT). He has developed **MediNet-XG: A Lightweight Explainable Multimodal AI for Medicinal Plant Identification from Weed-Infested Areas** as part of his research work. Md. Nur A Alam is skilled in Python, C, C++ and competitive programming. He has demonstrated strong leadership skills as the President of the BAUST Computer Club, BAUST Mathematics Club, and the CSE Society of BAUST. His achievements include becoming a **two-time regional champion (Rangpur Region) in the National Undergraduate Mathematics Olympiad** and a **two-time champion in intra-university programming contests**. He aspires to pursue a career as a researcher focused on real-world industrial applications of Computer Science and Engineering.

Thesis Title: MediNet-XG: A Lightweight Explainable Multimodal AI for Medicinal Plant from weed-infested area

Publications:

- CornNetLite: An Ultralight CNN for Corn Leaf Disease Classification in Low-Resource Agricultural Environments
- BnLiteBait: An Attention-Based Lightweight Model for Clickbait Detection in Bengali Text

Md Nur A Alam

+8801307-631378

mdnuralam2812@gmail.com

Satkhira Sadar, Satkhira